TOOLKIT · BACKEND ENGINEERING HANDBOOK

AWS COMPLETE HANDBOOK

Cloud Foundations, IAM, VPC Networking, EC2 Compute, Load Balancing, Auto Scaling, S3, RDS PostgreSQL, Containers (ECS/Fargate), Serverless (Lambda), Messaging (SQS/SNS/EventBridge), CI/CD & Production Reliability

Phase	IV — Container Orchestration & Cloud Infrastructure
Tool ID	14 of 14 (Cloud Engineering Capstone Platform)
Curriculum Scope	Complete 6-Chunk Comprehensive Cloud Curriculum
Deliverable Type	Consolidated Tool Engineering Handbook (Release Build)
Source Authority	content/aws/ (01 through 06 authoritative markdown sources)
Target Audience	Backend Software Engineers, SDE-1 / SDE-2, DevOps Engineers, Cloud Infrastructure Architects
Engineering Focus	Cloud Foundations, Least Privilege IAM, Multi-AZ VPC Networking, Stateless EC2/ECS Compute, Managed RDS Relational Storage, Event-Driven Decoupling, Golden Signals, CI/CD, and Incident Runbooks

This authoritative handbook provides an exhaustive, production-grade guide to **Amazon Web Services (AWS)** for backend software engineers and cloud architects. Spanning six comprehensive pedagogical chunks across three rigorous layers—Mental Models, Practical Workflows, and FAANG-Level Incident Mastery—it transitions engineers from deploying code on local machines to operating highly available, secure, and cost-effective distributed systems. Readers master cloud foundations, the shared responsibility model, least-privilege IAM roles and instance profiles, multi-AZ VPC network topologies, route tables, Internet & NAT Gateways, stateful Security Groups vs stateless NACLs, compute scaling with EC2 Auto Scaling Groups & Application Load Balancers, object storage with S3 and direct presigned uploads, managed databases with Amazon RDS PostgreSQL (Multi-AZ failover, read replicas, point-in-time recovery), container orchestration with Amazon ECS on AWS Fargate, serverless compute with AWS Lambda, event-driven decoupling with SQS, SNS, and EventBridge, resolving the dual-write problem via the Transactional Outbox pattern, observability with CloudWatch Golden Signals, CloudTrail auditing, automated CI/CD pipelines, and comprehensive real-world incident simulations.

Curriculum Structure & Core Topics (6 Chunks)

Part I — Cloud Foundations, IAM & Backend Architecture: Cloud computing rationale, shared responsibility model, AWS global infrastructure (Regions, AZs, Edge Locations), IAM identities (Users, Groups, Roles, Policies), least privilege, and designing a Spring Boot backend.

Part II — VPC, Subnets, Routing, Gateways & Security Groups: Virtual Private Clouds, CIDR arithmetic, public vs private subnets, route tables and local routes, Internet Gateways, NAT Gateways, stateful Security Groups vs stateless NACLs, and request tracing.

Part III — EC2, Load Balancers, Auto Scaling & Compute: Virtual servers, AMIs, instance types, EBS volumes, instance metadata (IMDSv2), Application Load Balancers (Layer 7) vs NLBs, Auto Scaling Groups (min/desired/max), health checks, and graceful draining.

Part IV — Storage, Managed Databases, Backups & High Availability: S3 object storage (presigned URLs, lifecycle, versioning), EBS block storage, EFS network filesystem, Amazon RDS PostgreSQL, Multi-AZ failover vs read replicas, PITR backups, RPO, and RTO.

Part V — Containers, Serverless, Messaging & Event-Driven Architecture: ECS on Fargate, AWS Lambda execution limits, SQS queue lifecycle, SNS pub/sub fan-out, EventBridge event routing, Transactional Outbox pattern, consumer idempotency, and DLQs.

Part VI — Production Security, Observability, CI/CD & Final Capstone: Workload identity, Secrets Manager vs SSM, CloudWatch Golden Signals, CloudTrail audit trails, automated GitHub Actions pipelines, SLIs/SLOs/error budgets, cost engineering, and end-to-end Capstone.

Part I: Cloud Foundations, IAM & Backend Architecture

Chunk 1 of 6 · Backend Engineering Handbook · Cloud Engineering Track

AWS — Chunk 1 of 6: Cloud Foundations, IAM & Your First Backend Architecture

Learning Objectives

By the end of this chapter, you should be able to:

- Explain cloud computing and why engineering teams use it.
- Distinguish regions, Availability Zones, and edge locations.
- Understand the shared responsibility model.
- Navigate the basic AWS account and resource model.
- Explain IAM users, groups, roles, policies, and temporary credentials.
- Apply least-privilege access to a backend application.
- Design a first AWS architecture for a Java Spring Boot API.
- Reason about cloud costs, security incidents, and availability failures.

Prerequisites: Basic HTTP, REST APIs, Java, databases, and familiarity with deploying an application locally. No cloud experience is required.

Layer 1 — Fundamentals & Mental Models

1. What Is Cloud Computing?

Imagine you build a Spring Boot API on your MacBook. It works locally, but your laptop is not a production hosting platform. If you close it, the API disappears from the internet. You must also manage networking, power, operating-system updates, and availability yourself.

Now imagine renting computing resources from a provider that operates large data centers and exposes them through APIs and a management console.

That is the basic idea behind cloud computing.

Cloud providers offer computing, storage, networking, databases, identity, monitoring, and other capabilities as services that you can provision when needed.

AWS stands for Amazon Web Services.

Instead of purchasing a physical server before you can deploy your application, you can provision a virtual server or a managed application service and pay according to the service's pricing model.

The core mental model

Think of AWS as a collection of programmable infrastructure building blocks.

- **Compute** runs code.
- **Storage** keeps data or files.
- **Networking** controls how systems communicate.
- **Databases** store and retrieve structured or specialized data.
- **Identity and access management** controls who can do what.
- **Monitoring** helps you understand whether systems are working.
- **Managed services** let AWS operate some of the underlying infrastructure for you.

You choose the components, configure them, connect them, and decide how the application should behave when things fail.

2. Why Companies Use AWS

Cloud infrastructure is useful for several reasons.

2.1 Elasticity

Suppose your API normally receives 100 requests per second but experiences 2,000 requests per second during a major event.

Elastic infrastructure can add or remove capacity according to demand, provided that the application and its scaling configuration support it.

Without elasticity, you might have to purchase capacity for peak traffic and leave much of it unused during normal periods.

Important: Cloud elasticity is not automatic for every resource. Some services require explicit scaling policies, and databases or stateful components can have different scaling constraints.

2.2 Pay-as-you-go pricing

Instead of purchasing every server upfront, many cloud services charge based on consumption, allocated capacity, requests, storage, or data transfer.

This reduces some upfront investment but introduces a new engineering responsibility: controlling usage and understanding the bill.

A badly configured cloud system can cost more than an appropriately sized physical server.

2.3 Managed infrastructure

Compare two database approaches:

- Install PostgreSQL on a virtual machine and manage its updates, backups, recovery, and operating system.
- Use a managed database service such as Amazon RDS, where AWS operates many underlying database-management tasks.

Managed services reduce operational burden, but they do not eliminate your responsibility for schema design, access control, query performance, backup strategy, or application correctness.

2.4 Global reach

AWS provides infrastructure in multiple geographic regions. You can select regions appropriate for your users, compliance requirements, latency targets, and availability strategy.

A globally distributed architecture can improve user experience, but it also introduces complexity involving replication, consistency, networking, cost, and failure recovery.

3. Cloud Service Models: IaaS, PaaS, and SaaS

These categories describe how much of the computing stack you operate yourself.

Model	Description	AWS Examples
IaaS — Infrastructure as a Service	You provision infrastructure such as virtual machines and networking, while managing much of the operating system and application stack.	Amazon EC2, Amazon VPC, Amazon EBS
PaaS — Platform as a Service	The platform abstracts more infrastructure and operational tasks so you can focus on application delivery. AWS services offer different levels of platform abstraction; not every managed service fits neatly into a single textbook category.	AWS Elastic Beanstalk, AWS App Runner, Amazon RDS
SaaS — Software as a Service	You consume a finished application instead of provisioning the underlying infrastructure. Examples include hosted email, collaboration, and CRM applications.	Amazon WorkDocs, SaaS partner applications

An important interview distinction

These categories are useful mental models, not strict labels that perfectly describe every cloud product.

For example, a managed database still requires database engineering, while a container platform may abstract servers but leave networking, identity, scaling, and application reliability to you.

The practical question is:

Which responsibilities remain yours, and which responsibilities does the provider operate?

That question becomes especially important during incidents.

4. AWS Global Infrastructure

Three concepts matter early: regions, Availability Zones, and edge locations.



4.1 Regions

A region is a separate geographic area containing AWS infrastructure.

You choose a region when creating many AWS resources.

For example, a service deployed in one region does not automatically become available in another region.

Region selection can affect:

- Latency for your users.
- Service availability.
- Data residency and compliance.
- Price.
- Disaster recovery options.
- Cross-region data-transfer costs.

4.2 Availability Zones

An Availability Zone, or AZ, is an isolated location within an AWS region, designed to reduce the impact of certain failures affecting other zones.

A region contains multiple Availability Zones.

For higher availability, a backend might run application instances in multiple AZs and use a load balancer to distribute requests.

However, multiple AZs do not automatically make every part of an application highly available. A single-AZ database, a single external dependency, or an application-level bottleneck can still cause an outage.

4.3 Edge locations

AWS operates edge infrastructure used by services such as Amazon CloudFront to deliver content closer to users.

For example, an API's static documentation or images may be delivered through a content delivery network. Dynamic API requests may still need to reach an origin or backend service.

Do not confuse edge locations with regions or Availability Zones.

Quick comparison

Concept	Main purpose	Example use
Region	Geographic deployment boundary	Host a backend near its users
Availability Zone	Isolation and resilience within a region	Spread application capacity across zones
Edge location	Deliver supported content and services near users	Cache static content through CloudFront

4.4 Multi-AZ vs. Multi-Region

A multi-AZ architecture improves resilience against failures affecting an individual zone.

A multi-region architecture can help with broader regional outages and geographic distribution, but it is substantially more complex.

Multi-region designs must consider:

- How data is replicated.
- Whether writes can occur in multiple locations.
- Consistency and conflict handling.
- DNS or traffic failover.
- Recovery time and data-loss objectives.
- Cross-region transfer costs.

For a fresher's first backend deployment, a well-designed single-region architecture is a sensible starting point. Learn multi-AZ architecture next, then study multi-region recovery as an advanced design decision.

5. The AWS Resource Model

A useful way to understand AWS is to distinguish an account, a region, and an individual resource.

Account

An AWS account provides an administrative and resource boundary. It is also important for billing and access control.

Region

Many resources are created in a selected region. Some AWS capabilities are global or account-level instead, so do not assume every resource is regional.

Resource

A resource is an individual cloud object, such as an EC2 instance, an S3 bucket, or an RDS database.

A simplified mental model:

```
AWS organization
■■■ AWS account
    ■■■ Region A
    ■   ■■■ Virtual network
    ■   ■■■ Application compute
    ■   ■■■ Database
    ■■■ Region B
        ■■■ Disaster-recovery resources
```

This is illustrative, not a mandatory architecture.

In a larger company, separate accounts may be used for development, staging, production, security, or shared infrastructure. AWS Organizations can help manage multiple accounts.

Resource names and identifiers

Cloud resources may have names, identifiers, and Amazon Resource Names (ARNs).

An ARN identifies an AWS resource in a structured form. The exact format depends on the service.

For example, IAM policies may use ARNs to specify which resources an action applies to.

Do not assume every resource supports the same naming rules or global uniqueness requirements.

6. The Shared Responsibility Model

One of the most important cloud concepts is the division of responsibility between the provider and the customer.

AWS is responsible for the security of the cloud, including the underlying infrastructure it operates. Customers are responsible for security in the cloud, with the precise boundary depending on the service.

Example: EC2

When you run an application on an EC2 virtual machine, you generally have responsibilities for:

- Operating-system configuration and updates.
- Application security.
- Application dependencies.
- Access control.
- Network rules you configure.
- Protecting credentials and data.

AWS operates the underlying physical infrastructure and virtualization layer.

Example: Managed database

With a managed database, AWS handles more of the underlying infrastructure and operational work, but you still need to consider:

- Database permissions.
- Network access.
- Encryption configuration.
- Data classification.
- Query performance.

- Backup retention and recovery requirements.
- Application-level data correctness.

Interview takeaway: Moving a workload to the cloud changes the operational boundary; it does not remove security and reliability responsibilities.

7. What Is IAM?

IAM means Identity and Access Management.

IAM controls which identities can perform which actions on which AWS resources.

Imagine your Spring Boot API needs to upload an invoice to an S3 bucket. The application should be allowed to upload to the intended bucket, but it should not automatically receive permission to delete every bucket in your AWS account.

IAM expresses those permissions.

The key concepts are:

- **Principal:** An identity that makes a request.
- **Authentication:** Establishing which identity is making the request.
- **Authorization:** Determining whether that identity may perform the requested action.
- **Policy:** A document describing allowed or denied actions and their scope.
- **Role:** An identity that can be assumed to obtain a set of permissions, often through temporary credentials.

Authentication and authorization are separate. A user can authenticate successfully but still lack permission to perform an operation.

8. IAM Users, Groups, Roles, and Policies

IAM user

An IAM user is an identity within an AWS account. It can have credentials and permissions assigned directly or through group membership.

Long-lived IAM user access keys require careful management. Avoid creating permanent access keys for applications when a role-based credential mechanism is available.

IAM group

An IAM group helps organize IAM users and attach shared permissions to them.

Groups are useful for organizing human access, but they are not a general mechanism for assigning application workloads their own identity.

IAM role

An IAM role has permissions that an authorized principal can assume. It provides temporary credentials when assumed.

Roles are central to AWS workload identity, cross-account access, and temporary elevated access.

For example:

- An EC2 instance can use an instance profile to obtain credentials associated with an IAM role.
- A supported container workload can obtain credentials through its configured task or pod identity mechanism.
- A CI/CD system can assume a role to deploy a service.

IAM policy

A policy specifies permissions. A simplified identity-based policy might look like this:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::my-invoices-bucket/invoices/*"
    }
  ]
}
```

This illustrates permission to upload objects to a particular object path. It is not a complete policy for every S3 operation or application.

The bucket name and path must be replaced with real, intended values.

How to read the policy

- **`Effect`**: whether the statement allows or denies an operation.
- **`Action`**: which API actions it matches.
- **`Resource`**: which resources the statement applies to.

Other policy elements can include conditions and principal-related information, depending on the policy type.

Important: AWS authorization can involve multiple policy types. An explicit deny overrides an applicable allow. The final decision depends on the complete authorization context, including resource policies, permissions boundaries, session policies, and organization-level controls where applicable.

9. Least Privilege

Least privilege means granting only the permissions needed for a task, at the narrowest practical scope.

Consider two policies for an API that uploads invoices.

Overly broad example:

```
{
  "Effect": "Allow",
  "Action": "s3:*",
  "Resource": "*"
}
```

Narrower example:

```
{
  "Effect": "Allow",
  "Action": "s3:PutObject",
  "Resource": "arn:aws:s3:::my-invoices-bucket/invoices/*"
}
```

The second policy is narrower, assuming the application needs only that operation and object path.

Real applications may also need additional permissions for encryption, reading objects, or multipart uploads. Add permissions based on actual requirements, not by default.

Why this matters

If an application credential is compromised, the permissions attached to that identity influence what an attacker can do.

Least privilege limits the potential blast radius.

It is not enough to protect the credential itself; the credential must also have appropriately restricted permissions.

Layer 2 — Practical Engineering & Workflows

10. Your First AWS Backend Architecture

Suppose we want to deploy a Spring Boot Orders API backed by PostgreSQL.

The architecture could evolve like this:

```
                  Internet
                     |
                     v
              Public HTTPS entry
                     |
                     v
                Load Balancer
                     |
          ---------------------
          |                   |
          v                   v
    API instance        API instance
          |                   |
          ---------+-----------
                   |
                   v
             Managed PostgreSQL
```

This is a conceptual design. A real deployment needs VPC networking, subnet placement, security groups, application identity, secrets management, database configuration, logging, monitoring, and a deployment workflow.

For the first lesson, focus on the responsibilities of the components.

Component responsibilities

Load balancer: Distributes incoming requests across healthy targets when correctly configured.

Application instances: Run the Spring Boot application.

Database: Stores durable application state.

IAM role: Grants the application or deployment workflow only the AWS permissions it needs.

Monitoring: Collects signals needed to understand application and infrastructure behavior.

Network controls: Restrict which components can communicate.

A critical design rule

Your application should not need unrestricted database access from the public internet.

A common design places the database in private subnets and restricts database access to the intended application security group or other narrowly scoped network path.

Network isolation and IAM permissions are separate controls. A correct IAM policy does not replace network restrictions, and a private subnet does not replace database authentication.

11. Create an AWS Learning Environment Safely

You can learn the concepts before deploying paid infrastructure.

Step 1 — Secure your account

If you create an AWS account:

1. Enable multi-factor authentication for the root user.
2. Do not use the root user for routine engineering tasks.
3. Use an appropriately secured identity for administration.
4. Create only the permissions needed for your labs.
5. Enable suitable billing visibility and budget notifications.
6. Review the current AWS Free Tier and pricing conditions before creating resources.

A budget notification is useful, but it should not be treated as a guaranteed hard spending cap. Some costs can be incurred before an alert is delivered.

Step 2 — Choose a region deliberately

For learning, select a region that makes sense for your location, service availability, and cost. Use the same region consistently unless a lab specifically teaches regional differences.

Step 3 — Install the AWS CLI

The AWS Command Line Interface lets you manage supported AWS services from your terminal.

Check whether it is installed:

```
aws --version
```

If it is missing, install AWS CLI v2 using the official AWS installation instructions for your operating system.

Step 4 — Configure credentials securely

For human access, prefer an AWS-supported identity and credential flow appropriate to your environment, such as IAM Identity Center or another approved role-based setup.

Then inspect the active identity:

```
aws sts get-caller-identity
```

This command reports the AWS account and identity associated with the credentials being used.

Avoid pasting access keys into source files, shell scripts, chat messages, or Git commits.

Step 5 — Understand profiles and regions

You may use named CLI profiles to separate identities and environments.

For example:

```
aws configure list-profiles
```

A named profile can be used with commands through `--profile`, provided that the profile has been configured.

Always verify the active account and region before running commands that create, modify, or delete resources.

12. First Practical Lab: Inspect AWS Identity

This lab is intentionally read-only.

Run:

```
aws sts get-caller-identity
```

Inspect the output:

- Which AWS account is active?
- Which principal is making the request?
- Are you using the intended learning or development identity?

Then run:

```
aws configure list
```

This helps inspect which configuration and credential sources the CLI is using. Sensitive values may be masked, but never share credential output publicly.

Troubleshooting

Error: Unable to locate credentials

Possible causes:

- No supported credential source is configured.

- The selected profile is missing.
- The environment is not configured for role-based credentials.

Error: AccessDenied

Possible causes:

- The identity lacks the required permission.
- A permission boundary or organization-level policy restricts the operation.
- A role trust policy or session policy prevents the requested access.

Error: ExpiredToken

Possible causes:

- Temporary credentials expired.
- The credential session needs to be refreshed.
- The application or CLI is using stale credentials.

The right fix depends on the identity flow. Do not solve credential problems by granting administrator access indiscriminately.

13. Application Credentials: The Correct Pattern

Suppose your API needs to upload an invoice to S3.

Avoid hardcoding credentials:

```
// Never embed real AWS credentials in application source code.
String accessKey = "AKIA...";
String secretKey = "secret...";
```

Instead, use the AWS SDK's supported credential provider chain or a specifically configured workload identity mechanism.

For example, with the AWS SDK for Java v2, the default credentials provider can resolve credentials from supported sources:

```
import software.amazon.awssdk.auth.credentials.DefaultCredentialsProvider;
import software.amazon.awssdk.services.s3.S3Client;

public class S3ClientFactory {

    public static S3Client create() {
        return S3Client.builder()
                .credentialsProvider(
                    DefaultCredentialsProvider.create()
                )
                .build();
    }
}
```

This is a basic illustration. The SDK dependency and region configuration must also be supplied by your project.

For production, configure the workload's identity using an appropriate AWS-supported mechanism, such as an EC2 instance profile or an ECS task role, depending on where the application runs.

The exact credential source depends on the runtime environment.

Why temporary credentials are preferable

Temporary credentials:

- Expire automatically.
- Can be refreshed by supported credential providers.
- Reduce reliance on long-lived access keys.
- Allow permissions to be tied to a workload identity.

They still need appropriate permissions and secure delivery. Temporary credentials are not inherently safe if an attacker can retrieve them from a compromised workload.

14. Your First IAM Design Exercise

Imagine an Orders API with two responsibilities:

1. Read a configuration object from a designated S3 path.
2. Upload invoice files to a designated S3 path.

Before writing a policy, answer:

- Which bucket does the API need?
- Which object prefixes does it need?
- Does it need to list the bucket?
- Does it need to read existing objects?
- Does it need to delete objects?
- Is access to other buckets necessary?

Build the policy from these requirements.

Do not begin with `s3:*` and attempt to remove permissions later. Begin with the smallest permission set and add permissions only when a verified requirement demands them.

Human access versus workload access

Separate the permissions of the engineer deploying the application from the permissions of the running application.

A deployment identity might need permission to update a service. The application identity might need permission to upload invoices. Those responsibilities should not automatically share one unrestricted credential.

This separation makes incidents easier to contain and permissions easier to review.

Layer 3 — Production Engineering & Interview Mastery

15. Production Security Principles

15.1 Protect the root user

The root user has broad account-level capabilities and should not be used for ordinary development tasks.

Protect it with MFA and reserve it for operations that specifically require root-user access.

15.2 Avoid long-lived access keys where possible

Prefer short-lived, role-based credentials for supported human and workload environments.

If long-lived keys are genuinely required, establish a secure storage, rotation, monitoring, and revocation process.

Never commit credentials to Git.

15.3 Restrict network exposure

A security group is a virtual firewall controlling traffic to and from associated resources.

For a typical API architecture:

- Public ingress should expose only the intended entry point.
- Application instances should accept traffic only from intended sources.
- Database access should be restricted to the application or other approved clients.
- Administrative access should use a controlled management path.

Security groups are stateful. Network ACLs are a separate network control with different behavior. Both should be understood before designing complex network rules.

15.4 Use encryption thoughtfully

Consider encryption:

- In transit, using TLS where appropriate.
- At rest, using supported encryption mechanisms for storage and databases.
- For secrets, using a dedicated secrets-management approach.
- For backups, ensuring that the protection and access policies match the sensitivity of the data.

Encryption does not replace access control, monitoring, or key-management practices.

16. Availability and Failure Reasoning

Cloud architecture is not just about selecting services. It is about understanding which failures a design can tolerate.

Scenario A — An application instance fails

If the application runs on a single instance, a failure may make the API unavailable.

A multi-instance design behind a correctly configured load balancer can continue serving traffic if enough healthy capacity remains.

But this requires:

- More than one healthy instance.
- Appropriate health checks.
- Capacity to handle remaining traffic.
- A deployment and recovery process.

- No single shared dependency that defeats the design.

Scenario B — The database becomes unavailable

Two healthy API instances cannot solve a database outage if both depend on the same unavailable database.

The application may need:

- Sensible connection timeouts.
- Bounded retries with backoff.
- Connection-pool limits.
- Graceful degradation where appropriate.
- A recovery and backup strategy.
- Clear observability for database errors and latency.

Do not blindly retry writes. A request may have committed before the connection failed, leaving the client uncertain about the outcome. Idempotency and reconciliation may be necessary.

Scenario C — An entire Availability Zone fails

A multi-AZ application design may continue operating if application capacity and required dependencies are available in surviving zones.

However, failover behavior depends on the architecture and service configuration. A zonal dependency, insufficient remaining capacity, or a failed network path may still cause an outage.

Scenario D — An AWS region becomes unavailable

A single-region application may be unavailable during a severe regional failure.

Multi-region recovery can reduce that risk, but requires explicit design for data replication, routing, failover, consistency, and recovery procedures.

For most early projects, understand the trade-off before implementing a multi-region architecture. Complexity has a cost.

17. Cost Engineering for Beginners

Cloud costs are an engineering concern, not merely a finance concern.

Potential cost sources include:

- Running compute resources.
- Allocated database capacity.
- Storage.
- Requests and data processing.
- Data transfer.
- Load balancers and other supporting services.
- Monitoring, logs, and retained data.

Cost controls for learning

1. Create a budget and configure alerts.
2. Choose the smallest suitable resources for a lab.
3. Check whether services have hourly or usage-based charges.
4. Review the cost of public IPv4 addresses, networking, and data transfer where applicable.
5. Stop or delete resources when no longer needed.
6. Verify what remains after cleanup, including storage, snapshots, addresses, and other persistent resources.

Important: Stopping an instance does not necessarily stop all related charges. For example, attached storage can continue to incur costs. Some services have additional charges even when they are not receiving requests.

Free-tier eligibility and limits change over time. Check the current terms rather than assuming a service is free.

18. Incident Simulation: The API Suddenly Returns AccessDenied

Situation: Your Spring Boot API previously uploaded invoices successfully. After a deployment, uploads start failing with an AWS authorization error.

Step 1 — Establish the impact

Determine:

- Which endpoint fails?
- Does every request fail, or only uploads?
- Did the issue begin after a deployment or configuration change?
- Are reads and writes affected differently?

Step 2 — Verify the active identity

Confirm which role or credential source the application is actually using.

A common mistake is assuming the deployed workload uses the same credentials as a developer's laptop.

Step 3 — Inspect the permission scope

Check:

- The action requested.
- The bucket and object path.
- The role's identity-based policies.
- Applicable bucket policies.
- Permissions boundaries or organization-level restrictions.
- Relevant encryption-key permissions, if the operation uses a customer-managed key.

Step 4 — Inspect recent changes

Compare:

- Role assignments.
- Policy changes.
- Bucket or key policies.
- Environment configuration.

- Deployment identity and runtime environment.

Step 5 — Remediate safely

Restore only the required permission or correct the identity configuration.

Avoid granting unrestricted access to make the error disappear.

Step 6 — Prevent recurrence

Add deployment checks, permission tests where feasible, clear application error logging, and documented ownership of the IAM role.

The goal is not just to restore service. It is to preserve least privilege while preventing the same incident.

19. Incident Simulation: Unexpected Cloud Spending

Situation: Your learning project has generated a much larger bill than expected.

Investigate:

1. Which account and region incurred the charges?
2. Which services and resources contributed to the cost?
3. Were resources left running after a lab?
4. Did data transfer, storage, or request volume exceed expectations?
5. Were credentials compromised or used unexpectedly?
6. Are cost and usage reports available for the relevant period?

If compromise is suspected, follow the organization's security process to contain the incident, investigate access, and rotate or revoke credentials where necessary.

Prevent recurrence with budgets, cost visibility, resource ownership tags where appropriate, cleanup procedures, and carefully scoped access.

Do not rely on budget notifications alone as a security or hard-spending control.

20. FAANG-Style Interview Questions

Q1. What is AWS?

AWS is a cloud platform providing computing, storage, networking, databases, identity, monitoring, and other services that can be provisioned and managed programmatically.

Q2. Why might a company choose cloud infrastructure over purchasing physical servers?

Cloud can reduce upfront infrastructure investment, provide managed services, and enable elastic provisioning. It also introduces usage-based costs and requires cloud-specific security and operational practices.

Q3. What is the difference between a region and an Availability Zone?

A region is a geographic deployment area. An Availability Zone is an isolated location within a region. Multi-AZ designs can improve resilience against certain zonal failures.

Q4. Does deploying in multiple Availability Zones guarantee high availability?

No. Availability depends on the full architecture, including application capacity, health checks, data stores, network paths, and shared dependencies.

Q5. What is the shared responsibility model?

AWS operates and secures the underlying cloud infrastructure, while customers retain responsibilities for security in the cloud. The exact division depends on the service.

Q6. What is IAM?

Identity and Access Management controls which identities can perform which actions on which AWS resources.

Q7. What is the difference between authentication and authorization?

Authentication establishes identity. Authorization determines which operations that identity is permitted to perform.

Q8. What is an IAM role?

An IAM role is an identity with associated permissions that an authorized principal can assume, typically receiving temporary credentials.

Q9. Why use IAM roles for applications instead of embedding access keys?

Role-based credential mechanisms can provide temporary, renewable credentials and avoid hardcoding long-lived secrets in application source code.

Q10. What is least privilege?

Granting only the permissions necessary for a task, scoped as narrowly as practical.

Q11. What is an explicit deny?

An applicable explicit deny in AWS authorization policy evaluation overrides an applicable allow.

Q12. Why is a public database dangerous?

It increases the database's exposure to network attacks and unauthorized access attempts. Restricting network access, using strong authentication, and applying least privilege reduces risk.

Q13. Does a private subnet automatically secure a database?

No. Network placement is one control. Security groups, routing, database authentication, authorization, encryption, and monitoring also matter.

Q14. What happens if an EC2 instance fails?

The instance stops serving requests. Recovery or replacement depends on the configured deployment and orchestration approach. A load balancer can route traffic to healthy targets if sufficient capacity remains.

Q15. Why might two healthy application instances still fail?

They may depend on a common failed database, shared network path, external service, or other single point of failure.

Q16. What is elasticity?

The ability to adjust resources to match demand. It requires appropriate service capabilities and configuration; it is not automatic for every workload.

Q17. What is the difference between a managed database and a database on EC2?

A managed database service operates more of the underlying database infrastructure and maintenance. A database on EC2 gives you more direct control but also more operational responsibility.

Q18. Why is multi-region deployment harder than multi-AZ deployment?

Multi-region designs introduce additional challenges involving data replication, consistency, routing, failover, recovery, and cross-region costs.

Q19. Why can cloud costs increase unexpectedly?

Unneeded resources, overprovisioning, data transfer, retained storage, high request volume, and unexpected usage can all increase costs.

Q20. How would you deploy a secure Spring Boot API on AWS?

Choose an appropriate compute and networking design, expose only the required entry point, restrict database access, use workload roles and least privilege, store secrets appropriately, enable monitoring, plan recovery, and track costs.

Capstone — Design Your First AWS Backend

Scenario

You are deploying an Orders API built with Spring Boot and PostgreSQL.

The API must:

- Accept HTTPS requests.
- Store order data durably.
- Upload invoices to S3.

- Avoid embedding AWS credentials in code.
- Limit database network exposure.
- Provide useful logs and metrics.
- Remain recoverable after a component failure.

Task 1 — Draw the architecture

Include:

- A public entry point.
- Application compute.
- PostgreSQL.
- An S3 bucket.
- An application IAM role.
- Network boundaries.
- Monitoring and logging.

Task 2 — Define access

Write down:

- What the engineer deploying the API needs to do.
- What the running API needs to do.
- What the database needs to accept.
- Which permissions should not be granted.

Task 3 — Analyze failures

Explain what happens when:

- An application instance fails.
- PostgreSQL becomes unavailable.
- S3 permissions are removed.
- An Availability Zone becomes unavailable.
- The AWS region becomes unavailable.

Task 4 — Estimate operational responsibilities

For each component, list:

- What AWS operates.
- What your team configures.
- What your team must monitor.
- What your team must recover.

Task 5 — Define the minimum viable production baseline

Write a checklist for identity, network access, secrets, logs, metrics, backups, and cost controls.

No paid deployment is required to complete the design exercise.

Mastery Assessment

Before moving to Chunk 2, answer these questions in your own words:

1. Why does cloud computing not eliminate infrastructure engineering?
2. Why is a multi-AZ design usually a different first step from a multi-region design?
3. What is the difference between an IAM user and an IAM role?
4. Why should an API not receive `s3:*` permissions by default?
5. Why are application identity and network isolation separate security controls?
6. What can still fail when two application instances are healthy?
7. Why is a cloud budget notification not a guaranteed spending cap?
8. How would you investigate an `AccessDenied` error without granting administrator access?

Completion Checklist

- [ ] I can explain cloud computing and its trade-offs.
- [ ] I understand regions, Availability Zones, and edge locations.
- [ ] I can explain the shared responsibility model.
- [ ] I understand AWS accounts and resources.
- [ ] I can distinguish IAM users, groups, roles, and policies.
- [ ] I understand least privilege and temporary credentials.
- [ ] I can sketch a basic AWS backend architecture.
- [ ] I can explain basic availability and cost risks.
- [ ] I can reason through an authorization incident.

Handbook Completion Protocol

Save this chapter as:

`content/aws/01-aws-cloud-foundations-iam-backend-architecture.md`

This is **Chunk 1 of 6**, not a standalone final handbook. Preserve the source as Markdown and do not generate a chunk PDF.

After all six AWS chapters are saved and verified, combine them into one complete AWS handbook:

`docs/aws/BEH_AWS_COMPLETE_HANDBOOK.pdf`

Update the main README only after verifying the source and final PDF. Commit only when the complete AWS source content and final PDF are verified, using the required commit message:

`AWS done`

No repository operation, PDF generation, or Git commit is implied by this chapter.

Part II: VPC, Subnets, Routing, Internet Gateways & Security Groups

Chunk 2 of 6 · Backend Engineering Handbook · Cloud Engineering Track

AWS — Chunk 2 of 6: VPC, Subnets, Routing, Internet Gateways & Security Groups

Backend Engineering Tools & Infrastructure Handbook · Cloud Engineering Track

Source file: `content/aws/02-aws-vpc-subnets-routing-gateways-security-groups.md`

In Chunk 1, we designed a Spring Boot API and PostgreSQL database. Now we'll answer the question that makes cloud deployment click:

How does an HTTP request travel from a user's browser to an application running inside AWS — and how do we stop unauthorized traffic from reaching it?

We'll build the networking concepts from scratch, then apply them to a realistic backend architecture.

Learning Roadmap

1. **Fundamentals & mental models:** VPCs, CIDR ranges, subnets, public vs. private networking, routing, and network boundaries.
2. **Practical engineering:** Build a network design for a Spring Boot API, load balancer, and PostgreSQL database. Understand routes, security groups, and request flow.
3. **Production & interview mastery:** Debug unreachable services, design least-exposure networks, reason about NAT costs, and handle connectivity incidents.

Learning Objectives

By the end of this chapter, you should be able to:

- Explain how an AWS Virtual Private Cloud works.
- Understand IPv4 addresses, CIDR notation, and subnet ranges.
- Distinguish public and private subnets.
- Explain route tables, the local route, Internet Gateways, and NAT Gateways.
- Understand security groups and Network ACLs.
- Design a network for a Spring Boot API and PostgreSQL database.
- Trace an incoming HTTPS request and an outgoing connection.
- Diagnose common connectivity failures using a systematic workflow.

Prerequisites: No prior cloud networking experience is required.

Layer 1 — Fundamentals & Mental Models

1. What Is a VPC?

VPC stands for **Virtual Private Cloud**.

An AWS VPC is a logically isolated virtual network in which you can launch and connect supported AWS resources.

Imagine an office building:

- The **VPC** is the building's overall network boundary.
- **Subnets** are network segments within the building.
- **Route tables** are directions that tell traffic where to go.
- **Gateways** connect the network to other networks.
- **Security groups** act like stateful firewalls attached to resources.
- **Network ACLs** provide another layer of subnet-level traffic control.

This analogy is not a complete description of AWS networking, but it gives you the basic mental model.

Why does a backend need a VPC?

Suppose you're deploying an Orders API with PostgreSQL.

You want customers to reach the API over HTTPS. However, you don't want arbitrary internet clients to connect directly to the database.

A VPC lets you organize resources into network segments and control their communication paths.

A typical architecture looks like this:



This is a conceptual architecture. In a production design, the subnets would typically be distributed across multiple Availability Zones for resilience, and routing and security rules would be configured separately for each component.

Core principle: Network placement and network permissions should reflect the intended communication paths between components.

2. IP Addresses and CIDR Notation

Before configuring a VPC, understand how IP address ranges work.

An IPv4 address contains 32 bits and is commonly written as four decimal numbers.

Example:

```
10.0.1.25
```

CIDR notation specifies a range of IP addresses.

Example:

```
10.0.0.0/16
```

The `/16` indicates that the first 16 bits identify the network portion of the range. The remaining bits identify addresses within that range.

For a conventional IPv4 CIDR block:

> **Formula:** Address count = 2^{32 - prefix length}

For `10.0.0.0/16`:

Formula: 2^{32 - 16} = 65,536

That is the total number of addresses in the IPv4 range, not the number of addresses available to workloads in an AWS subnet.

Common CIDR examples

CIDR	Total IPv4 addresses	Typical interpretation
`10.0.0.0/16`	65,536	Large private network range
`10.0.1.0/24`	256	Smaller subnet range
`10.0.1.0/28`	16	Small subnet range

AWS reserves five IPv4 addresses in each IPv4 subnet. For example, a `/24` subnet has 251 addresses available for use by resources, before accounting for any additional service-specific constraints.

A practical subnet layout

Suppose we allocate `10.0.0.0/16` to a VPC.

We could divide it into smaller ranges:

```
VPC: 10.0.0.0/16
|
+-- Public subnet A: 10.0.1.0/24
+-- Public subnet B: 10.0.2.0/24
|
+-- Private app A:   10.0.11.0/24
+-- Private app B:   10.0.12.0/24
|
+-- Private DB A:    10.0.21.0/24
+-- Private DB B:    10.0.22.0/24
```

These are example ranges, not a required AWS layout.

Planning address ranges matters because overlapping networks can complicate routing, particularly when connecting VPCs, corporate networks, or other environments.

Interview insight: CIDR planning is not merely an IP-address exercise. It affects future network connectivity and the ability to expand the architecture.

3. What Is a Subnet?

A subnet is a range of IP addresses within a VPC.

In AWS, each subnet is associated with one Availability Zone.

For example, a subnet might exist in Availability Zone A while another subnet exists in Availability Zone B.

You can place resources in different subnets to separate network paths and distribute infrastructure across zones.

Why use multiple subnets?

Consider three categories:

1. Public-facing entry points.
2. Private application workloads.
3. Private data services.

Separating them helps you establish distinct routing and security policies.

However, subnet names are just labels. Naming a subnet `private-subnet` does not make it private.

Its actual behavior depends on the associated routing configuration and other networking controls.

4. Public vs. Private Subnets

This distinction is fundamental.

Public subnet

A subnet is commonly described as public when its associated route table provides a route to an Internet Gateway for internet-bound IPv4 traffic.

For a resource to communicate directly with the public IPv4 internet, it also needs an appropriate public IPv4 address and suitable security rules.

A route to an Internet Gateway alone does not automatically make every resource in the subnet publicly reachable.

Private subnet

A private subnet typically does not have a direct route to an Internet Gateway for general internet-bound IPv4 traffic.

Resources in a private subnet can still communicate with other permitted resources inside the VPC. They may also access external services through a NAT Gateway or other explicitly configured network path.

A private subnet does not mean the resources are isolated from every network. It means their routing and access paths are more restricted.

Compare the two

Property	Public subnet	Private subnet
Typical direct internet route	Via Internet Gateway	No direct general-purpose internet route
Public IPv4 address	May be assigned	Usually not assigned
Common workloads	Public load balancer	Application servers and databases
Internet access	Depends on routes, addressing, and security rules	May use NAT or private endpoints
Is it automatically secure?	No	No

Important: A public subnet does not necessarily expose every resource. Exposure depends on addressing, routing, security groups, network ACLs, and service configuration.

5. Route Tables: Where Should Traffic Go?

A route table contains rules that determine where network traffic should be directed.

A route has a destination network and a target.

For example:

Destination	Target
`10.0.0.0/16`	`local`
`0.0.0.0/0`	`Internet Gateway`

The first route represents communication within the VPC's local IPv4 network range.

The second route sends IPv4 traffic whose destination does not match a more specific route toward the Internet Gateway.

This is a simplified example of a public subnet's route table.

The local route

A VPC route table includes a local route that supports communication within the VPC's associated network ranges.

You do not normally add a separate Internet Gateway route for every resource in the VPC.

The default route

`0.0.0.0/0` represents all IPv4 destinations.

A default route provides a path for destinations not matched by a more specific route.

But the target matters:

- `0.0.0.0/0` → `Internet Gateway` can provide an internet route from a public subnet.
- `0.0.0.0/0` → `NAT Gateway` can provide outbound internet access for workloads in a private subnet, subject to the network configuration.

AWS uses the most specific matching route when multiple routes match a destination. Route selection can also depend on the route target and route type.

A simple example

Your API instance has an address in `10.0.11.0/24`. It wants to connect to another resource at `10.0.21.15`.

If the destination is within the VPC's local address range, the local route can direct the traffic within the VPC.

If the API instead wants to reach an external IPv4 destination, its subnet's route table needs a suitable route to an external network path.

A route table provides a path, not permission. Security groups and network ACLs may still block the connection.

6. Internet Gateway

An Internet Gateway is a VPC component that enables communication between the VPC and the internet when routing, addressing, and security configuration allow it.

Think of it as the connection point between a VPC and the public internet.

For a typical inbound IPv4 connection to an EC2 instance, you need:

1. An Internet Gateway attached to the VPC.
2. A suitable route to the Internet Gateway.
3. An appropriate public IPv4 address or other valid public addressing configuration.
4. Security rules that permit the intended traffic.
5. An application or service listening on the relevant port.

If any required part is missing, the connection can fail.

What an Internet Gateway does not do

It does not:

- Automatically assign a public IP address to every instance.
- Automatically permit inbound traffic.
- Replace security groups.
- Replace application authentication.
- Guarantee that the application is listening on its intended port.

Interview takeaway: A network route and an access permission are different things.

7. NAT Gateway

A NAT Gateway can provide outbound internet connectivity for resources in private subnets without requiring those resources to have public IPv4 addresses.

Imagine a private application server that needs to download an external dependency or call a third-party API.

The application server sends traffic toward a NAT Gateway. The NAT Gateway translates the source address for eligible outbound IPv4 connections and uses its own public connectivity.

The return traffic can flow back through the NAT Gateway for established connections.

A common arrangement is:

```
Private API instance
        |
        v
Private subnet route table
        |
        v
NAT Gateway in public subnet
        |
        v
Internet Gateway
        |
        v
External API
```

NAT Gateway versus Internet Gateway

- An **Internet Gateway** supports internet connectivity for appropriately configured resources in a VPC.
- A **NAT Gateway** provides a controlled outbound connectivity path for private IPv4 workloads.

- A NAT Gateway does not automatically permit unsolicited inbound internet connections to the private instances using it.

Production considerations

NAT Gateways incur charges. Costs can include gateway usage and data processing, and applicable data-transfer charges may also apply.

Availability and routing design matter too. For example, placing a single NAT Gateway in one Availability Zone can create a dependency on that zone for workloads in other zones and may result in cross-zone data-transfer charges.

For production workloads, evaluate whether per-AZ NAT Gateways or another egress architecture best meets reliability and cost requirements.

For learning, understand the design before provisioning resources.

8. Security Groups: Stateful Network Firewalls

A security group controls inbound and outbound traffic associated with supported AWS resources.

Security groups are stateful.

That means return traffic for an allowed connection is automatically permitted by the security group's state tracking; you do not need to add a separate rule solely for the return packets of that connection.

Security groups are commonly used to express which components may communicate.

Example: Three security groups

```
Internet
    |
    | HTTPS (port 443)
    v
[Load Balancer SG]
    |
    | Application port (port 8080)
    v
[Application SG]
    |
    | PostgreSQL port (port 5432)
    v
[Database SG]
```

The intended policy might be:

- **Load balancer security group:** allow inbound HTTPS from intended public clients.
- **Application security group:** allow inbound application traffic from the load balancer security group.
- **Database security group:** allow inbound PostgreSQL traffic from the application security group.

This expresses the architecture more safely than opening every port to every source.

Example rules

Security group	Inbound source	Port	Purpose
Load balancer SG	Intended public clients	443	HTTPS
Application SG	Load balancer SG	8080	Spring Boot API
Database SG	Application SG	5432	PostgreSQL

The table describes the intended access rules, not a complete deployable configuration.

The application port could differ depending on the load balancer's target configuration and the application's actual listener.

Why reference another security group?

Instead of allowing all IPv4 addresses to connect to PostgreSQL, a database security group can permit the application security group as the source.

This makes the permission express a relationship between components.

It is generally easier to reason about and maintain than a broad source range.

Security group rules are not application authentication

A security group can permit a TCP connection to the API. It does not prove that the requester is an authorized application user.

You still need application-level authentication and authorization.

Network access control, IAM, database permissions, and application authentication solve different problems.

9. Network ACLs

A Network Access Control List, or NACL, is associated with a subnet and controls inbound and outbound traffic at that subnet boundary.

Unlike security groups, network ACLs are stateless.

For a connection to succeed, both the forward and return traffic must be permitted by the relevant NACL rules.

NACL rules are evaluated in rule-number order, and the first matching rule determines the outcome.

A NACL can provide an additional network-control layer, but it also introduces another place where connectivity can be blocked.

Security groups vs. NACLs

Property	Security group	Network ACL
Scope	Associated with network interfaces/resources	Associated with subnets
Stateful?	Yes	No
Rules	Allow rules	Allow and deny rules
Rule evaluation	Matching allowed traffic is permitted unless other controls block it	Rules evaluated in order

Property	Security group	Network ACL
Common use	Restrict communication between components	Additional subnet-level traffic filtering

For most beginner backend designs, learn to express the main component access relationships through security groups first. Introduce NACLs when a specific subnet-level control requirement justifies them.

10. The Full Request Path

Let's trace a user request to a Spring Boot API.

Assume the architecture has a public Application Load Balancer and private application instances.

```
1. Client resolves the API hostname
        |
        v
2. Client opens HTTPS connection
        |
        v
3. Public load balancer receives traffic
        |
        v
4. Load balancer forwards to a healthy API target
        |
        v
5. Spring Boot processes the request
        |
        v
6. API connects to PostgreSQL
        |
        v
7. Database returns the result
        |
        v
8. API returns the HTTP response
```

Each step has different possible failure points:

- DNS resolution may fail.
- TLS negotiation may fail.
- The load balancer may have no healthy targets.
- Security rules may block the load balancer-to-application connection.
- The application may not be listening on the expected port.
- The database route or security group may block the connection.
- The application may be healthy at the network layer but fail during request processing.

The debugging lesson: Do not treat every timeout as an application bug. Trace the request path and identify the first failing boundary.

Layer 2 — Practical Engineering & Workflows

11. Design a Network for the Orders API

Let's design a network before deploying anything.

Requirements

- Customers need HTTPS access.
- The application should not require a public IP address on each application instance.
- PostgreSQL should not be directly exposed to the public internet.
- Application instances should connect to PostgreSQL.
- The architecture should be able to expand across Availability Zones.

Step 1 — Choose a VPC range

For this example:

Formula: VPC CIDR: 10.0.0.0/16

Step 2 — Create subnet ranges

Example layout:

Subnet	CIDR	Intended role
Public A	10.0.1.0/24	Load balancer
Public B	10.0.2.0/24	Load balancer
App A	10.0.11.0/24	Application workloads
App B	10.0.12.0/24	Application workloads
DB A	10.0.21.0/24	Database subnet group
DB B	10.0.22.0/24	Database subnet group

The subnets should be placed in suitable Availability Zones. For managed databases, the subnet group and service-specific availability requirements also need to be considered.

Step 3 — Plan routing

A simplified routing plan:

Subnet category	Typical route
Public	VPC local route plus an internet route through the Internet Gateway
Private application	VPC local route plus NAT or other required egress routes
Private database	VPC local route and only the additional routes actually required

Do not assume the database needs general-purpose internet access. Design the required paths explicitly.

Step 4 — Define security group relationships

```
Internet clients
       |
       | TCP 443
       v
Load balancer SG
       |
       | TCP 8080
       v
Application SG
       |
       | TCP 5432
       v
Database SG
```

The arrows describe intended connection directions.

Security group rules must match the actual listener ports and traffic path. The load balancer may terminate TLS and use HTTP or HTTPS to reach its targets, depending on your configuration.

Step 5 — Validate the design

Ask these questions:

1. Does the load balancer have the routing and addressing it needs?
2. Are the application targets reachable from the load balancer?
3. Can the application reach PostgreSQL on its configured port?
4. Does the database accept traffic only from approved sources?
5. Are routes and network controls correct in every Availability Zone?
6. Can the design tolerate the loss of one zone?
7. Is outbound connectivity restricted to what the workload needs?

This is the beginning of cloud network design: describe the intended traffic first, then implement the smallest set of routes and rules that supports it.

12. Learn to Inspect AWS Networking

You do not need to create paid resources to begin learning how to inspect the environment.

If you have an AWS identity with the required read permissions, the AWS CLI can query VPC resources.

List VPCs

```
aws ec2 describe-vpcs \
  --query 'Vpcs[*].[VpcId,CidrBlock,State]' \
  --output table
```

List subnets

```
aws ec2 describe-subnets \
  --query 'Subnets[*].[SubnetId,VpcId,AvailabilityZone,CidrBlock]' \
  --output table
```

List route tables

```
aws ec2 describe-route-tables \
  --query 'RouteTables[*].[RouteTableId,VpcId]' \
  --output table
```

List Internet Gateways

```
aws ec2 describe-internet-gateways \
  --query 'InternetGateways[*].[InternetGatewayId,Attachments]' \
  --output table
```

List security groups

```
aws ec2 describe-security-groups \
  --query 'SecurityGroups[*].[GroupId,GroupName,VpcId]' \
  --output table
```

These are read-only inspection commands. They do not create or modify network resources.

The CLI identity needs the relevant describe permissions. An empty result may simply mean that the selected account or region has no matching resources.

Always verify the account and region before interpreting the output.

13. Troubleshooting: API Is Unreachable

Suppose your load balancer returns an error or the API request times out.

Use this workflow rather than randomly opening ports.

Step 1 — Check DNS

Verify that the API hostname resolves to the expected destination.

If DNS resolves incorrectly, investigate the DNS records and configuration before changing security groups.

Step 2 — Check the load balancer

Inspect:

- Listener configuration.
- Listener rules.
- Target group configuration.
- Target health.
- Health-check path and port.

An unhealthy target can result from an incorrect health-check path, an application startup problem, a port mismatch, or a network restriction.

Step 3 — Check the application security group

Verify that the application accepts the expected port from the intended load balancer security group.

Avoid opening the application port to all public addresses merely to test connectivity.

Step 4 — Check the application listener

Confirm that the application is actually listening on the expected port and interface.

For a Linux-based host, useful diagnostic commands can include:

```
ss -lntp
curl -v http://127.0.0.1:8080/actuator/health
```

These commands are examples for an environment where the corresponding tools and endpoint exist. A local health endpoint can succeed even when the load balancer cannot reach the instance, so this is only one diagnostic step.

Step 5 — Check routing and network controls

Verify:

- The relevant subnet route tables.
- Security group rules.
- NACL inbound and outbound rules.
- The route and target configuration for the traffic direction.
- Any required return path.

Step 6 — Check application and infrastructure logs

Use the evidence from load balancer health checks, application logs, and available network telemetry to identify the failing boundary.

Do not assume the application is at fault until the network path has been checked.

14. Troubleshooting: Application Cannot Reach PostgreSQL

Assume the API runs but database connections time out.

Investigate in this order:

1. Confirm the database hostname and port in application configuration.
2. Verify that the database is available and accepting connections.
3. Confirm that the database security group permits the application source on the database port.
4. Inspect the application's subnet route table and the database network path.
5. Review NACL rules, including return traffic.
6. Check DNS resolution and database endpoint configuration.
7. Review connection-pool exhaustion, database connection limits, and application logs.

Distinguish network failure from authentication failure

A **connection timeout** often suggests a network path or availability issue, although timeouts can have several causes.

An **explicit authentication error** means the connection reached a point where the database rejected the supplied credentials or authentication method.

These symptoms call for different investigations.

Do not respond to a database authentication failure by opening more network ports. Similarly, do not rotate database passwords at random when the connection cannot reach the database at all.

15. Common Networking Mistakes

Mistake	Why it fails	Better approach
Assuming a public subnet makes every resource reachable	Public reachability depends on several controls	Check routes, addressing, and security rules
Assuming a private subnet has no outbound connectivity	Private resources may use NAT or other paths	Inspect the actual route table
Opening SSH or database ports to all IPv4 addresses	Unnecessarily expands exposure	Restrict access to approved sources
Assuming a security group creates a route	Security groups control traffic; they do not establish a network path	Check routing separately
Forgetting NACL return rules	NACLs are stateless	Validate both traffic directions
Using one NAT Gateway without considering zone failures	Other zones may depend on that gateway	Evaluate availability and cost trade-offs
Treating a healthy local endpoint as proof of external reachability	The load balancer path may still be broken	Test each network boundary
Overlapping VPC CIDR ranges	Can complicate connectivity and routing	Plan address ranges before deployment

Layer 3 — Production Engineering & Interview Mastery

16. Production Network Design Principles

16.1 Minimize public exposure

Only components that require direct public reachability should have it.

A common backend architecture exposes a load balancer publicly while keeping application instances and databases in private subnets.

This reduces the number of public entry points and makes access relationships easier to reason about.

16.2 Design explicit communication paths

For each component, define:

- Which systems can initiate connections to it.
- Which ports are needed.
- Whether the connection is inbound or outbound.
- Whether the traffic needs to leave the VPC.
- Which security groups and routes support the connection.

This creates a network design that can be reviewed, tested, and debugged.

16.3 Plan for multiple Availability Zones

For production systems that require high availability, distribute appropriate resources across Availability Zones.

Do not assume that placing the API in two zones is enough. Check the availability of the load balancer, database, networking dependencies, and capacity in the surviving zone.

16.4 Control outbound traffic

Private workloads often need to call external APIs, download dependencies, or reach AWS services.

Do not automatically give every workload unrestricted internet access.

Consider:

- NAT-based egress.
- Private connectivity to supported AWS services.
- VPC endpoints where appropriate.
- Application-level destination controls.
- Monitoring for unexpected outbound traffic.

The right choice depends on which services the application needs and how tightly its network access must be restricted.

16.5 Use logs and network telemetry

When ordinary application logs cannot explain a connection failure, additional network telemetry can help.

Relevant AWS capabilities include:

- **VPC Flow Logs** for information about IP traffic to and from network interfaces and supported network resources.
- **Load balancer access logs** where supported and configured.
- **CloudTrail** for recording supported AWS API activity.
- **Application logs and metrics** for request-level behavior.

VPC Flow Logs provide useful network evidence, but they do not capture every detail of an application's request or prove that the application successfully processed a request.

17. Production Incident Simulation: Database Exposed to the Internet

Scenario: A security review discovers that a database may be reachable from public networks.

Investigation

1. Confirm the database's public-access configuration.
2. Inspect its subnet group and network placement.
3. Review the relevant route tables.
4. Inspect database security groups and any other applicable network controls.
5. Review database authentication, permissions, and encryption.
6. Examine available audit and access logs for evidence of unauthorized activity.

Remediation

1. Remove unnecessary public access.
2. Restrict database ingress to the intended application network identity or source.
3. Verify the application can still connect.
4. Review credentials and rotate them if compromise is suspected.
5. Investigate whether data or credentials were exposed.
6. Document the incident and add a preventive configuration check.

Do not assume that a public route alone proves the database was accessed. Establish the actual exposure path and investigate the evidence.

18. Production Incident Simulation: One Availability Zone Fails

Scenario: Application instances in one Availability Zone become unhealthy.

Questions to investigate

- Are healthy application targets available in another zone?
- Does the load balancer have healthy targets to serve?
- Can surviving instances handle the increased traffic?
- Is the database available?
- Does the remaining network path depend on a failed-zone resource?
- Are there NAT or egress dependencies that affect surviving instances?

Engineering response

1. Establish the scope of the failure.
2. Verify service health from the load balancer and application metrics.
3. Confirm that the remaining capacity can handle demand.
4. Restore or replace failed resources according to the recovery design.
5. Check whether any zonal dependency undermined resilience.
6. Update the architecture and runbook based on the findings.

Key lesson: Availability is a property of the complete system, not a checkbox on an individual AWS service.

19. Production Incident Simulation: Private API Cannot Reach an External Service

Scenario: A Spring Boot application runs in a private subnet but cannot reach a third-party API.

Possible causes

- No suitable route to an egress path.
- NAT Gateway configuration or availability issue.
- Security group outbound rules.
- NACL rules blocking traffic or return packets.
- DNS resolution failure.
- TLS or application-level failure.
- The external service is unavailable.

Investigation sequence

1. Confirm the destination hostname resolves.
2. Inspect the subnet route table.
3. Verify the intended egress mechanism.
4. Check security group and NACL behavior.
5. Test TCP connectivity where permitted.
6. Inspect TLS errors and application logs.
7. Confirm whether the external service is responding.

Avoid changing multiple network controls at once. Change one justified setting, validate it, and record the result.

20. Interview Questions and Answers

Q1. What is a VPC?

A logically isolated virtual network in AWS where supported resources can communicate through configured routing and network controls.

Q2. What is a subnet?

A range of IP addresses within a VPC associated with a single Availability Zone.

Q3. What is CIDR notation?

A way to express an IP address range using a prefix length, such as `10.0.0.0/16`.

Q4. What makes an AWS subnet public?

Typically, its route table provides an internet route through an Internet Gateway. Direct public IPv4 reachability also depends on addressing and applicable security controls.

Q5. What is the difference between an Internet Gateway and a NAT Gateway?

An Internet Gateway provides the VPC's internet connectivity path for appropriately configured resources. A NAT Gateway provides outbound internet connectivity for private IPv4 workloads without requiring public IPv4 addresses on those workloads.

Q6. Does a private subnet guarantee that resources cannot access the internet?

No. Private resources may have outbound internet connectivity through a NAT Gateway or another configured egress path.

Q7. What does 0.0.0.0/0 mean?

It represents all IPv4 destinations. In a route table, it is commonly used as a default route.

Q8. What is a security group?

A stateful virtual firewall associated with supported resources or network interfaces, used to control inbound and outbound traffic.

Q9. Why are security groups stateful?

They track permitted connections, so return traffic for an allowed connection is automatically permitted by the security group's state tracking.

Q10. What is a Network ACL?

A subnet-associated network access control list that filters traffic using ordered allow and deny rules. It is stateless.

Q11. What is the difference between a security group and a NACL?

Security groups are stateful and commonly define resource-level communication permissions. NACLs are stateless, subnet-level controls with ordered allow and deny rules.

Q12. Why might an EC2 instance in a public subnet still be unreachable?

It may lack a public IP address, have incorrect routing, be blocked by security groups or NACLs, or have no service listening on the intended port.

Q13. Why can an application reach the database locally but fail in AWS?

The AWS route, security group, NACL, DNS, database endpoint, or authentication configuration may be incorrect. A local test does not validate the cloud network path.

Q14. How would you secure PostgreSQL for a backend API?

Keep it off unnecessary public paths, restrict inbound access to approved application sources, enforce database authentication and least privilege, and configure appropriate encryption, backups, and monitoring.

Q15. What is a route table?

A collection of routes that determines where network traffic should be directed based on its destination.

Q16. Why can CIDR overlap be a problem?

Overlapping IP ranges can create ambiguous or incompatible routing when connecting networks, such as VPCs or corporate networks.

Q17. Why might a NAT Gateway become a reliability bottleneck?

If workloads depend on a single NAT Gateway, a failure or connectivity problem affecting its Availability Zone can disrupt their outbound access. Cross-zone routing can also affect costs and availability.

Q18. How would you investigate an unreachable API?

Trace the path from DNS through the load balancer, target health, application listener, routes, security groups, NACLs, and application logs. Find the first failing boundary before making changes.

Q19. Why does opening a port not always fix connectivity?

The route may be missing, the destination may be unreachable, the application may not be listening, or another network control may still block traffic.

Q20. How would you design a highly available AWS network for a backend API?

Use an appropriate VPC address plan, distribute suitable resources across Availability Zones, expose only the required entry point, keep internal workloads on restricted network paths, design database connectivity deliberately, and test failure behavior.

Capstone Lab — Draw and Debug the Orders API Network

Objective

Create a network design for a Spring Boot API and PostgreSQL database. You do not need to provision paid resources.

Task 1 — Draw the VPC

Include:

- One VPC CIDR.
- Two public subnet ranges.
- Two private application subnet ranges.
- Two private database subnet ranges.
- Availability Zone placement.

Task 2 — Design the routes

Write the intended route table for each subnet category. Identify which routes are required for local communication, internet-facing traffic, and outbound application connectivity.

Task 3 — Define security group rules

Document the intended traffic:

- Public client to load balancer.
- Load balancer to application.
- Application to database.
- Application to approved external destinations.

Specify the port and source for each required connection.

Task 4 — Simulate failures

Explain how you would investigate:

- The load balancer has no healthy targets.
- The application cannot connect to PostgreSQL.
- The application cannot reach an external API.
- A security group was changed accidentally.
- One Availability Zone becomes unavailable.

Task 5 — Review the security posture

Identify:

- Which components need public reachability.
- Which components should remain private.
- Which permissions are unnecessary.
- Which network paths are single points of failure.
- Which costs need consideration.

Mastery Assessment

Answer these without looking at the chapter:

1. Why is a public subnet not sufficient to make an EC2 instance publicly reachable?
2. How are a route table and a security group different?
3. Why are security groups stateful but NACLs stateless?
4. How does a private subnet reach an external IPv4 API through a NAT Gateway?
5. Why can an application and database be in the same VPC yet fail to communicate?
6. What would you inspect if a load balancer reports unhealthy targets?
7. Why should PostgreSQL usually not be exposed directly to public clients?
8. How would you design for an Availability Zone failure without making every component public?

Completion Checklist

- [ ] I understand VPCs and subnets.
- [ ] I can read common CIDR ranges.
- [ ] I understand public and private subnet routing.
- [ ] I can explain Internet Gateways and NAT Gateways.
- [ ] I understand security groups and NACLs.
- [ ] I can trace an HTTPS request to a backend.
- [ ] I can diagnose basic AWS network failures.
- [ ] I can design a network for a Spring Boot API and PostgreSQL.
- [ ] I can explain the cost and availability trade-offs of network choices.

Handbook Completion Protocol

Save this chapter as:

`content/aws/02-aws-vpc-subnets-routing-gateways-security-groups.md`

This is **Chunk 2 of 6**. Preserve it as Markdown. Do not generate a separate chunk PDF.

After all six AWS chapters have been saved and verified, create the complete handbook:

`docs/aws/BEH_AWS_COMPLETE_HANDBOOK.pdf`

Update the README only after verifying the source chapters and final PDF. Commit only after the complete AWS source content and final PDF have been verified.

Required commit message: `AWS done`

This chapter does not imply that any repository operation, PDF generation, or Git commit has already occurred.

Up next

AWS Chunk 3 of 6 — EC2, Application Load Balancers, Auto Scaling & Compute Architecture.

We'll move from networking into running the actual backend: what a virtual server is, how load balancing works, how instances scale, and why a running instance is not the same as a reliable service.

Part III: EC2, Application Load Balancers, Auto Scaling & Compute Architecture

Chunk 3 of 6 · Backend Engineering Handbook · Cloud Engineering Track

AWS — Chunk 3 of 6: EC2, Application Load Balancers, Auto Scaling & Compute Architecture

Backend Engineering Tools & Infrastructure Handbook · Cloud Engineering Track

Source file: `content/aws/03-aws-ec2-load-balancing-auto-scaling-compute-architecture.md`

Learning Objectives

By the end of this chapter, you should be able to:

- Explain how EC2 runs a backend application.
- Understand AMIs, instance types, EBS volumes, launch templates, and instance metadata.
- Explain how an Application Load Balancer distributes requests.
- Configure the conceptual architecture of an Auto Scaling Group.
- Trace a request from a client to a Spring Boot application running on EC2.
- Diagnose common 502/503 errors, unhealthy targets, and failed scaling events.
- Discuss compute reliability and scaling trade-offs at an FAANG SDE-1 interview level.

Layer 1 — Fundamentals & Mental Models

1. The Simplest Mental Model of EC2

Imagine that you have built a Spring Boot REST API on your laptop.

Your application works locally, but users on the internet cannot reliably access your laptop as a production server. You need a machine that can run continuously, accept network traffic, and provide sufficient compute resources.

Amazon EC2 (Elastic Compute Cloud) provides virtual machines in AWS.

Think of EC2 as renting a computer in an AWS data center. You select its operating system, CPU and memory capacity, storage configuration, and networking permissions.

You can install Java, run your Spring Boot JAR, configure a service, and expose the application through a controlled network path.

What EC2 gives you

- **Compute:** CPU and memory to execute application code.
- **Operating system:** Linux or Windows, depending on the selected image.
- **Storage:** Attached volumes for files and application data.
- **Networking:** Private IP addresses, optional public connectivity, and security groups.
- **Identity:** An instance profile can provide AWS permissions to the application.
- **Lifecycle control:** Start, stop, reboot, terminate, or replace instances.

EC2 is infrastructure, not a complete application platform. You still manage the guest operating system, runtime, application process, deployment strategy, and much of the operational configuration.

---

2. The Five Concepts You Must Understand

Understanding EC2 requires mastering five distinct concepts that work together to form the compute platform:

1. EC2 Instance — The Running Machine

The virtual server executing your application. Its available CPU, memory, network performance, and cost depend on the instance type and configuration.

2. AMI — The Machine's Starting Blueprint

An **Amazon Machine Image** contains the information required to launch an instance, including its operating system and configured software or image data. AMIs help standardize server creation across your fleet.

3. Instance Type — The Hardware Profile

Determines the machine's resource characteristics. Different families emphasize general-purpose workloads, compute, memory, storage, or accelerated processing.

4. EBS — Persistent Block Storage

Elastic Block Store volumes can hold operating-system data, application files, and other data that should survive an instance reboot. Volume lifecycle and deletion settings matter when instances are terminated.

5. IAM Instance Role — The Machine's AWS Identity

An attached **instance profile** makes role credentials available to software running on the instance. Use it to grant specific AWS permissions without embedding long-lived access keys in code or configuration files.

Summary Comparison

Concept	What It Is	Purpose
AMI	Launch blueprint	Machine image, OS baseline, and pre-baked software
Instance Type	Hardware profile	CPU, RAM, network throughput, and I/O capacity
EC2 Instance	Virtual machine	The actively running server executing workloads

Concept	What It Is	Purpose
EBS Volume	Block storage	Persistent local virtual disk attached over the network
Instance Role	AWS identity	Least-privilege IAM permissions for application code

A useful distinction

An AMI is not the same thing as a running instance:

- **AMI:** The launch blueprint.
- **Instance type:** The compute capacity.
- **EC2 instance:** The running virtual machine.
- **EBS volume:** Attached block storage.
- **Instance role:** The AWS permissions available to the workload.

You can launch multiple instances from the same AMI and instance type, each running a separate copy of your backend.

---

3. EC2 Instance Families and Choosing Capacity

AWS offers instance families optimized for different workloads. Exact names, availability, and pricing vary by region and change over time.

Workload	What matters most	Starting point	Example AWS Family
Ordinary REST API	Balanced CPU and memory	General-purpose instance	`t4g`, `m6i`, `m7g`
CPU-heavy processing	Sustained CPU throughput	Compute-optimized instance	`c6i`, `c7g`
Large in-memory workload	RAM capacity	Memory-optimized instance	`r6i`, `r7g`
Storage-intensive processing	Storage throughput and IOPS	Storage-optimized instance	`i3en`, `i4i`

For a small Spring Boot API, a general-purpose instance may be a sensible starting point. However, do not select production capacity based solely on a tutorial's recommendation.

Measure:

- CPU utilization and throttling where applicable (e.g., T-series burstable CPU credits).
- Heap usage and garbage-collection behavior.
- Process memory and operating-system memory pressure.
- Request rate and latency percentiles (p50, p95, p99).
- Network throughput and connection counts.
- Database connection-pool usage.

> **Interview Insight:** High CPU does not automatically mean that you need a larger instance. The bottleneck could be an inefficient query, excessive serialization, a tight retry loop, or an application bug. Diagnose before scaling vertically.

4. How Does a Client Reach an EC2 Application?

Suppose your application listens on port **`8080`**.

A simplified direct-access path looks like this:

```
Client
  |
  | HTTPS request
  v
Public network
  |
  v
EC2 public IPv4 address
  |
  v
EC2 security group permits the connection
  |
  v
Operating-system firewall and network stack
  |
  v
Spring Boot process listening on port 8080
```

This architecture can work for learning, but it is usually not the preferred production entry point for a horizontally scalable API.

Problems include:

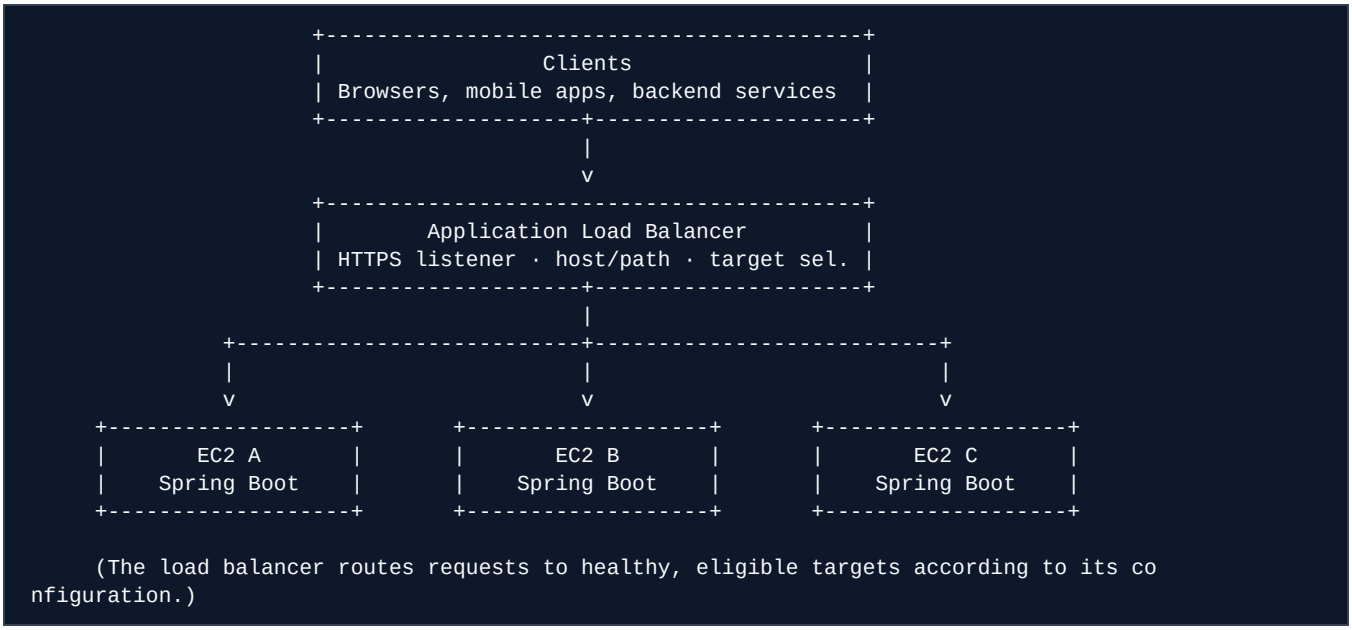
- A public instance IP can change after certain lifecycle events (e.g., stop/start).
- One instance is a single point of failure.
- Clients become tightly coupled to the instance's specific address.
- There is no managed layer distributing traffic across multiple instances.
- TLS termination, routing, and traffic-management responsibilities become harder to organize.

A more resilient architecture introduces a load balancer.

5. Application Load Balancer: The Traffic Director

An **Application Load Balancer (ALB)** distributes incoming HTTP and HTTPS requests across registered targets, such as EC2 instances.

It operates at **Layer 7** of the networking model, where it can understand HTTP-level information such as hostnames, paths, query parameters, and HTTP headers.



Important ALB components

- **Listener:** Checks incoming connections on a configured protocol and port, such as HTTPS on port `443`.
- **Listener rule:** Determines what action to take. A rule might route `/api/orders/*` to an orders target group.
- **Target group:** Defines the destination targets and health-check configuration.
- **Target:** The actual backend receiving requests, such as an EC2 instance on port `8080`.
- **Health check:** A configured probe used to determine whether a target is healthy enough to receive traffic.

Example routing

Imagine one domain hosts two backend services:

```
api.example.com/orders/*   -> Orders target group
api.example.com/payments/* -> Payments target group
```

The ALB can use host-based or path-based routing to direct traffic to the appropriate target group.

This does not mean the ALB understands your business logic. Your application still validates requests, authenticates users, enforces authorization, and executes the business operation.

ALB versus Network Load Balancer

Feature	Application Load Balancer (ALB)	Network Load Balancer (NLB)
Primary layer	Layer 7 (Application)	Layer 4 (Transport)
Traffic model	HTTP/HTTPS and gRPC aware	TCP, TLS, UDP and supported transport protocols
Routing	Host, path, HTTP header, and query string rules	Transport-level connection distribution (IP/Port)
Static IP	Assigned DNS name (IP addresses change dynamically)	Static Elastic IP per Availability Zone

Feature	Application Load Balancer (ALB)	Network Load Balancer (NLB)
Common use	Web applications, REST APIs, microservices	Ultra-low latency, gaming, streaming, static IP whitelisting

Choose according to the protocol and routing behavior you need, not because one product is universally better.

---

6. Auto Scaling Groups: Replacing Manual Server Management

Without automation, an operator might manually launch another EC2 instance whenever traffic increases.

That does not scale well.

An **Auto Scaling Group (ASG)** manages a group of EC2 instances according to configured capacity limits, health checks, and scaling policies.

Three values define its basic capacity boundaries:

- **Minimum capacity:** The lowest capacity the group should maintain.
- **Desired capacity:** The target number of instances the group should maintain.
- **Maximum capacity:** The upper limit it can scale to under the configured policy.

Example:

```
Minimum: 2
Desired: 2
Maximum: 8
```

If the group has two healthy instances and traffic increases, a scaling policy may increase desired capacity to four. The ASG launches additional instances, subject to configuration, quotas, capacity availability, and other constraints.

If an instance becomes unhealthy, the group can replace it according to its health-check and replacement behavior.

What enables repeatable instance creation?

A **launch template** defines the configuration used to launch instances, including items such as:

- AMI ID.
- Instance type.
- Instance profile (IAM role).
- Security groups.
- Storage configuration (EBS root and data volumes).
- User data (startup scripts).
- Key-pair settings, where used.

The ASG uses the launch template to create instances consistently.

Why multi-AZ placement matters

If all application instances are concentrated in one Availability Zone, a disruption in that zone can affect the entire application tier.

A resilient design distributes instances across multiple Availability Zones within the same AWS Region. Actual availability depends on subnet selection, capacity, health checks, load-balancer configuration, and the behavior of dependent services.

Multi-AZ is not the same as multi-region. A multi-region design introduces additional DNS or traffic-routing, data replication, failover, and consistency decisions.

---

7. Scaling Vertically versus Horizontally

When traffic grows or resource demands increase, systems can scale along two distinct dimensions:

Vertical Scaling — Scale Up

Move to a larger instance with more CPU or memory.

- **Characteristics:** Easy to apply without re-architecting software; useful when a single process requires large RAM or compute capacity.
- **Trade-offs:** Hard upper limits on maximum hardware size; requires instance replacement or downtime to resize; does not provide high availability or fault tolerance.

Horizontal Scaling — Scale Out

Add more instances and distribute requests across them.

- **Characteristics:** Elastic capacity that grows and shrinks dynamically; allows replacing failed nodes with zero downtime; supports multi-AZ fault tolerance.
- **Trade-offs:** Requires an application design that tolerates multiple concurrent instances; state must be externalized; increases network and database connection management complexity.

Dimension	Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
Mechanism	Resize machine to larger instance type	Add more machines to the target group / ASG
Fault Tolerance	Single point of failure remains	Redundant nodes across multiple AZs
State Handling	Monolithic state survives on single machine	Requires stateless design or external shared store
Upper Limit	Largest available cloud instance type	Virtually unlimited elasticity (within quotas)
Cost Profile	Exponential cost curve at high instance tiers	Linear pay-as-you-go cost matching demand

Horizontal scaling is not automatic application-level scalability. Your backend must be designed to handle requests arriving at any healthy instance.

For example, if a user's session exists only in the memory of EC2 A, a subsequent request routed to EC2 B may not find that session.

Better options include stateless authentication (such as JWTs), externalized session storage (such as Redis) where needed, or another deliberate session-management design.

---

8. User Data and Instance Initialization

EC2 **user data** allows you to provide startup instructions when launching an instance. On common Linux configurations, it is processed through `cloud-init` or related initialization mechanisms.

A simplified educational example:

```
#!/bin/bash
set -euxo pipefail

dnf install -y java-21-amazon-corretto-headless

install -d -o ec2-user -g ec2-user /opt/orders-api
```

This example installs Java and prepares a directory on a compatible Amazon Linux image. It does **not** download, configure, or start your application.

A production bootstrap process may also:

1. Install or verify the runtime.
2. Retrieve a versioned application artifact from an authorized location (such as private S3).
3. Obtain runtime configuration from a secure source (such as AWS Systems Manager Parameter Store or Secrets Manager).
4. Install a system service (`systemd`) or configure a deployment agent.
5. Start the application and verify readiness.
6. Send logs and metrics to a central monitoring system (such as CloudWatch).

[!CAUTION]

Do not put passwords, access keys, database credentials, or other secrets directly in user data. Instance initialization logs and metadata handling can expose information to unintended readers if permissions and access are poorly designed.

User data is also not a substitute for a repeatable image-building or deployment pipeline. Repeatedly performing complex installations at boot can increase startup time and create inconsistent results.

---

9. Instance Metadata and IMDSv2

The EC2 **Instance Metadata Service (IMDS)** provides information about an instance from inside its environment (`http://169.254.169.254/`). Depending on configuration and permissions, this can include instance identity information, network configuration, and temporary role credentials.

IMDSv2 uses a token-based request flow (session-oriented PUT request to acquire a token header) that helps protect metadata access against SSRF (Server-Side Request Forgery) attacks and unintended requests.

Production basics:

- **Require IMDSv2:** Disable IMDSv1 by configuring `HttpTokens=required` in launch templates.
- **Limit metadata hop count:** Set `HttpPutResponseHopLimit=1` to prevent containers or forwarded proxies from accessing instance credentials.
- **Avoid SSRF vulnerabilities:** Avoid exposing server-side request functionality that can be abused to reach metadata.
- **Narrowly scoped instance roles:** Follow least privilege for attached IAM roles.
- **Never treat a private IP address as a security boundary by itself.**

The application should retrieve AWS credentials through the supported AWS SDK credential provider chain rather than hardcoding credentials in source code.

Layer 2 — Practical Engineering & Workflows

10. Design a Basic Spring Boot Deployment

Suppose we are deploying an **`Orders API`** built with Java 21 and Spring Boot.

Its purpose is to accept HTTP requests and persist orders in PostgreSQL.

A simple learning architecture:

```
Internet
   |
   v
Application Load Balancer
   |
   v
Target group :8080
   |
   +------> EC2 instance A: Spring Boot
   |
   +------> EC2 instance B: Spring Boot
                 |
                 v
          PostgreSQL database
          in private networking
```

For a production design, the database would typically be managed or operated as a separate stateful tier (such as Amazon RDS), with appropriate private connectivity, credentials, backup strategy, and high availability.

The application instances should not depend on a particular instance's local disk or memory for durable business data.

Recommended application properties

```
server.port=8080

management.endpoints.web.exposure.include=health,info
management.endpoint.health.probes.enabled=true
```

This enables the configured management endpoints, including health and probe support. In a real application, verify the relevant Spring Boot Actuator dependency and version, and ensure management endpoints are not publicly exposed without appropriate controls.

A health endpoint can support load-balancer health checks, but it must be designed carefully:

- **Liveness** asks whether the process should be restarted.
- **Readiness** asks whether the instance is ready to serve traffic.

Do not automatically make every external dependency a liveness condition. If the database is briefly unavailable and every application instance reports itself as dead, the resulting restart loop may worsen the incident.

---

11. Configure the Load-Balancer Request Path

Before provisioning resources, understand the required network flow:

Source	Destination	Port	Purpose
Internet clients	ALB	443	Public HTTPS traffic
ALB security group	EC2 security group	8080	Application requests
EC2 security group	Database security group	5432	Application data access

The critical rule is that the EC2 security group should generally allow inbound application traffic from the **ALB security group**, rather than allowing the entire internet to connect directly to port `8080`.

For PostgreSQL, the application tier should be authorized to connect to the database tier. The database should not be publicly exposed merely because the API is public.

These are conceptual rules. You must also configure appropriate subnets, route tables, listeners, target groups, health checks, and database connectivity.

---

12. Understand the Health-Check Path

Assume the ALB checks:

```
Protocol: HTTP
Port: traffic port (8080)
Path: /actuator/health/readiness
```

The request path is:

```
ALB health checker
      |
      v
EC2 target:8080
      |
      v
Spring Boot readiness endpoint
      |
      v
HTTP response (200 OK)
```

The target must return a response accepted by the configured success-code matcher (typically HTTP `200`).

Common problems include:

- The application listens on a different port than the target group expects.
- The health-check path is incorrect or misspelled.
- The application takes too long to start (longer than the health-check interval $\times$ threshold).

- The EC2 security group blocks ALB traffic.
- The endpoint requires authentication when the health checker cannot provide it.
- The process starts but cannot initialize its essential dependencies.

A successful TCP connection alone does not prove that the application can serve useful requests.

---

13. Use AWS CLI to Inspect Compute Resources

The following commands are read-only inspection commands. They assume the AWS CLI is installed, configured for an appropriate account, and operating in the intended Region.

Verify identity and Region

```
aws sts get-caller-identity
aws configure list
```

These commands help verify which AWS identity and CLI configuration are active.

List EC2 instances

```
aws ec2 describe-instances \
  --query 'Reservations[].Instances[].{
    ID:InstanceId,
    State:State.Name,
    Type:InstanceType,
    AZ:Placement.AvailabilityZone,
    PrivateIP:PrivateIpAddress
  }' \
  --output table
```

This returns a compact view of instances visible to your permissions in the configured Region.

Inspect target groups

```
aws elbv2 describe-target-groups \
  --query 'TargetGroups[].{
    Name:TargetGroupName,
    Port:Port,
    Protocol:Protocol,
    VPC:VpcId
  }' \
  --output table
```

Inspect registered target health

```
aws elbv2 describe-target-health \
  --target-group-arn YOUR_TARGET_GROUP_ARN
```

Replace the placeholder with the actual target-group ARN. This command is particularly useful when a target is unhealthy or a deployment does not appear to receive traffic.

Inspect Auto Scaling Groups

```
aws autoscaling describe-auto-scaling-groups \
  --query 'AutoScalingGroups[].{
    Name:AutoScalingGroupName,
    Desired:DesiredCapacity,
    Min:MinSize,
    Max:MaxSize,
    Instances:Instances[].InstanceId
  }' \
  --output json
```

Inspect scaling activities

```
aws autoscaling describe-scaling-activities \
  --auto-scaling-group-name YOUR_ASG_NAME \
  --max-items 20
```

Scaling activity messages can explain why an instance was launched, terminated, or not launched successfully.

> [!NOTE]

> Inspection commands do not guarantee that the caller has sufficient permission to view every resource. Avoid copying state-changing examples from tutorials into a real account without reviewing their impact.

---

14. Build a Deployment Workflow

A reliable deployment should be repeatable and observable.

One possible workflow:

1. **Build and test** the Spring Boot application in CI.
2. **Package** a versioned JAR or container image.
3. **Publish** the artifact to an approved artifact repository (e.g., private S3 or Amazon ECR).
4. **Create or update** a launch template referencing the intended application version or image.
5. **Deploy** through a controlled rollout or instance refresh strategy.
6. **Wait** for new instances to become healthy in the target group.
7. **Verify** application-level health and business behavior.
8. **Monitor** latency, errors, saturation, and database pressure.
9. **Roll back** if defined safety conditions are violated.

A key operational principle: **an instance being `running` does not mean the application is healthy**.

The EC2 state, operating-system health, application readiness, target-group health, and end-to-end business success are different signals.

---

15. Graceful Shutdown and Draining

Imagine that an ASG needs to replace an EC2 instance during a deployment.

If the instance is terminated immediately while it is processing requests, users may receive errors or clients may retry operations that already partially completed.

A safer shutdown sequence is:

```
1. Deregister target -> ALB marks target as "draining" (stops routing new requests)
2. Connection draining delay -> In-flight requests complete cleanly within timeout
3. Termination lifecycle hook -> Application process receives SIGTERM
4. Spring Boot graceful shutdown -> Active HTTP handlers finish, connection pools close
5. Instance termination -> Operating system halts and EC2 instance terminates
```

Exact behavior depends on the deployment mechanism, load-balancer deregistration settings, instance lifecycle hooks, and application shutdown configuration.

For a Spring Boot service, configure graceful shutdown and appropriate request timeouts:

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=30s
```

Make sure long-running operations have explicit cancellation or completion semantics.

---

16. Autoscaling Policy Design

There is no universally correct scaling metric.

Metric	What it can tell you	Important limitation
Average CPU utilization	Compute pressure	A database-bound API may be slow with low CPU
Request count per target	Traffic volume per registered target	Request complexity can vary substantially
Memory utilization	Memory pressure, if correctly collected	Requires suitable CloudWatch agent telemetry
Queue depth	Backlog of asynchronous work	Scale-out delay and per-message processing time matter
Latency or saturation signals	User-visible pressure	Needs careful measurement and scaling-policy design

Consider an API that receives ten times its normal traffic. CPU-based scaling may help if requests are CPU-intensive, but may be ineffective if requests are waiting for PostgreSQL connections.

Before increasing the instance count, check whether the bottleneck is:

- Application CPU.
- Heap or native memory.
- Connection-pool exhaustion.
- Database CPU or I/O.
- Network throughput.
- A hot key or lock.
- An external dependency's rate limit.

> [!WARNING]
>
> Adding application instances can actually increase database load by creating more concurrent connections.

---

17. A Safe Hands-On Lab

This lab is designed to be completed without creating billable AWS resources. You can inspect existing resources if you already have authorized access, or draw and reason about the architecture locally.

Task A: Draw the architecture

Draw the path from client to ALB, from ALB to two EC2 targets, and from the application tier to a private PostgreSQL database. Mark each security-group relationship and port.

Task B: Define the launch template

Write down the following intended values:

- A supported Linux AMI (e.g., Amazon Linux 2023).
- A suitable instance type (e.g., `t4g.small` or `m7g.large`).
- An instance role with minimum required permissions.
- A security group that permits app traffic only from the ALB.
- Storage appropriate for the workload.
- A repeatable startup or deployment mechanism.

Do not launch the instance merely to complete the exercise.

Task C: Design target health checks

Specify a readiness endpoint and explain:

- What constitutes a healthy response?
- How long should startup be allowed to take?
- What happens if the database is temporarily unavailable?
- How will you distinguish a failed application from a failed health-check configuration?

Task D: Plan scaling

Assume the group has minimum capacity 2, desired capacity 2, and maximum capacity 8. Write down what should happen when:

1. One instance fails its health checks.
2. CPU rises steadily.
3. Request traffic spikes for two minutes.
4. PostgreSQL becomes saturated.
5. The selected instance type is temporarily unavailable in one AZ.

For each situation, identify the signal you would inspect before changing capacity.

Layer 3 — Production Engineering & Interview Mastery

18. Production Incident Simulations

Incident A: The ALB returns HTTP 502

Symptoms: Clients receive 502 responses, although EC2 instances appear to be running.

Investigate in this order:

1. Inspect ALB access logs or metrics where available (`HTTPCode_ELB_502_Count`).
2. Check target-group health and the exact health-check reason.
3. Confirm that Spring Boot is listening on the expected port (`ss -lntp`).
4. Check whether the application process has crashed or is restarting.
5. Verify ALB-to-instance security-group rules (ALB egress and EC2 ingress).
6. Review application logs, startup errors, and connection behavior.
7. Check whether the application closes connections unexpectedly or returns malformed upstream responses.

A 502 generally indicates a problem with the upstream response or connection path, but the precise cause depends on the architecture and protocol.

Do not conclude that the instance is healthy simply because EC2 reports it as `running`.

Incident B: The ALB returns HTTP 503

Symptoms: Requests receive 503 responses during deployment.

Possible causes include:

- No healthy registered targets in the target group.
- All targets temporarily unavailable.
- Incorrect listener or target-group configuration.
- A deployment that removed old targets before new targets became healthy.
- An application startup failure affecting every target.

Check target health, listener rules, deployment events, and application readiness. Restore healthy capacity and roll back if required by your incident procedure.

Incident C: Auto Scaling does not add instances

Symptoms: CPU is high, but the group remains at two instances.

Investigate:

1. Is the scaling policy enabled and receiving its metric?
2. Is desired capacity already at the maximum?

3. Are account or service quotas being reached (vCPU limits)?
4. Is the launch template valid?
5. Can the selected instance type be launched in the configured subnets and Availability Zones?
6. Are subnet IP addresses available?
7. Does the instance have required launch permissions and configuration?
8. Are cooldowns, warm-up periods, or policy evaluation delays affecting the response?

Inspect ASG scaling activities (`aws autoscaling describe-scaling-activities`) instead of guessing.

Incident D: New instances launch but never become healthy

Likely causes:

- The wrong AMI was selected.
- User data failed during execution.
- The JAR or application artifact could not be downloaded (e.g., S3 permission error).
- The instance role lacks required permissions.
- The application is bound to localhost (`127.0.0.1`) rather than `0.0.0.0`.
- The security group blocks the health checker.
- The configured readiness endpoint returns an unexpected status (e.g., 503 due to uninitialized datasource).
- Startup takes longer than the configured health-check expectations.

Engineering lesson: Launch success and application readiness are separate milestones.

Incident E: Scaling out makes the database slower

Suppose each application instance creates a connection pool of up to 40 database connections.

At two instances, the application tier can potentially request 80 connections. At eight instances, that upper bound becomes 320.

This is a simplified upper-bound calculation; actual open connections depend on pool configuration and runtime behavior.

If PostgreSQL cannot handle the additional concurrency, scaling the application tier may increase database latency and connection contention.

Possible responses:

- Establish a safe total connection budget.
- Tune pool size and acquisition timeouts.
- Improve query efficiency and transaction duration.
- Apply admission control or backpressure.
- Introduce connection pooling middleware (such as AWS RDS Proxy).
- Scale or optimize the database tier when evidence supports it.
- Avoid retry storms that amplify overload.

Horizontal scaling moves bottlenecks; it does not eliminate them.

Incident F: Deployment creates a traffic outage

A new launch-template version is deployed, but the new instances fail their health checks.

A robust deployment should limit blast radius:

- Maintain sufficient healthy capacity during rollout (e.g., minimum healthy percentage = 100%).
- Wait for readiness before sending production traffic.
- Monitor error rate and latency during the rollout.
- Stop or reverse the rollout when safety conditions fail.
- Keep a known-good version available.
- Verify the rollback actually restores healthy service.

The best rollback plan is one you have tested.

---

19. Security and Reliability Checklist

Before considering an EC2-based backend production-ready, verify the following.

Identity and access

- [ ] Use an instance role rather than embedded long-lived credentials.
- [ ] Grant only required AWS permissions (least privilege).
- [ ] Protect metadata access and require IMDSv2 where appropriate.
- [ ] Keep secrets in an appropriate secrets-management service.
- [ ] Restrict administrative access (use AWS Systems Manager Session Manager instead of open SSH ports) and audit it.

Network boundaries

- [ ] Place application instances in private subnets.
- [ ] Expose the ALB in public subnets rather than exposing application instances directly.
- [ ] Permit inbound application traffic only from the ALB security group.
- [ ] Keep the database private in dedicated database subnets.
- [ ] Restrict outbound traffic where practical (using NAT Gateways or VPC Endpoints).
- [ ] Separate administrative access from public application traffic.

Availability

- [ ] Use multiple Availability Zones when the availability requirement justifies it.
- [ ] Maintain sufficient minimum capacity across AZs.
- [ ] Configure target health checks appropriately.
- [ ] Ensure instances can be replaced safely (stateless application design).
- [ ] Design the application to tolerate instance replacement.
- [ ] Verify database failover and recovery separately.

Observability

Monitor at least:

- [ ] Request count and HTTP error rates (4xx, 5xx).
- [ ] Latency percentiles, such as p95 and p99.
- [ ] CPU and memory pressure.
- [ ] Healthy and unhealthy target counts.
- [ ] Desired versus actual ASG capacity.
- [ ] Instance launch and termination failures.
- [ ] Database connections and query latency.
- [ ] Application startup and readiness failures.

Cost

- [ ] Select instance types based on measured workload needs.
- [ ] Review utilization and idle capacity.
- [ ] Understand the billing implications of load balancers, storage, data transfer, and other associated services.
- [ ] Use scaling bounds to prevent uncontrolled capacity growth.
- [ ] Test cost-impacting changes before applying them.

---

20. FAANG-Style Interview Questions and Answers

Q1. What is Amazon EC2?

EC2 is an AWS service that provides configurable virtual machines for running workloads. Users choose the operating system image, instance type, storage, networking, and associated permissions.

Q2. What is the difference between an AMI and an instance?

An AMI is a machine image used to launch instances. An instance is the running virtual machine created from that image and its launch configuration.

Q3. What is the difference between an instance type and an AMI?

An AMI defines the machine image and software baseline. An instance type defines the compute resource profile, including CPU, memory, and networking characteristics.

Q4. What is EBS?

Elastic Block Store provides persistent network-attached block storage volumes that can be mounted to supported EC2 instances. Data durability and deletion behavior depend on the volume and instance configuration.

Q5. What does an Application Load Balancer do?

An ALB distributes HTTP/HTTPS requests across registered targets and supports Layer 7 routing based on configured rules (paths, hosts, headers).

Q6. What is a target group?

A target group defines a set of destinations, their traffic configuration, and associated health checks. An ALB listener routes requests to target groups based on listener rules.

Q7. What is the difference between an ALB and an NLB?

An ALB focuses on HTTP-level routing at Layer 7. An NLB distributes traffic at the transport/network level (Layer 4) for high throughput, low latency, and static IP requirements.

Q8. What is an Auto Scaling Group?

An ASG maintains a group of EC2 instances within configured capacity limits and can automatically launch or replace instances based on health checks and scaling policies.

Q9. What are minimum, desired, and maximum capacity?

Minimum is the lower capacity boundary, desired is the target instance count the group currently maintains, and maximum is the upper boundary for scaling out.

Q10. Why use a launch template?

It standardizes instance creation by defining the AMI, instance type, security groups, storage, instance profile, user data, and other launch settings in a versioned template.

Q11. What is the difference between vertical and horizontal scaling?

Vertical scaling increases the capacity of an individual machine (larger instance type). Horizontal scaling adds more machines and distributes work among them.

Q12. Why should application instances be stateless?

Stateless instances are easier to replace and scale because requests do not depend on local in-memory session state or durable local files. Shared or durable state belongs in suitable external systems like Redis or databases.

Q13. What does a target health check tell you?

It indicates whether a target satisfies the configured health-check criteria (e.g., returning HTTP 200 on `/actuator/health/readiness`). It does not prove every business operation or downstream dependency is functioning correctly.

Q14. Why can an EC2 instance be running but the API be unavailable?

The operating system may be running while the application is crashed, starting, listening on the wrong port, blocked by network rules, or unable to serve requests.

Q15. How would you debug an ALB 502?

Check target health, application logs, listening ports, security-group rules, startup status, and the upstream connection path. Use ALB access logs and metrics where available to narrow the failure.

Q16. Why might an ALB return 503?

The load balancer may have no healthy eligible targets or may be unable to route requests under the configured conditions. Inspect target health, listener configuration, and deployment events.

Q17. Why might Auto Scaling fail to launch an instance?

Potential causes include reaching maximum capacity, invalid launch configuration, unavailable capacity in the AZ, exhausted subnet IPs, quotas, permissions, or service constraints.

Q18. Why can adding EC2 instances make an application slower?

More instances can increase database connections, downstream traffic, lock contention, and request concurrency. If a shared dependency is saturated, scaling out the application tier may worsen the bottleneck.

Q19. What is the purpose of graceful shutdown?

It reduces interrupted requests and inconsistent behavior by allowing traffic draining, in-flight work completion, and orderly resource cleanup before termination.

Q20. Why is multi-AZ deployment not the same as multi-region deployment?

Multi-AZ distributes resources across Availability Zones within a single Region. Multi-region uses separate Regions and requires additional decisions about traffic failover, data replication, consistency, and disaster recovery.

---

21. Mastery Challenge

You are designing a production Orders API with these requirements:

- Two or more application instances during normal operation.
- HTTPS for public requests.
- No direct public access to the application instances.
- Automatic replacement of unhealthy instances.
- Scale-out when traffic rises.
- A private PostgreSQL database.
- Safe deployments and a rollback path.

Complete the following tasks before moving to Chunk 4:

1. Draw the request path from client to database.
2. Identify the roles of the ALB, target group, launch template, and ASG.
3. Define the security-group relationships.
4. Choose a readiness endpoint and explain its behavior.
5. Explain how a failed instance is replaced.
6. Explain how the design behaves when database capacity is exhausted.
7. List five metrics you would watch during deployment.
8. Describe how you would investigate an ALB 502.

9. Explain how you would prevent a faulty rollout from removing all healthy capacity.

10. Identify which parts of the design create costs even when traffic is low.

Self-assessment

Rate each skill from 0 to 2:

- **0:** Cannot explain it yet.
- **1:** Can explain it but need help applying it.
- **2:** Can explain it and solve a practical scenario.

Skill	Score
EC2, AMIs, instance types, and EBS	/2
ALB, listeners, and target groups	/2
ASG capacity and scaling	/2
Health checks and readiness	/2
Network and identity security	/2
Incident diagnosis	/2
Deployment and rollback	/2
Cost and bottleneck reasoning	/2

Target: At least 13/16, with no zero in health checks, security, or incident diagnosis. Review weak areas before advancing.

Chapter Completion Protocol

Chapter: AWS Chunk 3 of 6

Save as: `content/aws/03-aws-ec2-load-balancing-auto-scaling-compute-architecture.md`

Do not generate a PDF yet. Save all six AWS chapters first, verify the complete source content, then generate one final AWS handbook:

`docs/aws/BEH_AWS_COMPLETE_HANDBOOK.pdf`

After the source chapters and final PDF have been verified, the intended Git commit message is:

```
AWS done
```

This chapter completes the core compute foundation. The next AWS chapter will cover **storage and managed databases**, including S3, EBS/EFS, RDS, backups, and high availability.

Part IV: Storage, Managed Databases, Backups & High Availability

Chunk 4 of 6 · Backend Engineering Handbook · Cloud Engineering Track

AWS — Chunk 4 of 6: Storage, Managed Databases, Backups & High Availability

Backend Engineering Tools & Infrastructure Handbook · Cloud Engineering Track

Source file: `content/aws/04-aws-storage-s3-ebs-efs-rds-backups-high-availability.md`

Learning Objectives

By the end of this chapter, you should be able to:

- Choose between Amazon S3, EBS, and EFS based on access patterns and requirements.
- Explain why application servers should not store durable business data on their local disks.
- Understand Amazon RDS, Multi-AZ deployments, read replicas, and automated backups.
- Design a basic Spring Boot and PostgreSQL architecture on AWS.
- Explain durability, availability, consistency, RPO, and RTO.
- Diagnose common database connection, storage, and recovery failures.
- Discuss database scaling and disaster recovery in backend engineering interviews.

Layer 1 — Fundamentals & Mental Models

1. The Three Things Your Backend Needs to Store

Imagine that you are building an e-commerce backend.

Your system stores:

- Customer orders and payment records.
- Product images and downloadable invoices.
- Temporary files and application logs.
- Database backups.
- Files shared between application instances.

These are not all the same kind of data. They have different access patterns, durability requirements, and performance characteristics.

AWS provides different storage services for different jobs:

1. Amazon S3 — Object Storage

Store objects such as images, PDFs, exports, log archives, and backups. Applications access objects through APIs rather than treating the bucket as an ordinary disk.

2. Amazon EBS — Block Storage

Attach a volume to an EC2 instance and use it like a disk. Common uses include operating-system root volumes and workloads requiring low-latency block-device semantics.

3. Amazon EFS — Shared File Storage

A managed network file system that multiple compatible compute clients can access concurrently over NFS. Useful when applications genuinely need shared filesystem semantics across multiple instances.

4. Amazon RDS — Managed Relational Databases

Run supported relational database engines, including PostgreSQL, while AWS manages many operational tasks such as infrastructure provisioning, automated patching, backups, and configured failover capabilities.

Storage Service Mapping

Data Type	Access Pattern	Durability Needs	Recommended AWS Service
User avatars & invoice PDFs	REST API / SDK / HTTP	99.999999999% (11 9s)	Amazon S3
OS filesystem & runtime binaries	Local block storage	Instance-bound / snapshotable	Amazon EBS
Legacy CMS uploads shared by nodes	POSIX network filesystem	Multi-AZ shared concurrent	Amazon EFS
Orders, accounts, financial ledgers	ACID SQL transactions	Managed Multi-AZ / PITR	Amazon RDS (PostgreSQL)

The core principle is simple:

> **Core Principle:** Choose storage according to the data's access pattern and durability requirements, not merely according to which service you already know.

---

2. Object Storage versus Block Storage versus File Storage

A useful analogy:

- **Object storage:** A managed package-delivery system. You identify an object and retrieve or update it through an API.
- **Block storage:** A disk attached directly to a computer. The operating system and applications read and write raw blocks.

- **File storage:** A shared filing cabinet with directories and filenames that authorized clients can access through a filesystem interface.

Property	Amazon S3	Amazon EBS	Amazon EFS
Storage model	Objects (key-value + metadata)	Blocks (fixed-sized chunks)	Files and directories (POSIX)
Typical access	HTTP/REST API or AWS SDK	Attached virtual block device	Network filesystem (NFSv4)
Multiple application servers	Unlimited concurrent API access	Attached to one instance at a time (multi-attach has strict limits)	Concurrent access from thousands of instances
Typical use	Images, documents, exports, archives, data lakes	Instance OS disks, self-hosted databases	Shared application directories, CMS, batch processing
Filesystem semantics	No ordinary mounted filesystem by default	Yes, after formatting and mounting	Yes, fully POSIX compliant
Common mistake	Treating S3 like a normal POSIX disk	Assuming EBS automatically protects data from every failure	Using shared filesystem storage when object storage would be simpler

S3 is not a drop-in replacement for a filesystem, and EFS is not automatically the best choice for every shared file.

---

3. Amazon S3: Store Objects, Not Server-Local Files

Suppose your Spring Boot application accepts product images.

A basic design might be:

```
Client
  |
  v
Spring Boot API
  |
  | Upload object using an authorized AWS SDK
  v
Amazon S3 bucket
  |
  v
Object key and metadata recorded in PostgreSQL
```

Instead of storing each image inside an EC2 instance's local directory, the application uploads it to S3.

PostgreSQL stores the relevant business metadata, such as:

- Product ID.
- Object key.
- Content type (`image/webp`, `application/pdf`).
- Upload timestamp.
- Ownership or access-control information.

The actual image bytes live durably in S3.

What is a bucket?

A bucket is an S3 container for objects. Each object has a **key**, which identifies it within that bucket.

For example:

```
Bucket: shop-product-media
Object key: products/1042/front.webp
Object key: products/1042/side.webp
Object key: invoices/2026/order-8931.pdf
```

The slash-separated names are conventions for organizing object keys. They do not imply that S3 is an ordinary hierarchical filesystem.

What does S3 provide?

S3 offers object storage with durability and availability characteristics that depend on the service and storage class. Common capabilities include:

- **Object versioning:** Protects against accidental overwrites and deletions.
- **Lifecycle rules:** Automatically transitions older objects to cheaper tiers (e.g., S3 Standard $\to$ S3 Glacier) or expires them.
- **Encryption:** Server-side encryption with AWS managed keys (SSE-S3) or customer KMS keys (SSE-KMS).
- **Access policies:** Bucket policies and IAM policies for fine-grained authorization.
- **Event notifications:** Trigger AWS Lambda functions or SQS messages on object creation.
- **Retention controls:** S3 Object Lock for WORM (Write Once, Read Many) compliance.
- **Storage classes:** S3 Standard, S3 Intelligent-Tiering, S3 Standard-IA, S3 Glacier Instant Retrieval, S3 Glacier Flexible, and S3 Glacier Deep Archive.

These features solve different problems. Versioning can help recover overwritten objects, while lifecycle policies can transition or expire eligible objects according to configured rules.

S3 security basics

A production bucket should not be made public just because the application needs to upload files.

Use:

- **Block Public Access:** Keep all four Block Public Access settings enabled unless direct public website hosting is strictly required.
- **IAM roles:** Use instance profiles and narrowly scoped IAM permissions.
- **Bucket policies:** Enforce TLS-only transport (`aws:SecureTransport: "true"`).
- **Encryption:** Encrypt at rest and enforce HTTPS in transit.
- **Appropriate retention:** Apply lifecycle and versioning policies.
- **Logging and monitoring:** Enable S3 server access logging and AWS CloudTrail data events for sensitive workloads.

Avoid embedding long-lived AWS access keys in source code.

Direct uploads: a better scaling pattern

For large uploads, routing every byte through your Spring Boot API may waste application bandwidth, memory, and compute resources.

A common alternative is a **presigned S3 URL**:

```
1. Client asks API to start an upload (provides filename, content type, size).
2. API validates the request and authorizes the user.
3. API creates a short-lived presigned URL (e.g., valid for 15 minutes).
4. Client uploads bytes directly to S3 via HTTP PUT.
5. API verifies the resulting object and records metadata in PostgreSQL.
```

A presigned URL grants time-limited access to a specific operation under the permissions of the signer. Treat it as a bearer credential while it is valid.

The backend should still enforce ownership, allowed content types and sizes, expected object keys, and any required scanning or validation. A successful upload does not automatically mean the file is safe.

---

4. EBS: Persistent Block Storage for EC2

Consider an EC2 instance running a Linux operating system. Its root filesystem commonly resides on an EBS volume.

You can also attach additional EBS volumes for application data or supported workloads.

Important concepts:

- **Volume:** The network-attached block-storage resource (e.g., `gp3`, `io2`).
- **Snapshot:** A point-in-time, incremental backup of an EBS volume stored durably in S3.
- **Attachment:** The association between a volume and a supported instance in the same Availability Zone.
- **IOPS and throughput:** Performance characteristics that influence storage behavior (e.g., `gp3` baseline of 3,000 IOPS and 125 MB/s).
- **Encryption:** Transparent protection of volume data and subsequent snapshots using AWS KMS.

Why EBS is not the same as an instance

An instance and its storage have separate lifecycle considerations.

For example, an instance may be terminated while a separately configured data volume is retained. Conversely, a root volume may be deleted when the instance is terminated if its configuration specifies that behavior (`DeleteOnTermination = true`).

Never assume that terminating an instance either always deletes all attached storage or always preserves it. Inspect the actual configuration.

EBS snapshots

Snapshots can support recovery and creation of new volumes. They are not a complete backup strategy for every application.

For a database, a storage snapshot alone may not provide the required application-consistent recovery point unless the database and snapshot process are coordinated appropriately (e.g., freezing writes or flushing write buffers).

Database-native backups, transaction-log recovery, and tested restoration procedures may be necessary to satisfy business recovery requirements.

---

5. EFS: When Multiple Servers Need Shared Files

Suppose you have three EC2 instances behind an ALB.

A user uploads a file to a directory on EC2 A. A later request is routed to EC2 B, which does not have that local file.

This is a classic distributed-application problem.

Amazon EFS provides shared network filesystem access to compatible clients:

```
EC2 A ----\
           \
EC2 B -------> Shared Amazon EFS Filesystem (NFSv4)
           /
EC2 C ----/
```

All authorized clients can access files concurrently through standard POSIX file operations.

However, shared files introduce operational questions:

- Who can read or modify each file? (POSIX UID/GID ownership and permissions)
- What happens when two instances update the same file? (File locking and race conditions)
- Is the application's filesystem access pattern compatible with network storage latency?
- What are the latency and throughput implications?
- Does the workload actually require shared filesystem semantics?

For images, PDFs, and static downloadable content, S3 is often the more natural and cost-effective option. EFS is valuable when the application specifically needs a POSIX shared filesystem.

---

6. Amazon RDS: Managed Relational Databases

Now consider the Orders API.

The application needs transactions, relational constraints, joins, indexes, and SQL queries. PostgreSQL is a reasonable database choice.

You could install PostgreSQL directly on an EC2 instance, but then your team is responsible for many operational tasks:

- Database installation, configuration, and upgrades.
- Operating-system patching and security maintenance.
- Backup scheduling, verification, and retention.
- Storage provisioning, auto-scaling, and monitoring.
- Replication setup and failover automation.
- Disaster recovery testing.
- Database process crashes and host hardware failures.

Amazon RDS (Relational Database Service) manages much of the underlying database infrastructure and supports managed relational database engines, including PostgreSQL.

You remain responsible for database schema design, query performance, access control, application connection behavior, and selecting appropriate availability and recovery settings.

A basic architecture

```
Clients
   |
   v
Application Load Balancer (Public Subnets)
   |
   v
Spring Boot on EC2 instances (Private Application Subnets)
   |
   | Private database connection (Port 5432)
   v
Amazon RDS for PostgreSQL (Private Database Subnets)
```

The application instances should normally reach the database over private networking, with security-group rules that allow the required database traffic.

For PostgreSQL, the default port is **5432**, unless the database is configured differently.

RDS does not eliminate database engineering

Managed infrastructure does not automatically solve:

- Missing indexes or poor query planning.
- Slow queries consuming all CPU.
- Lock contention and deadlocks.
- Connection-pool exhaustion.
- Bad transaction boundaries and long-running transactions.
- Incorrect isolation level assumptions.
- Schema migration failures.
- Application-level data corruption.

RDS reduces some infrastructure-management work. It does not replace understanding PostgreSQL.

---

7. Multi-AZ Deployments versus Read Replicas

These concepts are easy to confuse in interviews and architecture discussions.

Multi-AZ: Availability and Failover

A Multi-AZ RDS deployment is designed to improve availability by maintaining a synchronous standby replica in a different Availability Zone.

- **Primary Goal:** High availability and automatic disaster recovery.
- **Replication:** Synchronous replication from primary to standby.
- **Traffic:** The standby does **not** serve ordinary read traffic; it is strictly a failover target.
- **Failover:** Automatic failover typically completes in 60–120 seconds. DNS endpoint updates automatically to point to the new primary.
- **Impact on writes:** Minor latency overhead on commits due to synchronous cross-AZ replication.

Read Replica: Read Scaling

A read replica is an asynchronous copy of the primary database designed to offload read-heavy query traffic.

- **Primary Goal:** Horizontal read scalability and performance.
- **Replication:** Asynchronous replication from primary to replica.
- **Traffic:** Replicas actively serve read-only queries (`SELECT`).
- **Failover:** Not automatically a synchronous failover standby; can be manually promoted to a standalone database if needed.
- **Replication lag:** Asynchronous replication means replicas can lag behind the primary, resulting in stale reads.

[!IMPORTANT]

Remember This Distinction:

- **Multi-AZ:** Primary goal is to improve database availability and automatic failover behavior (synchronous replication to standby; standby does not serve reads).

- **Read Replica:** Primary goal is to serve additional read traffic horizontally or support a planned cross-region replication/recovery strategy (asynchronous replication; serves active read queries).

Feature Comparison Matrix

Property	Multi-AZ Standby	Read Replica
Primary Goal	High availability & fault tolerance	Read scalability & reporting
Replication Type	Synchronous	Asynchronous
Serves Read Traffic?	No	Yes (Read-Only)
Failover Behavior	Automatic failover to standby	Manual promotion (or Aurora auto-failover)
Number of Copies	1 standby in another AZ	Up to 15 (RDS PostgreSQL)
Cross-Region Support	No (within same Region)	Yes (Cross-Region Read Replicas)
Performance Impact	Small write latency overhead	None on primary writes

Neither feature eliminates the need for regular backups.

---

8. Automated Backups and Point-in-Time Recovery

Imagine that a faulty deployment runs a SQL statement that accidentally deletes or truncates important tables.

High availability (Multi-AZ) keeps the database reachable, but it does **not** restore the deleted data—the destructive SQL query synchronously replicates to the standby immediately!

This is why **availability and recoverability are completely different properties**.

RDS automated backups support recovery within the configured backup retention period (1 to 35 days).

Point-in-Time Recovery (PITR)

RDS continuously archives database transaction logs (WAL in PostgreSQL) to S3 every 5 minutes. Using automated daily snapshots plus transaction logs, you can restore your database to any specific second within your retention window, up to the last 5 minutes.

A complete recovery strategy should consider:

- Automated backup retention periods.
- Manual snapshots for long-term archiving before major releases.
- Transaction-log and point-in-time recovery capabilities.
- Cross-Region backup replication for disaster recovery.
- KMS encryption-key access across regions/accounts.
- Restore duration and recovery validation procedures.
- Protection against accidental deletion (enable Deletion Protection on RDS).

> [!TIP]
>
> A backup that has never been tested in a restoration drill is an unverified recovery assumption.

---

9. Durability, Availability, RPO, and RTO

These four concepts appear frequently in production engineering and system-design interviews:

Definitions

- **Durability:** The probability that stored data survives without loss or corruption over time (e.g., S3's 99.999999999% durability).
- **Availability:** The percentage of time a system is operational and able to respond successfully to requests (e.g., 99.95% uptime).
- **Recovery Point Objective (RPO):** The maximum acceptable amount of data loss measured in time (e.g., "we can tolerate losing at most 5 minutes of data").
- **Recovery Time Objective (RTO):** The maximum acceptable duration of downtime before service is restored (e.g., "the database must be back online within 30 minutes").

Target Scenario

Suppose a business requires:

- **RPO:** 5 minutes.
- **RTO:** 30 minutes.

Your recovery strategy must aim to lose no more than five minutes of data and restore the service within thirty minutes under the agreed scenario.

These are targets, not guarantees that every AWS recovery operation will finish within the specified time. The architecture and recovery procedures must be tested against realistic failures.

Why availability does not equal durability

- **High Availability, Low Durability:** A database cluster is online and reachable, but an application bug executes `UPDATE orders SET status = 'CANCELLED'`. The database is 100% available, but data integrity is destroyed.
- **Low Availability, High Durability:** An earthquake disrupts power to an entire data center. The database is completely offline for 2 hours (0% availability), but every committed transaction is preserved on durable storage with 0 data loss once power returns.

Production systems must be designed for both.

Layer 2 — Practical Engineering & Workflows

10. A Sensible Storage Architecture for the Orders API

Use the following separation of responsibilities:

Data Element	Suggested AWS Service	Architecture Rationale
Orders, customers, inventory	RDS for PostgreSQL	Relational transactions, foreign keys, and ACID guarantees
Product images & avatars	Amazon S3	Highly durable object storage, direct CDN integration
Generated invoice PDFs	Amazon S3	Immutable document storage with presigned URL access
EC2 OS root disk	Amazon EBS (**gp3**)	Low-latency local block storage for OS and JVM binaries
Shared filesystem (legacy)	Amazon EFS (if required)	POSIX-compliant multi-instance shared directory
Database recovery points	RDS Backups & Snapshots	PITR point-in-time recovery and snapshot archiving

Do not put every kind of data into one service simply to reduce the number of services in your architecture.

Likewise, do not introduce EFS if S3 already satisfies the application's file-storage requirements.

---

11. Spring Boot Database Configuration

A Spring Boot application commonly receives its database configuration through environment variables or AWS Systems Manager Parameter Store:

```
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
    hikari:
      maximum-pool-size: 10
      minimum-idle: 5
      connection-timeout: 3000
      idle-timeout: 600000
      max-lifetime: 1800000
```

The example uses Spring Boot's default HikariCP connection pool settings.

The pool size is illustrative, not a universal production recommendation. Tune it using database capacity, instance count, request concurrency, transaction duration, and measured behavior.

A production configuration should also consider TLS requirements (`sslmode=verify-full`), credential rotation, secret retrieval via AWS Secrets Manager, connection validation, transaction timeouts, and database failover behavior.

Why the connection pool matters

Suppose:

- 4 EC2 instances run the application.
- Each instance allows up to 10 database connections.

The application tier could request up to:

> **Formula:** 4 × 10 = 40 connections

If the Auto Scaling Group scales out to 12 instances during a traffic surge, the theoretical pool maximum becomes:

> **Formula:** 12 × 10 = 120 connections

This does not mean every connection is always open. It means the potential database connection demand increases as the application tier scales.

The database's own capacity (`max_connections` parameter in PostgreSQL) and any connections used by administrative jobs, migration tools, or background workers must also be considered. If total pool demand exceeds `max_connections`, connections are refused and the API returns 500 errors.

---

12. Use the AWS SDK for Java to Upload to S3

A simplified example using AWS SDK for Java v2:

```
package com.example.orders.storage;

import software.amazon.awssdk.core.sync.RequestBody;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.PutObjectRequest;

import java.nio.file.Path;

public class ObjectStorageService {
    private final S3Client s3;

    public ObjectStorageService(S3Client s3) {
        this.s3 = s3;
    }

    public void upload(String bucket, String key, Path file) {
        PutObjectRequest request = PutObjectRequest.builder()
                .bucket(bucket)
                .key(key)
                .contentType("image/webp")
                .build();

        s3.putObject(request, RequestBody.fromFile(file));
    }
}
```

This illustrates the upload operation. A production implementation should also address:

- Bucket and key authorization.
- Input validation and file size limits.
- Error handling and bounded retries with exponential backoff.
- Upload timeouts and cancellation.
- Content type and required metadata tags.
- Server-side encryption headers (`ServerSideEncryption.AWS_KMS`).
- Safe handling of partial or duplicate application workflows.
- Multipart-upload behavior for files larger than 100 MB.

Configure the SDK to use its default credential provider chain (`DefaultCredentialsProvider.create()`). In AWS, an IAM instance profile or container task role is strongly preferred over credentials embedded in code.

---

13. AWS CLI: Inspect Storage and Database Resources

These commands are read-only inspection commands. They assume that you have the AWS CLI configured for the intended account and Region and that your identity has the required permissions.

List S3 buckets

```
aws s3api list-buckets \
  --query 'Buckets[].Name' \
  --output table
```

Inspect objects in a specific bucket

```
aws s3api list-objects-v2 \
  --bucket YOUR_BUCKET_NAME \
  --max-items 20
```

List RDS instances

```
aws rds describe-db-instances \
  --query 'DBInstances[].{
    Identifier:DBInstanceIdentifier,
    Engine:Engine,
    Status:DBInstanceStatus,
    MultiAZ:MultiAZ,
    Storage:AllocatedStorage,
    Endpoint:Endpoint.Address
  }' \
  --output table
```

Inspect automated backups

```
aws rds describe-db-instances \
  --query 'DBInstances[].{
    Identifier:DBInstanceIdentifier,
    BackupRetentionDays:BackupRetentionPeriod,
    BackupWindow:PreferredBackupWindow
  }' \
  --output table
```

Inspect RDS snapshots

```
aws rds describe-db-snapshots \
  --query 'DBSnapshots[].{
    Snapshot:DBSnapshotIdentifier,
    Status:Status,
    Created:SnapshotCreateTime,
    Source:DBInstanceIdentifier
  }' \
  --output table
```

Inspect EBS volumes

```
aws ec2 describe-volumes \
  --query 'Volumes[].{
    ID:VolumeId,
    State:State,
    SizeGiB:Size,
    Encrypted:Encrypted,
    AZ:AvailabilityZone
  }' \
  --output table
```

These commands inspect resources. They do not create, modify, delete, or restore infrastructure.

14. How to Reason About an S3 Upload Failure

Suppose users cannot upload product images.

Work through the request path systematically:

1. **Verify API receipt:** Confirm that the API received the upload request and logged the entry point.
2. **Inspect SDK exception:** Check application logs for the specific AWS SDK exception class (`S3Exception`, `SdkClientException`).
3. **Verify bucket & Region:** Confirm the bucket name exists and the client is querying the correct AWS Region.
4. **Confirm IAM identity:** Confirm the active workload identity is the expected IAM instance role via STS (`aws sts get-caller-identity`).
5. **Review permissions & policies:** Verify the IAM policy allows `s3:PutObject` on `arn:aws:s3:::bucket-name/*`. Check if bucket policies contain explicit `Deny` statements.
6. **Check KMS key permissions:** If using SSE-KMS, ensure the instance role has `kms:GenerateDataKey` and `kms:Decrypt` permissions on the KMS key.
7. **Check network & routing:** If running in a private subnet, ensure there is an active S3 Gateway VPC Endpoint or NAT Gateway route.
8. **Verify idempotency & retries:** Ensure partial uploads do not leave broken database records.

A `403 Forbidden` response indicates access restrictions, but the exact cause depends on IAM, bucket policies, KMS keys, or organization SCPs (Service Control Policies).

---

15. How to Reason About a Database Connection Failure

Suppose the API starts returning database connection errors (`Connection refused`, `Connection timeout`).

Check in this order:

1. **Database status:** Is the RDS instance in the `available` state, or is it modifying, rebooting, or failing over?
2. **Configuration accuracy:** Is the application using the correct RDS DNS endpoint, port (5432), database name, and credentials?
3. **Security groups:** Does the RDS security group contain an inbound rule allowing TCP 5432 from the EC2 security group ID?
4. **Subnet & routing:** Are application and database subnets within the same VPC, and do route tables allow local communication?
5. **Connection pool exhaustion:** Has HikariCP exhausted its maximum pool size? Check `ActiveConnections` vs. `max_connections` in RDS metrics.
6. **Database CPU & memory:** Is the database thrashing due to memory exhaustion (OOM killer) or CPU saturation?
7. **Failover recovery:** Did a Multi-AZ failover recently occur? Were stale connection pool sockets purged?
8. **Timeouts:** Are acquisition and query timeouts configured reasonably?

Separate a network failure (timeout) from an authentication failure (`FATAL: password authentication failed`) and from pool exhaustion (`Connection is not available, request timed out`).

---

16. A Database Failover Workflow

Imagine the primary database experiences a failure in a configured Multi-AZ RDS deployment.

The application will experience connection errors while failover takes place (typically 60–120 seconds).

A resilient application should:

```
1. Detect broken connections -> HikariCP marks connection as invalid and evicts from poo
l
2. Bounded backoff retry -> Application pauses brief intervals before attempting new con
nections
3. DNS resolution refresh -> Java/JVM DNS cache respects low TTL (<= 5 seconds) to resol
ve new primary IP
4. Reconnection handshake -> New TCP connection established to promoted primary standby
5. Idempotent retry logic -> Safely retry read queries or idempotent writes with idempot
ency keys
6. Resume normal traffic -> Alert operators and monitor error rates until steady state i
s restored
```

A database failover does not guarantee that an in-flight transaction completed. For a payment or order-creation request, the client may not know whether the transaction committed before the connection failed.

This is an application-level correctness problem, not just a networking problem.

An **idempotency key** or deduplication table can help prevent a client retry from creating duplicate orders or double payments.

---

17. Backup Restoration Lab

Do not perform a restore against production just to learn the workflow.

Use a disposable test environment or a documented tabletop exercise.

Exercise Scenario

Your database contains order records. A deployment script accidentally deletes orders created during the previous hour.

Define:

1. **Available recovery mechanisms:** Automated daily snapshots + continuous WAL transaction logs (PITR).
2. **Business RPO target:** Maximum 5 minutes of data loss.
3. **Business RTO target:** Maximum 30 minutes of downtime.
4. **Restoration procedure:** Restore the snapshot to a **separate, new database instance** (`orders-db-restored`) at the exact minute prior to the deletion script execution.
5. **Data validation:** Connect to `orders-db-restored`, verify table row counts and schema integrity, and export the deleted rows.
6. **Approval & reconciliation:** Obtain authorization from the lead architect/DBA. Re-insert the missing rows into the live production database (`orders-db-prod`) without overwriting new transactions created in the interim.
7. **Teardown:** Terminate `orders-db-restored` once validation and reconciliation are complete to avoid ongoing infrastructure costs.

> [!WARNING]
>
> Never assume that restoring a snapshot is equivalent to replacing a live database. In a live production system, simply swapping endpoints causes all writes that occurred between the incident and the restore time to be lost forever!

Layer 3 — Production Engineering & Interview Mastery

18. Production Incident Simulations

Incident A: Product images disappear after an EC2 deployment

Symptoms: Images uploaded yesterday return 404 errors after the Auto Scaling Group replaces instances during an application update.

- **Root Cause:** The application was configured to write uploaded files to a local directory (e.g., `/var/app/uploads/`) on the instance's ephemeral or EBS root disk. When instances were replaced, the local disks were destroyed.
- **Resolution:**

1. Refactor the application service to upload files to an S3 bucket using the AWS SDK.
2. Store only the S3 bucket name and object key in PostgreSQL.
3. Update existing records and verify all active instances read from S3.
4. Enable S3 Versioning and Default Encryption on the bucket.

Incident B: Database connections are exhausted

Symptoms: During a flash sale, API response times spike to 30 seconds and the application logs display:

```
java.sql.SQLTransientConnectionException: HikariPool-1 - Connection is not available, request timed out after 3000ms.
```

- **Root Cause:** The Auto Scaling Group scaled from 2 to 10 instances. Each instance had `maximum-pool-size: 30`, creating up to 300 concurrent connection requests. RDS PostgreSQL instance type was `db.t4g.small` with `max_connections = 150`.
- **Resolution:**

1. Immediately reduce `maximum-pool-size` to 10 per instance in configuration.
2. Implement **Amazon RDS Proxy** between the application instances and RDS to pool and multiplex database connections efficiently.
3. Add CloudWatch alarms on RDS `DatabaseConnections` metric at 80% of `max_connections`.
4. Evaluate resizing the RDS instance class if CPU or query load warrants it.

Incident C: Multi-AZ failover causes a persistent API outage

Symptoms: RDS completed automatic Multi-AZ failover within 90 seconds, but the Spring Boot application continues to throw connection exceptions for over 30 minutes.

- **Root Cause:** The JVM permanently cached the DNS resolution of the RDS endpoint to the old primary IP address (`networkaddress.cache.ttl` was set to `-1` or default infinite).
- **Resolution:**

1. Configure the JVM DNS cache TTL to a low value (e.g., 5 to 10 seconds):

```
-Dnetworkaddress.cache.ttl=10
```

2. Configure HikariCP `maxLifetime: 1800000` (30 minutes) and `connectionTimeout: 3000`.
3. Restart application instances to purge stale JVM DNS cache.

Incident D: A backup exists, but recovery fails

Symptoms: During a disaster recovery drill, the team attempts to restore a snapshot to a new region, but the operation fails.

- **Root Cause:** The snapshot was encrypted using the default AWS managed KMS key (`aws/rds`), which cannot be shared or used directly across regions.
- **Resolution:**

1. Re-encrypt snapshots using a Customer Managed Key (CMK) configured for multi-region replication.
2. Rehearse the snapshot copy and restore process monthly in automated testing.
3. Verify that IAM restore permissions include the necessary KMS decrypt and create-grant rights.

Incident E: A database is available but important data is missing

Symptoms: The API responds with HTTP 200, but orders created by users 30 minutes ago are missing from database tables.

- **Root Cause:** An engineer erroneously restored a snapshot directly over the production database or promoted an outdated read replica that had 30 minutes of replication lag.
- **Resolution:**

1. Lock down destructive RDS actions using IAM Service Control Policies (SCPs) and require dual-operator approval.
2. Enable Point-in-Time Recovery to recover lost transactions from transaction logs.
3. Always restore to a temporary validation instance first, extract the delta, and merge.

---

19. Advanced Design Trade-Offs

Should the application store files in PostgreSQL or S3?

For large files and user-generated media, S3 separates object storage from transactional metadata and prevents bloat in database backups, buffers, and replication streams.

PostgreSQL can still be appropriate for small binary data in some applications. The decision depends on transaction requirements, file size, query patterns, backup strategy, and operational complexity.

A common design stores file metadata and references in PostgreSQL while storing the bytes in S3.

This creates a **cross-service consistency problem**: the database transaction and S3 upload do not form one atomic distributed transaction. Your workflow must handle partial failures:

- *Failure mode:* S3 upload succeeds, but database transaction rolls back $\to$ Orphaned S3 object. Clean up via scheduled garbage-collection or S3 lifecycle rules.
- *Failure mode:* Database record written before upload, upload fails $\to$ Database points to non-existent object. Solved by uploading to S3 first with a temporary status, and confirming in DB only upon upload success.

Should we use RDS or run PostgreSQL on EC2?

- **RDS is preferred** when the team values automated backups, automated multi-AZ failover, push-button scaling, managed security patching, and reduced operational overhead.
- **Self-managed PostgreSQL on EC2** is appropriate only when the workload requires unsupported operating-system access, custom PostgreSQL extensions not approved by AWS, specialized hardware, or when an organization already employs a dedicated, specialized DBA team.

Should we add a read replica?

A read replica helps when read-heavy queries saturate database CPU and the application can tolerate **eventual consistency** (replication lag).

It will **not** improve write throughput. It also introduces the risk of stale reads: if a user submits an order and immediately redirects to an order confirmation page querying a lagging read replica, the user may see an empty page.

Is Multi-AZ enough for disaster recovery?

No. Multi-AZ protects against a single Availability Zone failure within a single AWS Region. It does **not** protect against:

- Catastrophic regional AWS outages.
- Accidental table truncation or malicious SQL injection.
- Application-level bugs that corrupt data.

Comprehensive disaster recovery requires automated backups, point-in-time recovery, cross-region snapshot replication, and documented, tested runbooks with explicit RPO and RTO metrics.

---

20. FAANG-Style Interview Questions and Answers

Q1. What is the difference between S3, EBS, and EFS?

S3 is object storage accessed via HTTP APIs for files, media, and backups. EBS is block storage attached directly to an EC2 instance as a virtual hard disk. EFS is an elastic, shared network file system accessible concurrently by multiple EC2 instances via NFS.

Q2. Why should application instances avoid storing durable uploads locally?

EC2 instances in an Auto Scaling Group are ephemeral. If an instance terminates or scales in, local storage is destroyed. Externalizing state to S3 ensures persistence and makes instances fully interchangeable.

Q3. What is an S3 bucket?

A globally unique container for storing objects in Amazon S3. Objects are identified within the bucket by unique alphanumeric keys and optional version IDs.

Q4. What is a presigned URL?

A cryptographically signed URL generated by an authorized entity that allows temporary, direct upload or download access to an S3 object without exposing AWS credentials to the client.

Q5. What is EBS used for?

EBS provides low-latency block storage volumes for EC2 instances, typically used for operating system filesystems, application runtimes, and local scratch space.

Q6. When would you choose EFS over S3?

When legacy applications or multi-instance workloads strictly require standard POSIX filesystem semantics, directory traversal, file locking, and immediate consistency across multiple running servers.

Q7. What does Amazon RDS manage?

RDS manages OS and database installation, automated minor version patching, automated backups, point-in-time recovery, disk auto-scaling, and synchronous Multi-AZ failover.

Q8. What is Multi-AZ RDS?

A high-availability feature where RDS synchronously replicates database writes to an active standby instance in a different Availability Zone. If the primary fails, RDS automatically fails over to the standby.

Q9. How does a read replica differ from a Multi-AZ standby?

A Multi-AZ standby is synchronous, does not serve read traffic, and is used strictly for automatic failover. A read replica is asynchronous, actively serves read traffic to scale queries, and does not provide automatic synchronous failover.

Q10. Why do read replicas introduce consistency concerns?

Because replication is asynchronous, replicas may lag behind the primary by milliseconds or seconds, leading to stale reads (reading uncommitted or delayed data).

Q11. What is point-in-time recovery?

An RDS feature that combines daily storage snapshots and 5-minute transaction log (WAL) backups to restore a database instance to any specific second within the backup retention period.

Q12. What is the difference between availability and durability?

Availability measures the percentage of time a service is reachable and responsive to requests. Durability measures the probability that stored data is preserved without corruption or loss over time.

Q13. What is RPO?

Recovery Point Objective: the maximum acceptable amount of data loss measured in time between the last recovery point and a disaster event.

Q14. What is RTO?

Recovery Time Objective: the maximum acceptable duration of clock time to restore a system and resume operations after an outage.

Q15. Why are backups necessary if the database has Multi-AZ enabled?

Multi-AZ replicates errors, corruption, and accidental deletions synchronously to the standby. Backups are required to recover historical states before the damaging operation took place.

Q16. Why can a database failover break application requests?

Existing TCP sockets are abruptly severed during failover. Applications must have robust connection pool eviction, bounded retries, low JVM DNS TTL, and idempotent request handling to recover gracefully.

Q17. How can scaling EC2 instances overload RDS?

Each new EC2 instance creates its own pool of open database connections. If the aggregate connections from all instances exceed `max_connections`, the database refuses new connections, crashing the API.

Q18. What is the risk of storing file metadata in PostgreSQL and bytes in S3?

There is no two-phase commit across S3 and RDS. If an S3 upload succeeds but the subsequent database commit fails, an orphaned S3 object is created. Systems must implement compensation logic or reconciliation jobs.

Q19. How would you test a database recovery strategy?

Perform a non-disruptive disaster recovery drill: restore a production snapshot to a test database instance, run validation queries to verify schema and record integrity, measure the elapsed time against RTO, and calculate data delta against RPO.

Q20. How would you design storage for an e-commerce backend?

Store transactional order records in Amazon RDS PostgreSQL (Multi-AZ enabled with automated backups); store product images, videos, and generated invoice PDFs in Amazon S3; use CloudFront CDN for caching static assets; and attach EBS `gp3` volumes for instance operating systems.

---

21. Mastery Challenge

Design the data layer for an e-commerce API with these requirements:

- Orders and inventory must use transactional relational storage.
- Customers upload product images.
- The application runs across multiple EC2 instances in an Auto Scaling Group.
- Database availability is critical.
- Accidental deletion must be recoverable.
- Database connections must remain within a safe budget.
- Recovery targets are RPO 5 minutes and RTO 30 minutes.

Complete these tasks:

1. **Storage service for product images:** Choose Amazon S3 with SSE-S3 encryption and private bucket access.
2. **Database service and engine:** Choose Amazon RDS for PostgreSQL (Multi-AZ deployment).
3. **Why EC2-local files fail for shared uploads:** EC2 instances are ephemeral and dynamically scaled; local disks are destroyed on termination and cannot be shared across multiple instances.
4. **Roles of Multi-AZ and backups:** Multi-AZ provides automatic failover for high availability during hardware/AZ failures. Automated backups and PITR provide recovery from data corruption, accidental deletion, and disaster recovery.
5. **Failover effect on pooled connections:** Existing pool connections break with socket errors. The pool must evict broken connections and reconnect to the new primary using the updated DNS record.
6. **Connection pool calculation:** 6 instances $\times$ pool size 12 = **72 potential connections**. Must ensure `max_connections` on RDS exceeds 72 plus administrative headroom.
7. **Presigned upload URLs:** The backend issues a short-lived presigned PUT URL. The backend must enforce authentication, validate file size/type constraints, and verify upload completion before persisting the reference.
8. **Restoring a backup safely:** Restore to a separate new RDS instance, run validation queries, extract missing data, and merge into production.
9. **Achieving RPO 5m and RTO 30m:** Requires enabling automated continuous transaction log archiving (WAL to S3), testing automated restore scripts, and practicing runbook execution.
10. **Handling partial S3/DB failures:** Implement a two-step commit pattern: upload to S3 first with a `PENDING` metadata state, commit to PostgreSQL, and finalize or purge unreferenced S3 objects via an automated cleanup worker.

Self-assessment

Score each item from 0 to 2:

Skill	Score
S3, EBS, and EFS distinctions	/2
RDS fundamentals and connectivity	/2
Multi-AZ versus read replicas	/2
Backups and point-in-time recovery	/2
RPO and RTO	/2
Connection-pool capacity	/2
Storage and IAM security	/2

Skill	Score
Incident diagnosis and recovery	/2

Target: At least 13/16, with no zero in backups, database connectivity, or recovery planning.

Chapter Completion Protocol

Chapter: AWS Chunk 4 of 6

Save as: `content/aws/04-aws-storage-s3-ebs-efs-rds-backups-high-availability.md`

Do not generate a PDF yet. Once all six AWS chapters have been saved and verified, generate the single final AWS handbook:

`docs/aws/BEH_AWS_COMPLETE_HANDBOOK.pdf`

After the full source set and final PDF are verified, the intended Git commit message is:

```
AWS done
```

Next chapter: AWS Chunk 5 of 6 — Containers, Serverless, Messaging & Event-Driven Architecture. We will compare ECS/Fargate, Lambda, SQS, SNS, and EventBridge, with practical backend examples and production trade-offs.

Part V: Containers, Serverless, Messaging & Event-Driven Architecture

Chunk 5 of 6 · Backend Engineering Handbook · Cloud Engineering Track

AWS — Chunk 5 of 6: Containers, Serverless, Messaging & Event-Driven Architecture

Backend Engineering Tools & Infrastructure Handbook · Cloud Engineering Track

Source file: `content/aws/05-aws-containers-serverless-messaging-event-driven-architecture.md`

Learning Objectives

By the end of this chapter, you should be able to:

- Understand Amazon ECS, AWS Fargate, and AWS Lambda.
- Choose between EC2-based deployment, managed containers, and serverless functions.
- Explain SQS, SNS, and EventBridge without confusing their responsibilities.
- Design asynchronous workflows for a Spring Boot backend.
- Handle retries, duplicate messages, dead-letter queues, and idempotency.
- Understand event-driven architecture failure modes and operational trade-offs.
- Answer backend interview questions about cloud-native application architecture.

Layer 1 — Fundamentals & Mental Models

1. Three Ways to Run Backend Code on AWS

In the previous chapters, we ran a Spring Boot API on EC2. That works, but operating virtual machines is not the only way to deploy backend software.

Consider three primary approaches:

Option A — Amazon EC2 (Virtual Machines)

You manage virtual machines and the application runtime. You have substantial control over the guest operating system, deployment process, filesystem, and instance configuration.

Option B — Amazon ECS with AWS Fargate (Managed Containers)

Package the application as a Docker container image and run it as a managed task. AWS manages the underlying server infrastructure for Fargate tasks, while you configure container resources, networking, task definitions, and deployment behavior.

Option C — AWS Lambda (Serverless Functions)

Run code in response to configured events or requests without managing servers or long-running daemons. AWS manages the execution infrastructure, autoscaling, and concurrency, while you remain responsible for code, permissions, configuration, and application behavior.

The Central Trade-Off

You are not choosing between a "good" architecture and a "bad" architecture. You are choosing how much infrastructure control and operational responsibility your workload requires:

Consideration	Amazon EC2	Amazon ECS with Fargate	AWS Lambda
Primary abstraction	Virtual machine	Container task or service	Function invocation
OS management	You manage guest OS & patches	AWS manages underlying Fargate OS	AWS manages execution environment
Deployment unit	Application on a machine	Container image (Docker/OCI)	Function zip package or container image
Long-running service	Yes (Daemon / Systemd)	Yes (Long-running service)	No (Max 15-minute execution limit)
Request/event-driven	Possible (always listening)	Possible (always listening)	Core native model
Scaling	Auto Scaling Groups (minutes)	ECS service autoscaling (tasks)	Concurrency scaling (milliseconds)
Operational control	High (full root / kernel)	Container-level control	Constrained runtime sandbox
Typical concern	OS patching & idle compute costs	Task sizing & networking config	Cold starts, concurrency limits, timeouts

The appropriate choice depends on the workload, team expertise, reliability requirements, deployment constraints, and total cost.

---

2. Amazon ECS: Running Containers as a Service

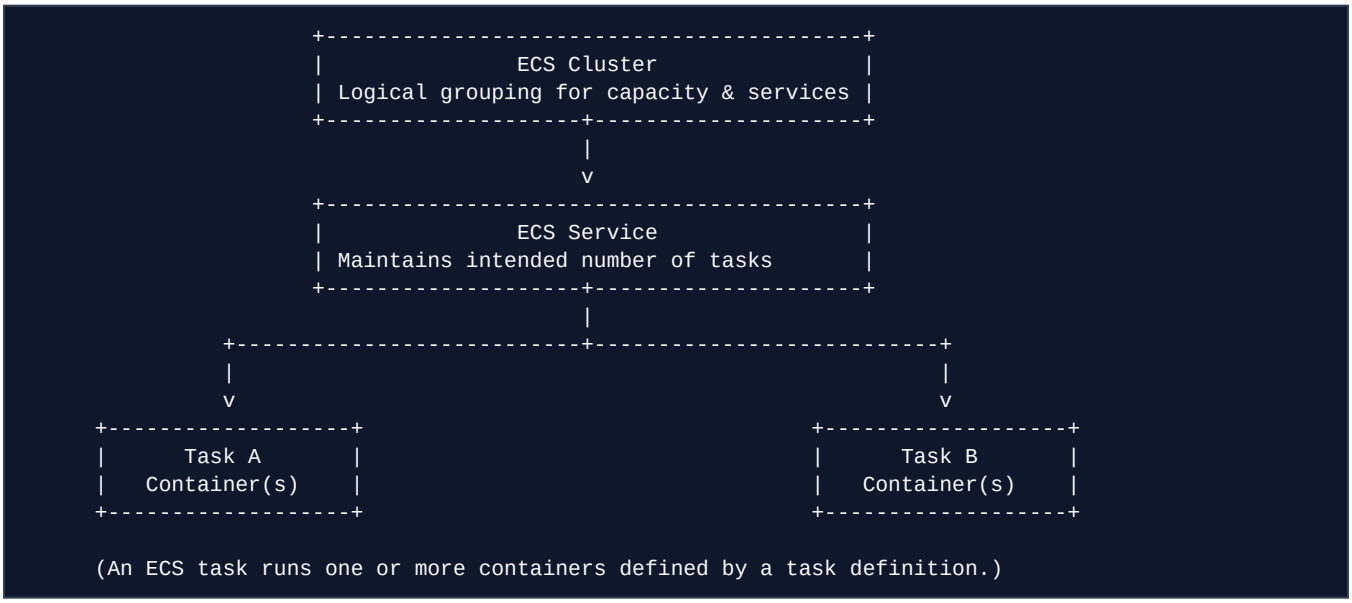
Suppose your Orders API has a Docker image:

```
orders-api:1.4.0
```

You want multiple copies running behind an Application Load Balancer, but you do not want to manually start each container, configure networking, and replace failed copies yourself.

Amazon Elastic Container Service (ECS) is a fully managed container orchestration service that manages the deployment, lifecycle, and scaling of containers according to a desired configuration.

ECS Mental Model



Important ECS Concepts

- **Cluster:** A logical grouping of ECS workloads, tasks, and compute capacity.
- **Task definition:** A declarative JSON blueprint describing container images, CPU and memory allocation, environment variables, secrets, ports, logging, and IAM roles.
- **Task:** A running instance of a task definition (equivalent to a Kubernetes Pod).
- **Service:** Maintains a desired number of tasks, integrates with an Application Load Balancer, and manages rolling deployment and task replacement behavior.
- **Container image:** The packaged application and its runtime dependencies stored in Amazon ECR (Elastic Container Registry).
- **Task role:** IAM permissions granted to the application code executing inside the container (e.g., access to S3 or DynamoDB).
- **Task execution role:** IAM permissions granted to the ECS container agent to pull private images from ECR and stream logs to CloudWatch.

> [!IMPORTANT]
>
> Do not confuse the **Task Role** with the **Task Execution Role**. The Execution Role belongs to the AWS infrastructure agent (pull image, send logs); the Task Role belongs to your application code.

ECS on EC2 versus ECS on Fargate

ECS can use two distinct compute launch types:

- **ECS on EC2:** You manage the EC2 instances providing container capacity. You handle host OS maintenance, Docker daemon updates, capacity planning, and scaling policies.
- **ECS on Fargate:** AWS manages the underlying virtual machine compute infrastructure. You simply specify CPU and memory per task (e.g., 0.5 vCPU, 1 GB RAM). Fargate eliminates server management, host patching, and cluster capacity management.

Fargate reduces host-management work. It does not eliminate the need to operate and monitor the application.

---

3. What Is AWS Lambda?

AWS Lambda is a serverless compute service that executes code in response to supported invocation sources without provisioning or managing servers.

For example, you might want to generate a thumbnail whenever an image is uploaded to S3:

```
User uploads image
       |
       v
   Amazon S3
       |
       | Object-created event (s3:ObjectCreated:*)
       v
   AWS Lambda
       |
       v
Thumbnail generation
       |
       v
Save thumbnail to S3 (processed-media/)
```

The application does not need to keep a dedicated server running 24/7 solely to wait for occasional image-upload events.

Ideal Use Cases for Lambda

- Event-driven processing (S3 uploads, DynamoDB Streams).
- Lightweight data transformations and ETL pipelines.
- Scheduled cron jobs (via EventBridge Scheduler).
- Webhooks and lightweight REST API handlers.
- Queue consumers (SQS) and stream processors (Kinesis).
- Operational automation and glue code between AWS services.

Hard Constraints and Operational Limits

Lambda has specific service-level boundaries you must plan around:

- **Maximum execution duration:** 15 minutes per invocation (900 seconds).
- **Memory allocation:** 128 MB to 10,240 MB (CPU scales proportionally with memory).
- **Ephemeral scratch disk:** 512 MB baseline up to 10 GB (`/tmp`).
- **Cold start latency:** The time required to initialize a new execution environment (download code, boot runtime, load framework). Can be noticeable for JVM-based Spring Boot applications.
- **Concurrency limit:** Default regional account limit (e.g., 1,000 concurrent executions), expandable via service quota requests.

Lambda versus a Continuously Running Spring Boot API

A traditional Spring Boot API is typically a continuously running process that listens on a TCP port for HTTP requests.

A Lambda function is invoked on-demand through an event bridge or API Gateway, executes within a sandbox, and suspends or tears down when idle.

While frameworks like AWS Serverless Java Container allow Spring Boot to run on Lambda, large frameworks incur substantial cold-start penalties (several seconds). For high-throughput, low-latency, complex REST APIs, running containers on ECS Fargate is generally more predictable and cost-effective than forcing Spring Boot into serverless functions.

---

4. SQS: A Queue That Decouples Work

Imagine that a user places an order.

The API needs to save the order in PostgreSQL, send an order confirmation email, and notify an external fulfillment warehouse.

If the API performs every operation synchronously, the client request becomes slow and fragile. If the third-party email provider experiences a 3-second delay or temporary downtime, an otherwise valid order fails or times out.

A queue lets you separate the immediate business transaction from asynchronous background work.

Amazon Simple Queue Service (SQS) is a fully managed, durable message queuing service:

```
Order API
   |
   | Send message ("OrderPlaced")
   v
Amazon SQS Queue
   |
   v
Background Worker
   |
   +----> Send confirmation email
   |
   +----> Dispatch order to fulfillment partner
```

The API enqueues work and returns `202 Accepted` or `201 Created` immediately. Consumers process the backlog asynchronously at their own sustained pace.

The Queue Mental Model

Think of SQS as a durable, distributed work inbox:

```
1. Producer sends message -> SQS writes message redundantly across multiple AZs
2. Consumer polls queue -> Receives message via ReceiveMessage API
3. Visibility Timeout begins -> Message is hidden from other workers for N seconds
4. Consumer processes work -> Sends email, calls third-party APIs
5. Consumer deletes message -> Explicit DeleteMessage API call removes it permanently
```

If the consumer crashes or processing fails before `DeleteMessage` is invoked, the visibility timeout expires and the message reappears in the queue for another consumer to process.

> **Golden Rule of Queues:** Always design consumers to tolerate duplicate message processing!

Standard versus FIFO Queues

Characteristic	Standard Queue	FIFO Queue (`.fifo`)
Message Ordering	Best-effort ordering (may arrive out of order)	Strictly First-In-First-Out within Message Group ID
Delivery Guarantee	At-least-once delivery (duplicates possible)	Exactly-once processing (within 5-minute deduplication interval)
Throughput	Virtually unlimited transactions per second	300 to 3,000 transactions per second (batching)
Deduplication	Application must implement idempotency	Content-based deduplication or deduplication ID
Common Use	Independent tasks, image resizing, emails	Financial transactions, inventory ledgers, state machines

A FIFO queue does not automatically make an entire distributed business workflow "exactly once." If a worker updates a database, encounters a network timeout, and crashes, retries can still occur. Application idempotency remains mandatory.

---

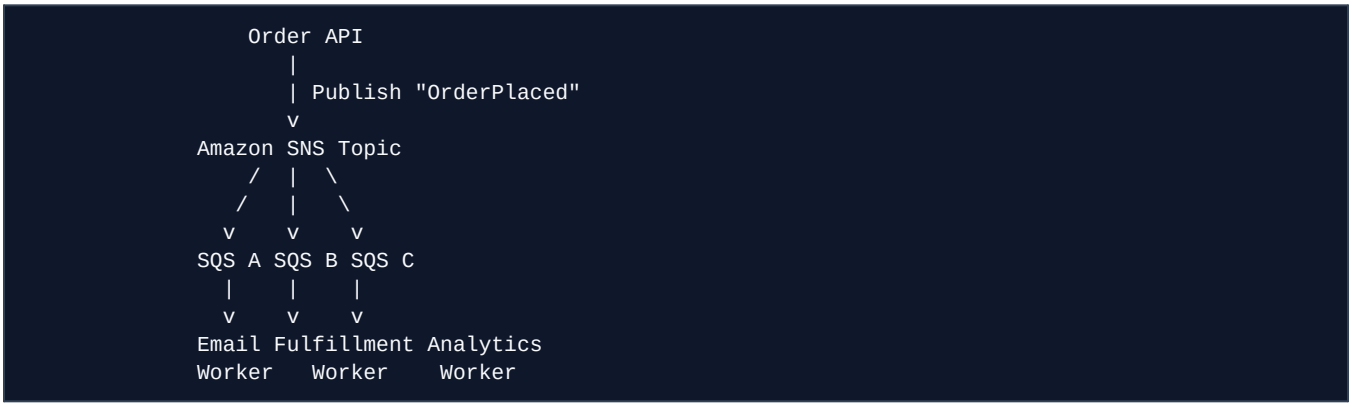
5. SNS: Publish a Message to Multiple Subscribers

Suppose placing an order requires notifying several independent internal and external systems:

- Email worker (sends customer receipts).
- Fulfillment worker (generates packing slips).
- Analytics pipeline (records revenue metrics).
- Fraud detection service (runs ML scoring).

If the Order API sends an SQS message directly to each worker's individual queue, the producer becomes tightly coupled to knowing every downstream consumer.

Amazon Simple Notification Service (SNS) provides a managed publish-subscribe (pub/sub) fan-out model:



The publisher emits one message to the SNS topic. SNS fans out the message to all subscribed endpoints, including SQS queues, HTTP webhooks, Lambda functions, or email addresses.

Why Combine SNS with SQS (Fan-Out Pattern)?

Publishing directly to HTTP webhooks or email endpoints lacks buffering. If an external subscriber is down, notifications are dropped or lost after retries.

By subscribing dedicated SQS queues to an SNS topic:

- **Isolated buffering:** If fulfillment takes 10 minutes to process a batch, its queue buffers messages without slowing down email or analytics.
- **Fault isolation:** An outage in the analytics service has zero impact on order fulfillment.
- **Independent retry policies:** Each queue configures its own visibility timeout, dead-letter queue, and retry limits.
- **Zero producer changes:** Adding a new fraud detection service requires only subscribing a new queue to the SNS topic; zero code changes are needed in the Order API.

---

6. EventBridge: Route Events Based on Their Meaning

Amazon EventBridge is a serverless event bus service that routes events between applications, integrated SaaS services (e.g., Zendesk, Stripe, Auth0), and AWS services using declarative rules.

Consider an e-commerce platform emitting diverse domain events:

```
OrderPlaced
PaymentAuthorized
PaymentFailed
ShipmentDispatched
CustomerAccountLocked
```

Different consumers care about different event types and specific payload attributes.

EventBridge evaluates event patterns (JSON rules) to route matching events:

```
Services and SaaS Event Sources
              |
              v
     Amazon EventBridge Bus
              |
      Declarative Rules & Filtering
          /      |       \
         v       v        v
      Lambda    SQS      Step Functions
      Worker   Queue     Workflow
```

Example EventBridge Rule Pattern

Filter only orders where payment succeeded and total exceeds $500:

```
{
  "source": ["ecommerce.orders"],
  "detail-type": ["OrderPlaced"],
  "detail": {
    "status": ["PAID"],
    "totalAmount": [{ "numeric": [">", 500] }]
  }
}
```

SQS vs. SNS vs. EventBridge: Deciding Which to Use

Dimension	Amazon SQS	Amazon SNS	Amazon EventBridge
Core Abstraction	Point-to-point work queue	Pub/sub topic fan-out	Event bus with content-based routing
Delivery Model	Consumer pulls (polls)	Push to subscribers	Push to targets
Filtering	None (consumer filters)	Basic message attributes	Deep payload JSON content filtering
Latency	Milliseconds	Sub-second	~100–500 milliseconds
SaaS Integrations	No direct SaaS integration	No direct SaaS integration	Native partner integrations (Stripe, Datadog)
Schema Registry	No	No	Built-in schema discovery & registry
Best Used For	Work buffering, load leveling	High-throughput fan-out to queues	Cross-service enterprise event routing

---

7. What Is an Event-Driven Architecture?

In a **request-driven (synchronous)** architecture, services communicate via direct blocking HTTP/REST or RPC calls:

```
Client -> Order Service -> Inventory Service -> Payment Gateway -> Notification Service
```

- *Disadvantages:* High coupling, cascading failures (if payment is slow, the entire chain stalls), higher latency (sum of all dependencies), and fragile error handling.

In an **event-driven (asynchronous)** architecture, services communicate by producing and consuming events representing past occurrences ("facts"):

```
Order Service -> [Emits "OrderPlaced"] -> Event Router -> Consumers react independently
```

Event-Driven Benefits and Operational Realities

- **Benefits:** Loosely coupled services, independent scaling, resilient to temporary downstream outages, and simpler addition of new features.
- **Challenges:**
- *Eventual consistency:* Data is not immediately synchronized across all services.
- *At-least-once delivery:* Consumers must handle duplicate messages.
- *Out-of-order processing:* Network delays can cause an `OrderCancelled` event to arrive before an `OrderCreated` event.
- *Tracing & debugging:* Requires distributed tracing headers (W3C TraceContext, AWS X-Ray) to correlate asynchronous logs across independent microservices.

---

8. A Realistic Architecture: Order Creation and Background Processing

Consider a complete order placement workflow:

1. Client submits an order.

2. Spring Boot Order API validates input and business rules.
3. Order is written to PostgreSQL.
4. An `OrderPlaced` event is published to messaging infrastructure.
5. Fulfillment and email workers react to the event.

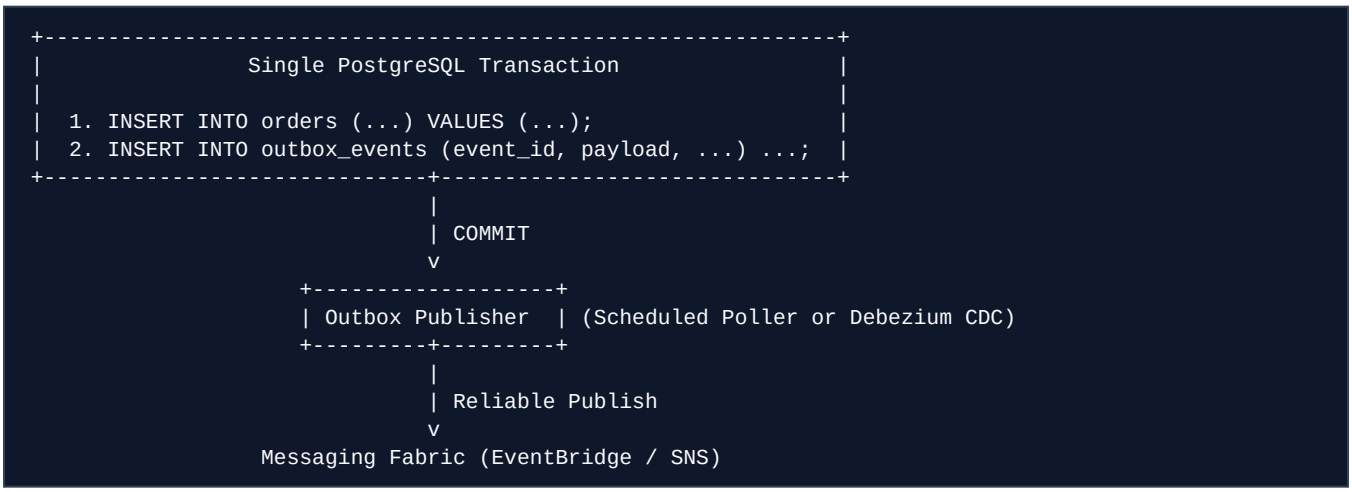
The Dual-Write Problem

What happens if step 3 succeeds (order saved in PostgreSQL), but the application crashes, network fails, or broker is unreachable during step 4?

- **Outcome A:** Order exists in the database, but downstream workers never process it (customer is charged, but no goods are shipped).
- **Outcome B:** If you reverse the order (publish event first, then write DB) and the DB transaction fails, ghost events trigger fulfillment for non-existent orders.

The Transactional Outbox Pattern

The **Transactional Outbox Pattern** solves the dual-write problem by writing both the business record and the event record into the **same relational database transaction**:



Because the outbox event and the order are committed atomically:

- If the transaction rolls back, neither exists.
- If the transaction commits, both exist durably.
- A background publisher polls the `outbox_events` table or uses Change Data Capture (CDC via PostgreSQL logical decoding), publishes to SNS/EventBridge, and marks the outbox record as `PUBLISHED`.
- If the publisher crashes after publishing but before marking the record, it retries upon restart. This produces duplicate events, which downstream idempotent consumers handle seamlessly.

Layer 2 — Practical Engineering & Workflows

9. Choose a Deployment Pattern for a Backend

When architecting a Java 21 Spring Boot API on AWS, evaluate these three proven production patterns:

Pattern A: Spring Boot on EC2 in Auto Scaling Group

- **Architecture:** `Client -> ALB -> EC2 ASG (Amazon Linux 2023) -> Spring Boot JVM`
- **Best fit:** Teams with established VM operational tooling, workloads requiring direct hardware manipulation, or legacy applications expecting local filesystems.
- **Trade-off:** High operational overhead (OS updates, AMI baking, slow ASG scale-out times of 3–5 minutes).

Pattern B: Containerized Spring Boot on ECS Fargate

- **Architecture:** `Client -> ALB -> ECS Service -> AWS Fargate Tasks (Docker)`
- **Best fit:** Standard modern backend APIs. Zero host maintenance, containerized immutability, automated rolling updates, and predictable task sizing.
- **Trade-off:** Slightly higher compute unit cost compared to reserved EC2 instances; requires container registry (ECR) management.

Pattern C: Event-Driven Processing with AWS Lambda

- **Architecture:** `EventBridge / S3 / SQS -> AWS Lambda Function (Java/Go/Node.js)`
- **Best fit:** Background asynchronous jobs, image processing, webhook ingestion, and bursty, infrequent event processing.
- **Trade-off:** Not recommended as the primary HTTP host for large Spring Boot monoliths due to cold starts; excellent for lightweight specialized workers.

---

10. Design an SQS Consumer Correctly

A robust message consumer requires an explicit, fail-safe processing lifecycle:



Configuring the Visibility Timeout

- The **Visibility Timeout** must be set to at least **6 times the expected task processing duration**, plus buffer.
- If task processing averages 10 seconds, set the visibility timeout to at least 60–90 seconds.
- If processing takes longer than anticipated, the consumer can send a heartbeat using `ChangeMessageVisibility` to extend the lease.

Dead-Letter Queues (DLQ) and Redrive Policies

- Every production queue must have an associated **Dead-Letter Queue (DLQ)**.
- Configure `maxReceiveCount` (typically 3 to 5).
- If a message fails processing `maxReceiveCount` times, SQS automatically moves it to the DLQ.
- Prevents "poison pill" messages from perpetually blocking queue processing or wasting consumer CPU.

---

11. Idempotency: The Key to Safe Retries

Because standard queues guarantee at-least-once delivery, consumers must guarantee that processing a duplicate message produces no additional side effects.

PostgreSQL Idempotency Table Schema

```
CREATE TABLE processed_events (
    event_id UUID PRIMARY KEY,
    event_type VARCHAR(64) NOT NULL,
    order_id VARCHAR(64) NOT NULL,
    processed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Atomic Deduplication Workflow in Spring Boot

```
@Transactional
public void processOrderEvent(OrderPlacedEvent event) {
    // 1. Attempt atomic insert into processed_events table
    try {
        jdbcTemplate.update(
            "INSERT INTO processed_events (event_id, event_type, order_id) VALUES (?, ?,
 ?)",
            event.getEventId(), event.getEventType(), event.getOrderId()
        );
    } catch (DuplicateKeyException e) {
        log.warn("Duplicate event detected: {}. Skipping business logic.", event.getEven
tId());
        return; // Idempotent exit; message will be acknowledged and deleted from queue
    }

    // 2. Perform business logic in the same transaction
    inventoryService.reserveStock(event.getOrderId(), event.getItems());
    orderRepository.markStatus(event.getOrderId(), OrderStatus.PROCESSING);
}
```

If the transaction succeeds, both the business modification and the idempotency record commit atomically. If the message is redelivered, the duplicate key exception safely aborts execution before external side effects occur.

---

12. Retries, Backoff, and Poison Messages

Not all errors are equal. A production consumer must categorize failures:

Transient Failures (Retryable)

- Network socket timeout to database.
- Downstream HTTP 429 (Too Many Requests / Throttling) or 503 (Service Unavailable).
- Deadlock detected during concurrent database transactions.
- **Handling:** Retry with exponential backoff and full jitter:

Formula: t_{sleep} = random(0, \min(M, B × 2^{attempt}))

Permanent Failures (Non-Retryable)

- JSON deserialization error (malformed syntax).
- Missing required fields (e.g., missing `orderId`).
- Business rule validation failure (e.g., negative payment amount).

- Unknown event version.
- **Handling:** Do **not** retry! Immediately route to DLQ or log as fatal error and acknowledge to purge from main queue. Retrying a malformed JSON payload 5 times wastes compute and delays other messages.

---

13. Lambda and SQS: Asynchronous Event Processing

When AWS Lambda consumes from SQS via an **Event Source Mapping (ESM)**:

- Lambda service internally polls SQS and invokes your function synchronously with batches of messages (e.g., batch size 10).
- If your function completes without exceptions, the Lambda service automatically deletes the batch from SQS.

```
Producer -> SQS -> Lambda Event Source Mapping (ESM) -> Lambda Handler -> Database
```

Preventing Whole-Batch Retries: Partial Batch Responses

By default, if 1 message in a batch of 10 throws an exception, Lambda considers the entire batch failed. All 10 messages become visible again and are reprocessed, causing 9 duplicates!

Solution: Enable **Report Batch Item Failures** in the ESM and return individual failed IDs:

```
public class SqsBatchHandler implements RequestHandler<SQSEvent, SQSBatchResponse> {
    @Override
    public SQSBatchResponse handleRequest(SQSEvent event, Context context) {
        List<SQSBatchResponse.BatchItemFailure> failures = new ArrayList<>();

        for (SQSEvent.SQSMessage msg : event.getRecords()) {
            try {
                processSingleMessage(msg);
            } catch (Exception e) {
                // Record only the specific message that failed
                failures.add(new SQSBatchResponse.BatchItemFailure(msg.getMessageId()));
            }
        }
        return new SQSBatchResponse(failures);
    }
}
```

Lambda deletes all successfully processed messages from SQS and leaves only the failing messages to retry according to the redrive policy.

---

14. ECS Task Roles and Least Privilege

In an ECS-based asynchronous backend, enforce strict role separation:

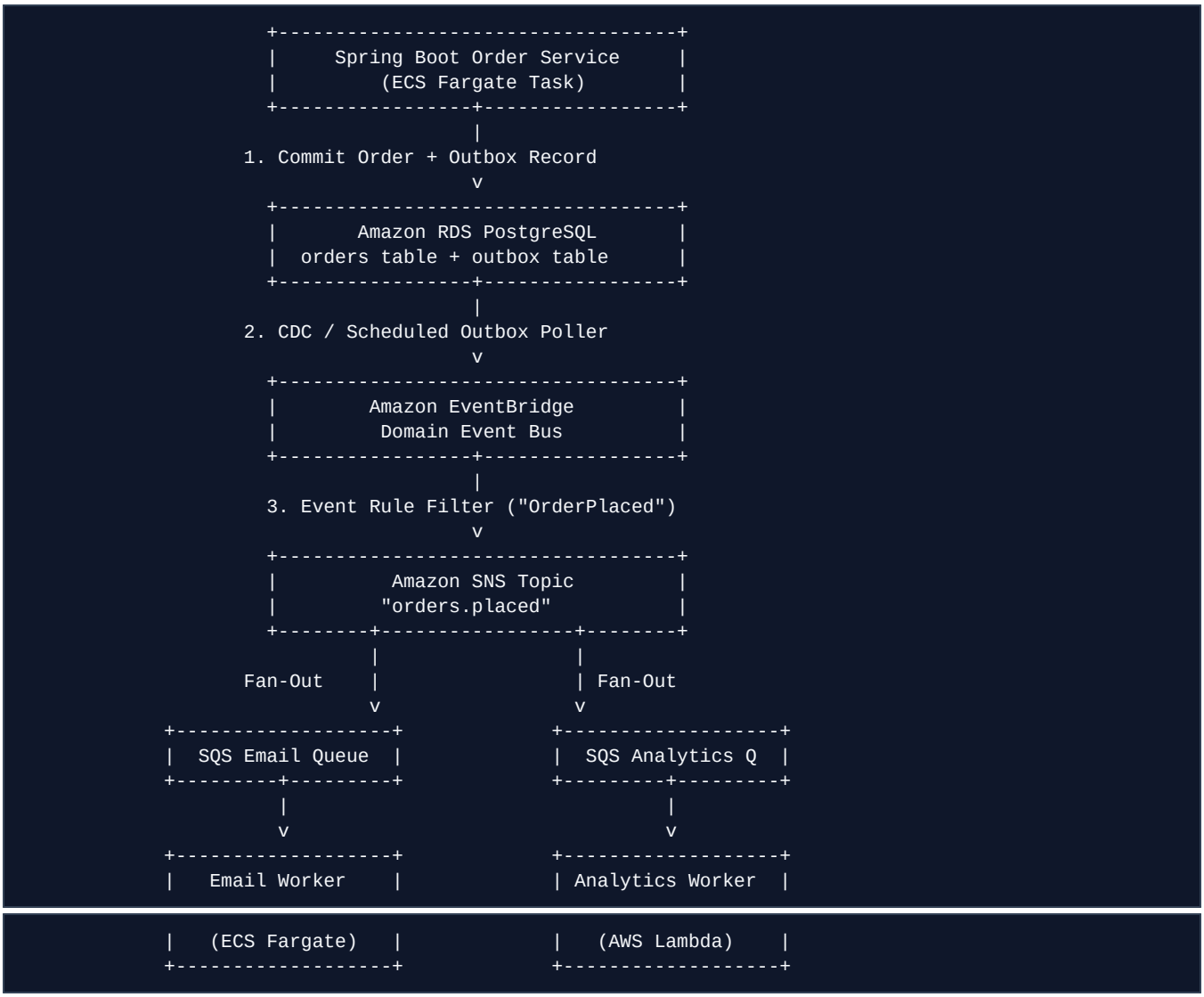


Never grant ECS tasks broad permissions like `sqs:*` or `s3:*`. Restrict ARNs to specific queue names and bucket prefixes.

---

15. Practical Design Exercise: Order Notifications Architecture

Let us construct a production-grade asynchronous order processing architecture:



Architectural Resilience Properties

1. **Email worker outage:** If the email worker process crashes, messages safely buffer in the SQS Email Queue for up to 14 days without dropping notifications or affecting the API.
2. **Analytics failure isolation:** If the analytics worker encounters a bug, it retries and eventually shunts failed records to its DLQ. The email worker and core order flow continue uninterrupted.
3. **Database safety:** Downstream consumers interact only with SQS, protecting PostgreSQL from sudden burst traffic spikes.

---

16. Hands-On Lab Without Provisioning AWS Resources

Task A: Define the Domain Event

Draft a canonical, versioned JSON domain event payload:

```
{
  "specversion": "1.0",
  "id": "f20e9e7c-2c8d-4c1b-a6bd-21f0f1b2a100",
  "source": "https://api.orders.example.com",
  "type": "com.example.orders.OrderPlaced",
  "datacontenttype": "application/json",
  "time": "2026-10-23T08:00:00Z",
  "data": {
    "orderId": "order-8931",
    "customerId": "cust-5021",
    "currency": "USD",
    "totalAmount": 149.99,
    "items": [
      { "sku": "SKU-PRO-01", "quantity": 1, "unitPrice": 149.99 }
    ]
  }
}
```

Design rationale: Stable event UUID (`id`) guarantees consumer deduplication; explicit `type` enables EventBridge rule routing; schema follows CloudEvents open standards.

Task B: Consumer Logic Pseudocode

```
def handle_sqs_message(record):
    event = json.loads(record.body)
    event_id = event["id"]

    with db.transaction():
        # Check idempotency
        if db.query("SELECT 1 FROM processed_events WHERE id = %s", event_id):
            logger.info(f"Event {event_id} already processed. Skipping.")
            return

        # Perform business operation
        send_order_email(event["data"])

        # Persist idempotency record
        db.execute("INSERT INTO processed_events (id) VALUES (%s)", event_id)

    sqs.delete_message(ReceiptHandle=record.receipt_handle)
```

Task C: Crash Simulation

If the worker crashes after `send_order_email()` but before `sqs.delete_message()`:

1. SQS visibility timeout expires.
2. Message is redelivered to another worker.
3. The new worker checks `processed_events`.
4. *Correction note:* If the email provider is external, the email might be sent twice unless an external idempotency key is passed to the email API! Always forward the `event_id` as the email provider's `Idempotency-Key` header.

Task D: DLQ & Redrive Plan

- Set `maxReceiveCount = 3`.
- Set `VisibilityTimeout = 120s`.

- Configure CloudWatch alarm on `ApproximateNumberOfMessagesVisible` on DLQ > 0.
- Diagnostic runbook: inspect DLQ message attributes, fix parsing bug in codebase, deploy patch, and initiate redrive via SQS StartMessageMoveTask API.

Task E: AWS Service Selection

1. **Buffer image-processing jobs:** Amazon SQS Standard Queue.
2. **Notify multiple independent systems of published events:** Amazon SNS Topic fanning out to multiple SQS queues.
3. **Route events based on payload metadata:** Amazon EventBridge.
4. **Execute lightweight code on S3 file upload:** AWS Lambda.
5. **Run a high-throughput, continuously listening Spring Boot REST API:** Amazon ECS with AWS Fargate.

Layer 3 — Production Engineering & Interview Mastery

17. Production Incident Simulations

Incident A: Queue Depth Grows Continuously

Symptoms: CloudWatch metric `ApproximateNumberOfMessagesVisible` on the orders queue rises steadily from 50 to 100,000 messages.

- **Investigation:**

1. Inspect consumer task count: Are ECS worker tasks running and healthy?
2. Inspect consumer logs: Are tasks crashing or encountering database connection timeouts?
3. Inspect consumer CPU and memory: Are workers CPU-throttled?
4. Inspect visibility timeout: Is the timeout too short, causing tasks to time out and re-enqueue messages in a loop?
5. Check downstream database: Is PostgreSQL saturated with lock contention?

- **Resolution:**
- If workers are healthy but overwhelmed, scale out ECS task count.
- If downstream database is saturated, throttle consumer concurrency and optimize queries.
- Fix any recurring transient exception that is resetting the visibility timeout.

Incident B: Customer Receives Duplicate Order Confirmation Emails

Symptoms: Multiple customer support tickets report receiving 3 to 5 identical order confirmation emails for a single checkout.

- **Root Cause:**

1. The worker process sends the email via external API (takes 2,500 ms).
2. SQS visibility timeout was misconfigured to 2,000 ms.
3. SQS marks the message visible while the first worker is still finalizing the network call.
4. Worker 2 receives the message and sends another email.

- **Resolution:**

1. Increase SQS queue `VisibilityTimeout` to 30 seconds.
2. Pass `orderId` or `eventId` as the idempotency key to the email provider API.
3. Record the idempotency entry in PostgreSQL before or during the send operation.

Incident C: Lambda Invocations Fail Under Spike, Crashing Database

Symptoms: Marketing sends a push notification. SQS receives 50,000 messages in 10 seconds. Lambda scales to 1,000 concurrent executions. RDS PostgreSQL crashes with `FATAL: remaining connection slots are reserved for non-replication superuser connections`.

- **Root Cause:** SQS triggered Lambda without concurrency throttling. 1,000 concurrent Lambda instances opened 1,000 simultaneous connections to RDS, completely exhausting PostgreSQL `max_connections` (150).
- **Resolution:**

1. Set **Reserved Concurrency** on the Lambda function to 20 to limit simultaneous database connections.
2. Implement **Amazon RDS Proxy** between Lambda and PostgreSQL to pool and reuse connections.
3. Configure SQS ESM maximum concurrency controls.

Incident D: ECS Deployment Replaces Tasks, Causing Total API Outage

Symptoms: ECS service initiates a rolling update (`orders-api:2.0.0`). Within 2 minutes, the ALB returns HTTP 503 Service Unavailable for 100% of incoming traffic.

- **Root Cause:** The new task definition specified an environment variable that failed decryption, causing Spring Boot to terminate immediately on startup (`CrashLoopBackOff`). The ECS service had `minimumHealthyPercent` configured to `0`, allowing ECS to terminate all running 1.0.0 tasks before verifying that 2.0.0 tasks were healthy.
- **Resolution:**

1. Set `minimumHealthyPercent: 100` and `maximumPercent: 200` to guarantee that existing capacity remains active until new tasks pass ALB target health checks.
2. Roll back immediately to task definition revision `orders-api:1.4.0`.
3. Fix the missing KMS decryption permission on the Task Role.

Incident E: A Poison-Pill Message Blocks Queue Processing

Symptoms: SQS message throughput drops to near-zero. CPU on worker tasks is 100%.

- **Root Cause:** A client sent an order containing an unexpected null character that causes an unhandled `NullPointerException` in the JSON parser. The queue had no DLQ configured (`maxReceiveCount` was disabled), so the message was retried infinitely every time the visibility timeout expired.
- **Resolution:**

1. Attach a Dead-Letter Queue with `maxReceiveCount = 3`.
2. Add robust input validation and wrap message parsing in a `try-catch` block that discards or dead-letters malformed payloads.

Incident F: Database Commits Order, but Downstream Consumers Never Hear About It

Symptoms: Orders are stored in PostgreSQL, but warehouse fulfillment is never triggered.

- **Root Cause:** Dual-write bug. The application executed `orderRepository.save(order)` and then attempted `snsClient.publish(...)`. During a network blip to AWS, the SNS call threw an exception. The DB transaction was already committed, leaving the order orphaned without an event.
- **Resolution:**

1. Implement the **Transactional Outbox Pattern**: write the event to an `outbox` table in the same DB transaction.
2. Run a resilient CDC or poller service that guarantees at-least-once publication to SNS/EventBridge.

---

18. Seven Production Engineering Principles

1. **Design for At-Least-Once Delivery:** Always assume messages can be duplicated, delayed, or reordered. Idempotency is non-negotiable.
2. **Never Perform Dual Writes Directly:** Never update a database and publish to a network broker as two independent, uncoordinated steps. Use the Transactional Outbox pattern.
3. **Queue Age Is More Important Than Queue Depth:** A queue with 1,000 messages processing in 500 ms is healthy; a queue with 10 messages that have been stuck for 4 hours is an active production incident. Monitor CloudWatch `ApproximateAgeOfOldestMessage`.
4. **Isolate Workloads with Fan-Out:** Never let slow or non-critical consumers (e.g., analytics) share a queue with mission-critical consumers (e.g., payment/fulfillment).
5. **Enforce Backpressure and Concurrency Limits:** Protect stateful databases from serverless compute spikes using connection pools, RDS Proxy, and concurrency caps.
6. **Treat Event Contracts as APIs:** Events are public schemas. Version event payloads (`v1`, `v2`) and ensure backward compatibility before modifying fields.
7. **Make Dead-Letter Queues Actionable:** Never let DLQs become data graveyards. Configure automated alarms, build replay tooling (`aws sqs start-message-move-task`), and inspect failure headers.

---

19. FAANG-Style Interview Questions and Answers

Q1. What is Amazon ECS?

Amazon Elastic Container Service is a fully managed container orchestration service that executes, scales, and manages Docker/OCI containers across EC2 instances or AWS Fargate.

Q2. What is the difference between ECS on EC2 and ECS on Fargate?

With ECS on EC2, you manage the underlying virtual machines, host OS, and cluster capacity. With AWS Fargate, AWS provisions and manages the serverless compute infrastructure, allowing you to pay strictly for the CPU and memory allocated per task.

Q3. What is an ECS task definition?

A declarative JSON blueprint defining one or more containers, their Docker image repositories, CPU/memory limits, port mappings, environment variables, storage mounts, and IAM roles.

Q4. What is the difference between an ECS Task Role and an Task Execution Role?

The Task Role provides AWS credentials to the application container code (e.g., S3/DynamoDB access). The Task Execution Role is used by the ECS container agent to perform infrastructure tasks like pulling private ECR images and writing logs to CloudWatch.

Q5. What is AWS Lambda?

A serverless event-driven compute platform that executes stateless code in response to events, automatically managing server provisioning, OS patching, and concurrency scaling.

Q6. When would you choose ECS Fargate over AWS Lambda?

Choose ECS Fargate for long-running processes, complex Spring Boot monoliths with heavy startup times, consistent high-throughput traffic, or services requiring raw TCP listening sockets.

Q7. What is Amazon SQS?

A distributed, fully managed message queuing service designed to decouple and scale microservices, distributed systems, and serverless applications via asynchronous message buffering.

Q8. What is the difference between SQS Standard and SQS FIFO queues?

Standard queues provide best-effort ordering and at-least-once delivery with nearly unlimited throughput. FIFO queues preserve strict order within message groups, guarantee deduplication, but have lower default throughput limits.

Q9. What is an SQS Visibility Timeout?

The duration during which a message is hidden from other workers after being retrieved by a consumer. If the consumer fails to delete the message before the timeout expires, it becomes visible again for other consumers to process.

Q10. What is a Dead-Letter Queue (DLQ)?

A secondary queue attached to a source queue or Lambda function that isolates messages that repeatedly fail processing after a configured number of retries (`maxReceiveCount`).

Q11. What is Amazon SNS?

A fully managed pub/sub messaging service that coordinates the delivery of messages from publishers to multiple subscribing endpoints (fan-out).

Q12. What is Amazon EventBridge?

A serverless event bus service that ingests events from applications, SaaS integrations, and AWS services, routing them to targets based on content-matching rules.

Q13. When should you use SQS instead of SNS?

Use SQS when a single consumer needs to pull and buffer work to process at its own pace. Use SNS when a publisher needs to broadcast an event immediately to multiple independent subscribers.

Q14. What is the difference between Amazon SNS and Amazon EventBridge?

SNS is optimized for high-throughput, low-latency pub/sub fan-out (primarily to SQS or webhooks). EventBridge is an enterprise event bus supporting complex JSON content filtering, schema registries, and native SaaS integrations.

Q15. What is the dual-write problem?

The architectural hazard where an application attempts to write to a local database and publish to a message broker without distributed transaction coordination, leading to data inconsistency if one operation fails.

Q16. How does the Transactional Outbox pattern prevent lost events?

By writing the business entity and an outbox event record within the same ACID database transaction. A separate, resilient process reads the outbox table and publishes events to the broker with guaranteed delivery.

Q17. Does the Transactional Outbox pattern guarantee exactly-once message delivery?

No. If the outbox publisher crashes after publishing but before updating the outbox table, it will publish the message again upon restart. It guarantees at-least-once delivery; consumers must be idempotent.

Q18. How do you implement idempotency in a distributed consumer?

By storing unique event IDs in a relational database table with a unique primary key constraint inside the same database transaction that updates business state. If a duplicate arrives, the unique constraint violation safely skips processing.

Q19. Why can scaling consumer concurrency degrade overall system throughput?

Excessive concurrency can overwhelm downstream shared dependencies (such as PostgreSQL connection limits or third-party API rate limits), causing timeouts, retry storms, and connection exhaustion.

Q20. How would you design a scalable, fault-tolerant order notification system?

Use Spring Boot with the Transactional Outbox pattern on Amazon RDS PostgreSQL; emit events to Amazon EventBridge; fan out via Amazon SNS to dedicated SQS queues for Email, Fulfillment, and Analytics; and process queues with idempotent workers running on ECS Fargate with DLQs and CloudWatch alarms.

---

20. Mastery Challenge

Design an asynchronous order-processing system with these requirements:

- Orders are stored in PostgreSQL.
- Email and fulfillment process independently.
- A temporary downstream outage must not immediately fail order creation.
- Duplicate event delivery must not create duplicate fulfillment.

- Failed messages must be diagnosable and recoverable.
- The system must expose processing delay and failure metrics.

Complete these tasks:

1. **Architecture diagram:** Client $\to$ ALB $\to$ ECS Orders API $\to$ PostgreSQL (Orders + Outbox tables) $\to$ Outbox Poller $\to$ EventBridge $\to$ SNS Topic $\to$ SQS Queues (Email & Fulfillment) $\to$ Workers.
2. **Compute choice for workers:** ECS Fargate for high-throughput fulfillment; Lambda for bursty email notifications.
3. **Messaging workflow:** EventBridge for domain event routing; SNS for fan-out; SQS for consumer isolation and buffering.
4. **Preventing lost publication:** Transactional Outbox commits order and outbox record atomically in PostgreSQL.
5. **Enforcing idempotency:** Workers insert event UUID into `processed_events` table before executing side effects; skip on duplicate key violation.
6. **Retry & DLQ parameters:** `VisibilityTimeout = 90s`, `maxReceiveCount = 4`, with dedicated DLQ and alarm.
7. **Crash recovery scenario:** If worker crashes post-side-effect, idempotency check prevents duplicate fulfillment on redelivery; external API calls must pass idempotency keys.
8. **Replaying DLQ messages:** Fix bug, verify schema, initiate SQS message move task back to primary queue.
9. **Backlog metrics:** CloudWatch `ApproximateAgeOfOldestMessage` (latency signal) and `ApproximateNumberOfMessagesVisible` (queue depth).
10. **Protecting PostgreSQL from traffic surges:** Use RDS Proxy, set maximum task counts on ECS consumer services, and tune HikariCP connection pool limits.

Self-assessment

Score each item from 0 to 2:

Skill	Score
ECS, Fargate, and Lambda distinctions	/2
SQS, SNS, and EventBridge separation	/2
Queue delivery and visibility timeout	/2
Idempotency and safe retries	/2
Transactional outbox pattern	/2
Dead-letter handling and redrive	/2
IAM task roles and least privilege	/2
Production troubleshooting & backpressure	/2

Target: At least 13/16, with no zero in idempotency, retries, or the transactional outbox.

Chapter Completion Protocol

Chapter: AWS Chunk 5 of 6

Save as: `content/aws/05-aws-containers-serverless-messaging-event-driven-architecture.md`

Do not generate a PDF yet. After all six AWS chapters are saved and verified, generate one complete AWS handbook:

```
docs/aws/BEH_AWS_COMPLETE_HANDBOOK.pdf
```

After verifying all source chapters and the final PDF, use the intended Git commit message:

```
AWS done
```

Next chapter: AWS Chunk 6 of 6 — Production Security, Observability, CI/CD, Reliability, Cost Engineering & Final Architecture Capstone.

Part VI: Production Security, Observability, CI/CD, Cost Engineering & Final Architecture Capstone

Chunk 6 of 6 · Backend Engineering Handbook · Cloud Engineering Track

AWS — Chunk 6 of 6: Production Security, Observability, CI/CD, Reliability, Cost Engineering & Final Architecture Capstone

Backend Engineering Tools & Infrastructure Handbook · Cloud Engineering Track

Difficulty: Beginner-friendly foundations → production engineering → FAANG-level interview mastery

Prerequisites: AWS Chunks 1–5, basic Java/Spring Boot, HTTP, PostgreSQL, Docker, Git

Source file:
`content/aws/06-aws-production-security-observability-cicd-cost-architecture-capstone.md`

Learning Objectives

By the end of this chapter, you should be able to:

- Secure AWS workloads using IAM roles, least privilege, encryption, and managed secrets.
- Explain the responsibilities of AWS CloudTrail, CloudWatch, and AWS X-Ray.
- Design a CI/CD pipeline that builds, tests, scans, and deploys a backend application.
- Deploy safely, detect regressions, and roll back failed releases.
- Design for availability, recoverability, and measurable service objectives (SLIs, SLOs, SLAs).
- Understand AWS cost drivers and prevent accidental spending.
- Diagnose common security, performance, queueing, and availability incidents.
- Explain a complete production-minded AWS architecture in an SDE-1 or backend interview.

Layer 1 — Fundamentals & Mental Models

1. Production Engineering: The Real Objective

Running an application in the cloud is not the same as operating it reliably.

A production system must answer five fundamental questions:

1. **Security:** Who can access this resource, and what can they do?
2. **Reliability:** What happens when a component fails?

3. **Observability:** How do we discover, trace, and diagnose problems?
4. **Delivery:** How can we release changes continuously without unnecessary risk?
5. **Cost:** Are we paying for resources that deliver genuine business value?

Imagine an Orders API built with Spring Boot. It accepts an order, writes it to PostgreSQL, stores an invoice in S3, and publishes an event to SQS.

The API might work perfectly in a local Docker container. Production introduces completely different questions:

- What if an attacker obtains an AWS credential?
- What if PostgreSQL becomes temporarily unavailable or exhausts its connection pool?
- What if SQS accumulates a million unprocessed messages?
- What if a deployment introduces a 10% error rate on checkout?
- What if an autoscaling policy creates too many instances and saturates the database?
- What if nobody notices the errors until angry customers post on social media?

Production engineering is the discipline of anticipating, detecting, and recovering from these failures.

---

2. AWS Security: Identity First

AWS security is built around identities, permissions, resource policies, network boundaries, and encryption.

2.1 IAM Users, Roles, and Policies

An **IAM policy** defines allowed or denied actions on AWS resources.

An **IAM role** is an identity that a trusted principal can assume to obtain temporary, automatically rotating credentials.

A workload should normally use a role rather than a long-lived access key stored in application configuration:

- A Spring Boot service running on EC2 obtains permissions through its attached **instance profile**.
- A service running as an ECS task uses its **Task Role** to access application resources (S3, SQS, Secrets Manager).
- A Lambda function uses its **Execution Role**.

The application does not need to know the underlying secret access key. The AWS SDK obtains temporary credentials automatically through the default credential provider chain.

2.2 Least Privilege

Least privilege means granting only the permissions strictly necessary for a task.

Suppose an order service must write invoice files to one S3 bucket. It does not need unrestricted access to every S3 bucket in the AWS account.

A simplified illustrative identity policy looks like this:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::example-invoice-bucket/invoices/*"
    }
  ]
}
```

This example allows object uploads strictly to the specified bucket and key prefix.

[!IMPORTANT]

Authorization Precedence: An IAM allow statement does not automatically guarantee access. Explicit denies, permissions boundaries, Service Control Policies (SCPs), resource policies, and session policies all take precedence. Any explicit `Deny` unconditionally blocks access.

2.3 Temporary Credentials versus Static Credentials

Always Prefer:

- IAM roles for EC2 workloads via instance profiles.
- ECS task roles for container application code.
- Lambda execution roles for serverless functions.
- Federated, short-lived OIDC credentials for CI/CD (GitHub Actions $\to$ AWS via OIDC).
- AWS IAM Identity Center (formerly AWS SSO) for human engineers.

Strictly Avoid:

- Hardcoding access keys (`AKIA...`) in Java or configuration source code.
- Committing AWS credentials to Git repositories.
- Permanent administrator keys for deployment pipelines.
- Passing credentials through Docker build arguments, which bakes them into image layers.

If a credential leaks, simply removing it from the latest Git commit is useless (it remains in Git history). Treat it as compromised: deactivate it immediately, audit CloudTrail for unauthorized API calls, and rotate affected credentials.

---

3. Encryption and Key Management

Encryption protects data in two major states:

State	Concrete Example	Common AWS Capability
At Rest	PostgreSQL storage, S3 objects, EBS volumes	AWS KMS (SSE-S3, SSE-KMS, EBS Encryption)
In Transit	REST API calls, database connections	TLS 1.3, ACM (AWS Certificate Manager)

AWS Key Management Service (KMS) manages cryptographic keys and controls their use through permissions and policies:

- **AWS Managed Keys:** Managed automatically by AWS (e.g., `aws/s3`, `aws/rds`). Free to use, but key policies cannot be modified.
- **Customer Managed Keys (CMKs):** Created and controlled by your organization. Allows fine-grained key policies, cross-account sharing, and automatic annual rotation.

Encryption is not a substitute for authorization. An encrypted S3 object can still be read if an overly permissive IAM policy grants both `s3:GetObject` and `kms:Decrypt` to an unauthorized principal.

---

4. Secrets Manager versus Systems Manager Parameter Store

Applications need database passwords, API tokens, and other sensitive configuration:

AWS Secrets Manager

- Specifically designed for confidential credentials (passwords, OAuth tokens).
- Built-in automatic rotation integrations for Amazon RDS, Aurora, and DocumentDB.
- Generates random credentials and updates database and application secrets simultaneously.
- Cost: ~$0.40 per secret per month + API request costs.

AWS Systems Manager Parameter Store

- Stores hierarchical configuration parameters (`/orders-api/prod/db-url`).
- Supports plain text (`String`, `StringList`) and encrypted values (`SecureString` backed by KMS).
- Cost: Standard parameters are free; Advanced parameters incur small costs.

Dimension	AWS Secrets Manager	SSM Parameter Store
Primary Use	Database passwords, API keys, credentials	App configuration, flags, license keys
Automatic Rotation	Native built-in Lambda rotation	Manual or custom Lambda rotation
Cross-Account Access	Native resource policies on secrets	More complex cross-account setup
Pricing	$0.40 / secret / month	Free (Standard tier)

Neither service eliminates the need for least privilege. A workload should be allowed to retrieve only the specific secrets it requires.

---

5. Observability: Metrics, Logs, and Traces

Observability is the ability to understand the internal health and behavior of a system from the external signals it produces:

1. Metrics — What is happening?

Numerical measurements aggregated over time intervals: request rate, 5xx error rate, p95/p99 latency, CPU utilization, heap memory usage, active database connections, and SQS queue depth.

2. Logs — What happened?

Structured event records detailing specific occurrences: an order failed schema validation, a database connection acquisition timed out, or a deployment initiated.

3. Traces — Where did the request spend its time?

A distributed request timeline following a transaction across microservices, database queries, SQS queues, and external third-party HTTP calls.

```
Request Timeline (Total: 2,050 ms)
+-----------------------------------------------------------+
| ALB Routing (20 ms)                                       |
+--+--------------------------------------------------------+
   | Spring Boot Handler (30 ms)                            |
   +----+---------------------------------------------------+
        | PostgreSQL: INSERT INTO orders (1,850 ms) <-- BOTTLENECK!
        +----+----------------------------------------------+
             | SQS: SendMessage (150 ms)                    |
             +----------------------------------------------+
```

5.1 Amazon CloudWatch

Central hub for operational monitoring:

- **CloudWatch Metrics:** Ingests infrastructure and custom application metrics (via Micrometer in Spring Boot).
- **CloudWatch Logs:** Collects structured application logs into Log Groups and Log Streams.
- **CloudWatch Alarms:** Triggers automated remediation or pager alerts when metrics cross thresholds.
- **CloudWatch Dashboards:** Real-time visual monitoring for engineering teams.

5.2 AWS CloudTrail

CloudTrail records **management and API activity** across your AWS account:

- Records *who* made *what* API call, *when*, from *which IP*, on *which resource*.
- Essential for security auditing, compliance, and investigating configuration drift.
- **Rule of thumb:** CloudWatch monitors application & system performance; CloudTrail audits AWS API interactions.

5.3 AWS X-Ray and OpenTelemetry

- **OpenTelemetry (OTel):** The vendor-neutral industry standard for instrumenting metrics, logs, and distributed traces.
- **AWS X-Ray:** Ingests distributed traces, generates service maps, and highlights latency bottlenecks across microservices.

---

6. CI/CD: From a Commit to Production

Continuous Integration and Continuous Delivery (CI/CD) automates the path from code check-in to verified production execution:

```
Developer pushes commit to main
        |
        v
[CI Stage: Quality & Safety Checks]
  - Compile Java 21 bytecode
  - Execute JUnit 5 unit tests
  - Execute integration tests (Testcontainers)
  - Static code analysis & linting
  - Dependency vulnerability scan (Trivy / Snyk)
        |
        v
[Build & Package Stage]
  - Build immutable Docker image
  - Scan image layers for CVEs
  - Tag with Git commit SHA: orders-api:7f8a12c
  - Push image to Amazon ECR
        |
        v
[Deployment Stage: Staging]
  - Update ECS task definition or Helm release
  - Execute automated smoke tests
  - Verify database migrations (Flyway / Liquibase)
        |
        v
[Production Rollout: Controlled Deployment]
  - Rolling update or Canary deployment (10% -> 50% -> 100%)
  - Continuous health check evaluation
  - Automated rollback on alarm trigger
```

[!TIP]

Never deploy a mutable tag like `:latest` to production. Always use immutable identifiers such as the short Git commit SHA (`:7f8a12c`) to guarantee traceability and reproducible rollbacks.

---

7. Reliability: Design for Failure

A production-grade cloud backend assumes that every server, disk, network link, and dependency will eventually fail:

- **Redundancy:** Multi-AZ deployment across at least two Availability Zones.
- **Health Checks:** Separate liveness (is JVM hung?) from readiness (is DB pool initialized?).
- **Timeouts:** Enforce strict client and server timeouts on every network call.
- **Bounded Retries with Jitter:** Prevent retry storms by combining exponential backoff with full jitter:

Formula: $t = random(0, \min(M, B \times 2^{attempt}))$

- **Circuit Breakers:** Cut off calls to struggling downstream services (using Resilience4j) to prevent cascading failures.
- **Bulkheads:** Isolate resource pools (separate thread pools for payment vs. recommendation APIs).
- **Graceful Shutdown:** Complete in-flight requests cleanly within a draining window before termination.

---

8. SLI, SLO, and SLA

Engineering teams use three distinct terms to define and measure service reliability:

- **SLI (Service Level Indicator):** A quantifiable metric measuring real-time performance.

> **Formula:** SLI = (Count of Successful Requests (HTTP status < 500 and latency < 500ms) / Total Valid Requests) × 100

- **SLO (Service Level Objective):** The internal target reliability agreed upon by the engineering team (e.g., 99.9% successful requests over any rolling 30-day window).
- **SLA (Service Level Agreement):** The legal or commercial commitment to customers, which carries financial or contractual penalties if breached (e.g., 99.5% uptime).
- **Error Budget:** The allowed margin of unreliability:

> **Formula:** Error Budget = 100% - SLO = 100% - 99.9% = 0.1%

Over 1,000,000 requests, the error budget allows at most 1,000 failed requests before halting non-critical releases.

---

9. Cost Engineering: Understand the Bill

AWS costs are driven by:

- **Compute:** Running EC2 instances, Fargate vCPU/RAM per second, Lambda invocation count and duration.
- **Storage:** S3 GB-months, EBS provisioned volume size and IOPS (`gp3`), EFS storage.
- **Networking & Data Transfer:** NAT Gateway data processing fees ($0.045/GB), cross-AZ traffic, egress to the internet.
- **Databases:** RDS instance hours, provisioned storage, Multi-AZ licensing, backup retention beyond free limits.
- **Observability:** CloudWatch log ingestion ($0.50/GB), custom metrics, and metric alarms.

Cost engineering means making spending visible through cost-allocation tags, configuring AWS Budgets, eliminating orphaned EBS volumes or idle NAT Gateways, and right-sizing workloads based on utilization telemetry.

Layer 2 — Practical Engineering & Workflows

10. Workload Identity in Practice

10.1 Local Development

Use AWS CLI profiles or temporary SSO session tokens:

```
aws configure list
aws sts get-caller-identity
```

10.2 Production Workload

Configure the Spring Boot application to use the AWS SDK default credential provider chain:

- On EC2: Automatically retrieves credentials from the EC2 instance profile (`http://169.254.169.254/latest/meta-data/iam/security-credentials/`).
- On ECS: Automatically retrieves credentials from the ECS Task Role (`AWS_CONTAINER_CREDENTIALS_RELATIVE_URI`).
- On Lambda: Uses the Lambda execution role environment variables.

---

11. Retrieving Secrets Safely in Spring Boot

In `application.yml`, reference environment variables injected by the container runtime:

```
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
    hikari:
      maximum-pool-size: 15
      connection-timeout: 3000
```

On ECS Fargate, inject secrets directly from AWS Secrets Manager in the task definition:

```
{
  "name": "DB_PASSWORD",
  "valueFrom": "arn:aws:secretsmanager:ap-south-1:123456789012:secret:prod/orders/db-xyz
:password::"
}
```

The ECS agent securely resolves the secret at container launch and sets it as an environment variable without writing it to disk or committing it to source control.

---

12. CloudWatch: Inspect and Monitor a Workload

12.1 Inspect Log Groups via AWS CLI

```
aws logs describe-log-groups \
  --region ap-south-1
```

```
aws logs describe-log-streams \
  --log-group-name "/aws/ecs/orders-api-prod" \
  --order-by LastEventTime \
  --descending \
  --limit 5 \
  --region ap-south-1
```

12.2 Practical Golden Signals Monitoring Set

Golden Signal	Metric Name	Recommended Alarm Threshold
Latency	`TargetResponseTime` (ALB)	p95 > 500 ms for 3 consecutive 1-minute evaluation periods
Traffic	`RequestCount` (ALB)	Anomaly detection: $\pm 3\sigma$ deviation from historical trend
Errors	`HTTPCode_Target_5XX_Count`	5xx error rate > 1% of total requests over 5 minutes
Saturation	`CPUUtilization` (ECS / RDS)	Sustained CPU > 80% for 10 minutes
Queue Backlog	`ApproximateAgeOfOldestMessage`	Age > 300 seconds (5 minutes)
Database Pool	`DatabaseConnections` (RDS)	Active connections > 80% of `max_connections`

12.3 Structured JSON Logging Example

```
{
  "timestamp": "2026-10-23T10:30:00Z",
  "level": "ERROR",
  "service": "orders-api",
  "environment": "production",
  "traceId": "1-6718d8f0-1a2b3c4d5e6f7a8b9c0d1e2f",
  "requestId": "req-98234-a1b2",
  "event": "order_persistence_failed",
  "errorType": "DatabaseTimeoutException",
  "message": "Connection to PostgreSQL primary timed out after 3000ms"
}
```

---

13. CloudTrail: Investigate an Unauthorized Configuration Change

When a critical security group rule or IAM policy changes unexpectedly:

```
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=AuthorizeSecurityGroupIngres
s \
  --max-results 10 \
  --region ap-south-1
```

Examine the returned CloudTrail record:

- `userIdentity`: Identify the IAM user, assumed role, or access key used.
- `sourceIPAddress`: Check if the call originated from corporate VPN or an unknown IP.
- `requestParameters`: Determine exact ports and CIDRs opened (e.g., `0.0.0.0/0` on port 5432).
- `responseElements`: Verify whether the API call succeeded.

---

14. CI/CD Pipeline Implementation (GitHub Actions Example)

```
name: Deploy Orders API to ECS Fargate

on:
  push:
    branches: [ main ]

permissions:
  id-token: write   # Required for AWS OIDC federation
  contents: read

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Source Code
        uses: actions/checkout@v4

      - name: Set up JDK 21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'corretto'

      - name: Build & Test with Gradle/Maven
        run: ./mvnw clean verify

      - name: Configure AWS Credentials via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-actions-deploy-role
          aws-region: ap-south-1

      - name: Log in to Amazon ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2
```

```
      - name: Build, Tag, and Push Docker Image
        env:
          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
          IMAGE_TAG: ${{ github.sha }}
        run: |
          docker build -t $ECR_REGISTRY/orders-api:$IMAGE_TAG .
          docker push $ECR_REGISTRY/orders-api:$IMAGE_TAG

      - name: Deploy to ECS Fargate
        run: |
          aws ecs update-service \
            --cluster prod-cluster \
            --service orders-service \
            --force-new-deployment \
            --region ap-south-1
```

15. Infrastructure as Code: Terraform Hygiene

Maintain strict IaC discipline:

```
terraform fmt -check
terraform validate
terraform plan -out=tfplan.binary
```

Before applying:

- Review the plan output: verify zero unintended resource destructions.
- Enforce remote state locking using an S3 bucket with Amazon DynamoDB state locking.
- Maintain strict environment separation: isolate `prod` and `staging` state files in separate buckets and accounts.

---

16. Budgets, Billing Visibility, and Cleanup

- **AWS Budgets:** Configure automated alarms when forecasted monthly spending exceeds target budgets.
- **Cost Allocation Tags:** Tag all resources with `Environment` (`prod`/`staging`), `Service` (`orders-api`), and `Owner` (`backend-team`).
- **Clean Up Non-Production Resources:**
- Terminate unused EC2 instances and idle NAT Gateways.
- Delete unattached EBS volumes (`aws ec2 describe-volumes --filters Name=status,Values=available`).
- Delete obsolete manual RDS snapshots.
- Set S3 Lifecycle rules to expire temporary staging artifacts after 7 days.

---

17. Pre-Production Release Readiness Checklist

Before approving any backend service for production release, verify every item:

Security

- [ ] Workload uses IAM roles (instance profile / task role) rather than static access keys.
- [ ] IAM permissions strictly adhere to least privilege (specific resource ARNs and actions).
- [ ] Secrets are retrieved dynamically via Secrets Manager / SSM; none are committed to Git.
- [ ] Security groups restrict traffic to known sources (no open 0.0.0.0/0 on databases or SSH).
- [ ] S3 buckets have Block Public Access enabled; data is encrypted at rest and in transit (TLS 1.3).

Reliability

- [ ] Health checks correctly separate liveness probes from readiness probes.
- [ ] Network timeouts and bounded retries with jitter are configured on all dependencies.
- [ ] Multi-AZ redundancy is enabled for compute (ASG/ECS across 2+ AZs) and database (RDS Multi-AZ).
- [ ] Automated daily database backups and continuous point-in-time recovery (PITR) are active.
- [ ] Tested rollback and graceful shutdown procedures are documented in runbooks.

Observability

- [ ] Logs are output in structured JSON format with distributed correlation IDs (`traceId`).
- [ ] Golden signals (Latency, Traffic, Errors, Saturation) are tracked in CloudWatch.

- [ ] CloudWatch alarms are configured with actionable thresholds and PagerDuty/SNS alerts.
- [ ] Application deployment version is recorded alongside metrics and logs.

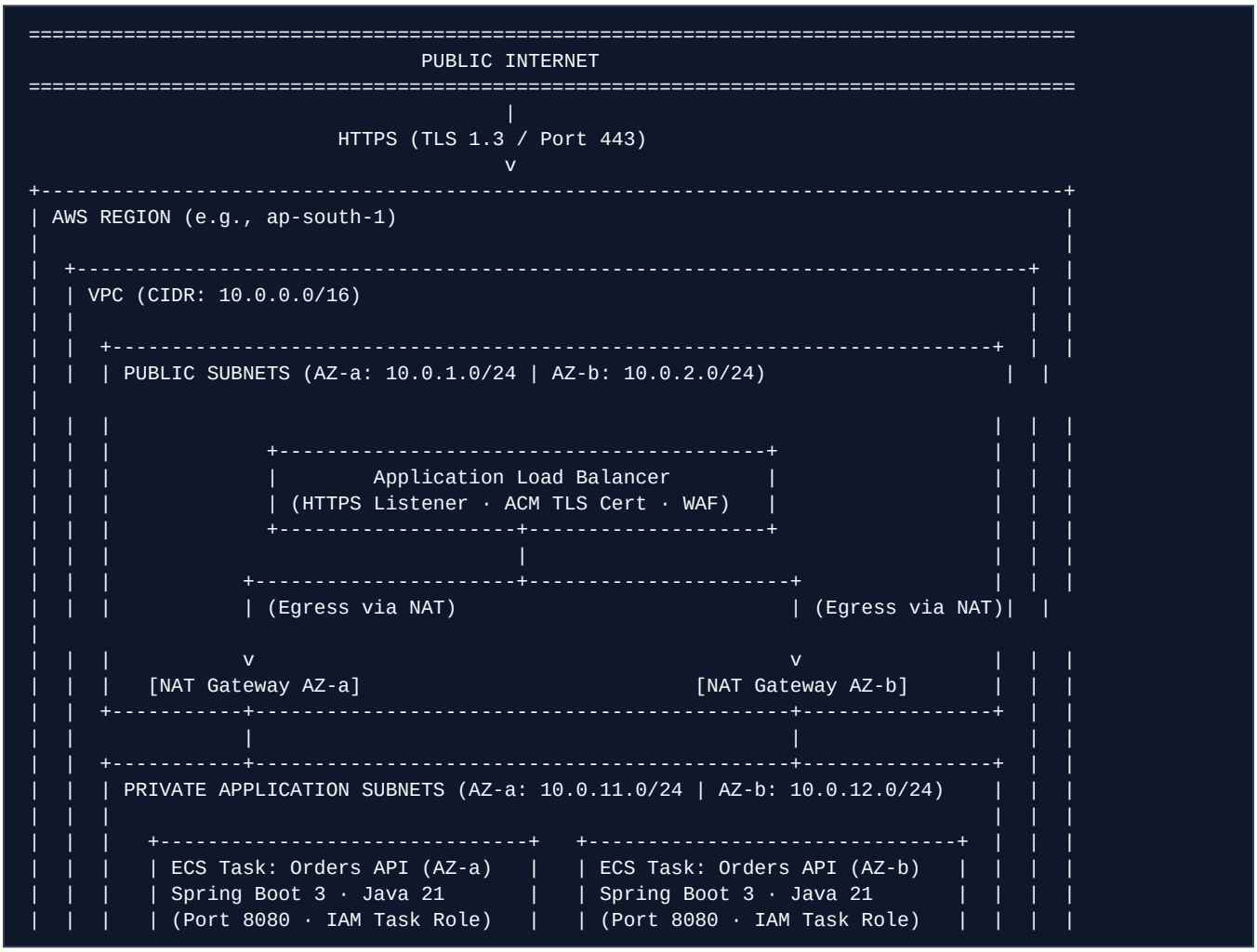
Delivery and Cost

- [ ] CI/CD pipeline enforces unit tests, integration tests, and container vulnerability scanning.
- [ ] Container images use immutable tags (Git commit SHA); `:latest` is prohibited.
- [ ] Infrastructure changes are reviewed via `terraform plan` before application.
- [ ] Cost allocation tags are applied to all resources; AWS Budget alerts are active.

Layer 3 — Production Engineering & Interview Mastery

18. Production Architecture Capstone

Below is the comprehensive, end-to-end production architecture for the e-commerce Orders API, integrating all 6 chunks of the handbook:




PRIVATE DATABASE SUBNETS (AZ-a: 10.0.21.0/24 | AZ-b: 10.0.22.0/24)
TCP 5432
Amazon RDS PostgreSQL (Multi-AZ)
Primary (AZ-a) <---sync---> Standby
(orders table + outbox_events table)
Transactional Outbox Publisher
Amazon EventBridge Event Bus
(Rule: "OrderPlaced")
SQS: Order Fulfillment
(Visibility: 90s, DLQ)
SQS: Customer Email
(Visibility: 60s, DLQ)
Fulfillment Worker
(ECS Fargate Task)
Email Notification
(AWS Lambda Function)
Upload PDF Invoices (SSE-KMS)
Amazon S3 Bucket
(Invoices & Media)
CROSS-CUTTING PLATFORM SERVICES
- AWS Secrets Manager: DB passwords and third-party API keys
- AWS KMS: Customer Managed Keys (CMK) for EBS, S3, RDS, Secrets
- Amazon CloudWatch: Container Insights, Metrics, Alarms, Logs
- AWS CloudTrail: Regional management event auditing
- AWS X-Ray: Distributed tracing and dependency map

19. Five Production Incident Simulations

Incident A — Leaked AWS IAM Credentials

- **Detection:** An alert triggers from AWS GuardDuty: `UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.OutsideAWS`. Unfamiliar EC2 instances spin up in an unapproved region (`us-west-2`).
- **Response:**

1. Immediately deactivate or delete the compromised access key via AWS CLI:

```
aws iam update-access-key --access-key-id AKIA... --status Inactive
```

2. Query CloudTrail to identify all API actions performed by the compromised key in the last 24 hours.
3. Terminate all unauthorized instances, security groups, and IAM roles created by the attacker.
4. Force password reset and MFA re-enrollment for the affected IAM user.
5. Audit source repositories using GitGuardian/TruffleHog to locate how the credential leaked.

Incident B — S3 Bucket Public Exposure

- **Detection:** Security review finds an S3 bucket policy containing `"Principal": "*"` and `"Action": "s3:GetObject"`.
- **Response:**

1. Immediately apply S3 Block Public Access at the account and bucket level:

```
aws s3api put-public-access-block --bucket invoice-bucket --public-access-block-configuration
"BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true"
```

2. Query CloudTrail S3 Data Events to audit all IP addresses that accessed objects during the exposure window.
3. Validate if any personally identifiable information (PII) was exfiltrated; report according to GDPR/data breach protocols.
4. Implement an AWS Config rule (`s3-bucket-public-read-prohibited`) to automatically remediate future policy misconfigurations.

Incident C — Post-Deployment 5xx Spike

- **Detection:** ALB 5xx rate jumps from 0.01% to 14% immediately after an ECS deployment.
- **Response:**

1. Check CloudWatch deployment metrics and correlate with the exact time of the ECS service update.
2. Inspect target group health: 50% of targets are unhealthy due to failing `/actuator/health` checks.
3. Immediately roll back to the previous stable task definition:

```
aws ecs update-service --cluster prod --service orders --task-definition orders-api:42
```

4. Once healthy capacity is restored, examine container logs from the failed tasks: discover that a required environment variable (`SPRING_PROFILES_ACTIVE`) was omitted in the new revision.

Incident D — SQS Backlog Growth and Consumer Starvation

- **Detection:** PagerDuty alert: `ApproximateAgeOfOldestMessage` on `orders-fulfillment-queue` exceeds 15 minutes.
- **Response:**

1. Check consumer task metrics: ECS fulfillment workers are running at 100% CPU.

2. Check worker logs: workers are executing slow, unindexed database queries on each message.
3. Add the missing database index in PostgreSQL (`CREATE INDEX CONCURRENTLY idx_orders_customer...`).
4. Scale out ECS fulfillment worker task count from 2 to 10 to drain the 50,000 message backlog.
5. Once the backlog is drained, scale back to baseline task count and verify DLQ message count is 0.

Incident E — Unexpected AWS Bill Spike ($10,000 Jump)

- **Detection:** AWS Budgets email alert: forecasted monthly spending exceeds target budget by 300%.
- **Response:**

1. Open AWS Cost Explorer and group costs by Service and Usage Type.
2. Identify the culprit: **NAT Gateway Data Processing fees** in `ap-south-1` accounted for $8,500.
3. Root cause: An ECS worker was downloading 50 GB of machine learning models from an S3 bucket every 10 minutes over the public internet through the NAT Gateway.
4. Fix: Create a **Gateway VPC Endpoint for S3** in the VPC. Traffic to S3 now routes privately over AWS internal networking at **$0.00 data transfer fee**, completely eliminating the NAT Gateway charge.

---

20. FAANG-Level Interview Questions & Answers

1. Why should an application use an IAM role instead of hardcoded access keys?

IAM roles eliminate long-lived credentials from source code and configuration files. AWS automatically provisions, distributes, and rotates temporary credentials (valid for 1–6 hours) via the instance profile or container metadata service, dramatically reducing the blast radius of credential leaks.

2. What is the difference between authentication and authorization?

Authentication (AuthN) verifies the identity of a principal ("who are you?"). Authorization (AuthZ) verifies what actions that authenticated principal is permitted to execute on a target resource ("what are you allowed to do?").

3. What is least privilege?

The foundational security principle of granting an identity strictly the minimum set of permissions required to complete its intended tasks, scoped to specific resource ARNs, actions, and conditions, and nothing more.

4. What is the difference between Secrets Manager and SSM Parameter Store?

Secrets Manager is tailored for sensitive credentials with built-in automatic rotation for databases, cross-account sharing, and per-secret monthly pricing. SSM Parameter Store is a general-purpose configuration store supporting free plain text strings and KMS-encrypted `SecureString` parameters.

5. What does AWS KMS provide?

AWS KMS provides centralized, hardware security module (HSM)-backed management and generation of cryptographic keys, controlling encryption and decryption operations across AWS services with audit logging in CloudTrail.

6. Does encryption at rest eliminate the need for authorization?

No. Encryption protects data confidentiality if underlying storage media is stolen or accessed out-of-band. However, if an unauthorized principal has IAM permissions that permit `kms:Decrypt` and `s3:GetObject`, the system will happily decrypt and return the data. Authorization controls access; encryption enforces confidentiality.

7. What is the difference between CloudWatch and CloudTrail?

CloudWatch collects, visualizes, and alerts on operational telemetry (metrics, logs, traces, system performance). CloudTrail audits management and API operations across the AWS account (who called which API, when, and from where).

8. Metrics versus logs versus traces: how do they differ?

Metrics show aggregate numerical trends over time (what is happening). Logs provide detailed, timestamped event records of specific occurrences (what happened). Traces map the end-to-end journey of a single request across multiple distributed services and databases (where time was spent).

9. Why monitor p95 and p99 latency instead of average latency?

Averages hide severe tail latency issues. If 95% of users experience 50 ms latency, but 5% experience 10 seconds, the average might look acceptable (500 ms), but 1 in 20 customers experiences a broken, frustrating checkout.

10. What is an SLO and how is it used?

A Service Level Objective is an agreed-upon internal target for an indicator (e.g., "99.9% of order requests return HTTP 200 in < 500 ms"). It establishes an Error Budget that guides engineering trade-offs between rapid feature deployment and reliability investments.

11. Why can aggressive retries worsen a database outage?

When a database is struggling under high load, thousands of client instances retrying immediately create a "thundering herd" or retry storm. This multiplies incoming traffic exponentially, driving database CPU to 100% and preventing it from recovering. Retries must be bounded, with exponential backoff, jitter, and circuit breakers.

12. What is graceful shutdown?

A process termination sequence where an application stops accepting new requests from the load balancer, allows currently in-flight requests to complete cleanly within a timeout window, flushes buffers, closes database connection pools, and exits cleanly with return code 0.

13. Why must CI/CD pipelines deploy immutable artifacts?

Deploying an immutable container image tagged with a Git commit SHA ensures that the exact binary that passed automated unit, integration, and security tests in staging is what runs in production. It eliminates configuration drift and guarantees reproducible rollbacks.

14. What is the difference between blue-green and canary deployments?

Blue-green maintains two identical production environments (Blue and Green), routing 100% of traffic from one to the other via a load balancer switch. Canary deployment gradually routes a small percentage of real user traffic (e.g., 5%, then 25%, then 100%) to the new version while actively monitoring error rates.

15. What is the Transactional Outbox pattern?

An architectural pattern that resolves the dual-write problem by saving a business entity (e.g., order) and an event record (e.g., `OrderPlaced`) into the same database transaction. A separate publisher reads committed outbox events and publishes them to the message broker, ensuring reliable event publication without data loss.

16. Why is consumer idempotency mandatory in SQS architectures?

Amazon SQS Standard queues guarantee at-least-once delivery, meaning network retries or expired visibility timeouts can cause the same message to be delivered multiple times. Idempotent consumers guarantee that processing duplicate messages produces no duplicate side effects.

17. Multi-AZ versus Multi-Region: what are the operational trade-offs?

Multi-AZ provides high availability and automatic failover within a single geographic region with sub-millisecond synchronous replication. Multi-Region provides disaster recovery against regional catastrophic failure, but requires complex asynchronous data replication, handling of eventual consistency, split-brain mitigation, and significantly higher networking costs.

18. How do you reduce AWS costs without impacting reliability?

1. Replace NAT Gateway S3/DynamoDB traffic with free Gateway VPC Endpoints.
2. Right-size over-provisioned EC2/RDS instances using CloudWatch telemetry.
3. Purchase Savings Plans or Reserved Instances for predictable baseline compute.
4. Set S3 Lifecycle rules to transition objects to Glacier and expire old logs.
5. Delete unattached EBS volumes, obsolete snapshots, and unused load balancers.

19. How would you design an AWS backend to handle a 10x traffic spike?

1. Front the architecture with an Application Load Balancer and CloudFront CDN for static caching.
2. Deploy containerized stateless Spring Boot tasks on ECS Fargate with target tracking autoscaling based on ALB request count.
3. Protect the database using Amazon RDS Proxy to pool and multiplex database connections.
4. Offload heavy processing (invoices, emails, analytics) asynchronously via SQS queues and background workers.
5. Implement Redis (ElastiCache) caching for read-heavy database queries.

20. How would you investigate a suspected unauthorized AWS API change?

1. Open AWS CloudTrail Event History and search by resource name or suspicious event name.
2. Identify the calling IAM ARN, source IP, user agent, and timestamp.
3. Check AWS GuardDuty and VPC Flow Logs for associated network anomalies.
4. Immediately revoke permissions or quarantine the compromised IAM identity.
5. Revert unauthorized infrastructure modifications using Terraform.

21. Final Hands-On Capstone

To achieve complete mastery, implement the following local-first architecture on your workstation:

- **Phase 1 — Application Core:** Implement a Spring Boot 3 REST API in Java 21 with `/api/v1/orders`. Configure PostgreSQL with Flyway migrations. Implement the Transactional Outbox pattern: write to `orders` and `outbox_events` atomically.
- **Phase 2 — Asynchronous Processing:** Implement a background poller or worker that reads `outbox_events`, simulates dispatching to SQS, and writes to a local `processed_events` idempotency table.
- **Phase 3 — Observability:** Add Spring Boot Actuator with Micrometer. Export structured JSON logs containing `traceId`, `orderId`, and `timestamp`. Configure health probes on `/actuator/health/readiness` and `/actuator/health/liveness`.
- **Phase 4 — Containerization & CI/CD:** Write a multi-stage Dockerfile using Eclipse Temurin 21. Create a GitHub Actions workflow that compiles code, executes tests, builds the container image, and tags it with `${{ github.sha }}`.
- **Phase 5 — Production Readiness Review:** Complete the 20-point pre-production readiness checklist and document the failure modes for database connection loss, queue consumer crashes, and rolling updates.

---

22. AWS Final Readiness Assessment

Test your understanding with these core architectural scenarios:

Question 1

A production Spring Boot API running on ECS needs to upload invoice PDFs to Amazon S3. How should credentials be supplied?

- (A) Hardcode an IAM User access key in `application.yml`.
- **(B) Attach an IAM Task Role to the ECS task definition with a policy allowing `s3:PutObject` on the specific invoice bucket.** *(Correct)*
- (C) Use the root account credentials stored in an environment variable.
- (D) Assign the permission to the underlying EC2 host's instance profile.

Explanation: Workloads should use IAM Task Roles to obtain temporary credentials scoped to the specific container task, adhering to least privilege.

Question 2

Which AWS service is specifically designed for auditing management API calls, such as who modified a security group or deleted an RDS instance?

- **(A) AWS CloudTrail** *(Correct)*
- (B) Amazon CloudWatch Metrics
- (C) AWS X-Ray
- (D) Amazon GuardDuty

Explanation: CloudTrail records AWS account activity and API calls for governance and compliance. CloudWatch monitors application and infrastructure operational telemetry.

Question 3

An SQS consumer queue depth is climbing rapidly. What is the first operational step?

- (A) Immediately increase consumer worker count to 100 tasks.
- (B) Delete all messages in the queue to clear the backlog.
- **(C) Inspect CloudWatch metrics for message age, consumer error logs, processing duration, and downstream database saturation.** *(Correct)*
- (D) Reduce the SQS visibility timeout to 1 second.

Explanation: You must identify the root bottleneck first. If the downstream database is saturated, adding more consumers will cause total database failure.

Question 4

A rolling deployment is initiated. Immediately, ALB returns HTTP 500 errors on 15% of requests. What is the safest immediate action?

- **(A) Stop the deployment and trigger an immediate automated rollback to the previous known-good task revision.** *(Correct)*
- (B) SSH into the running containers and inspect heap dumps.
- (C) Increase the request timeout on the Application Load Balancer to 60 seconds.
- (D) Wait 2 hours to see if errors clear on their own.

Explanation: Contain customer impact immediately by rolling back to the known-good version, then investigate logs and root causes safely in a staging environment.

Question 5

Why is an AWS Budget alert not a guarantee against unexpected cloud charges?

- (A) AWS Budgets only tracks compute instances, not storage.
- **(B) Budgets are passive monitoring and notification tools; they do not automatically terminate running resources or stop network transfers, and billing telemetry has a propagation delay.** *(Correct)*
- (C) Budgets only trigger after an invoice is officially billed at the end of the month.
- (D) Budgets require manual approval before sending emails.

Explanation: Budgets send notifications when thresholds are crossed; they do not enforce a hard spending cap or automatically shut down infrastructure unless configured with custom automation scripts.

Question 6

Why is the Transactional Outbox pattern preferred over publishing directly to SNS/SQS from the API handler?

- (A) It allows SQS to execute SQL queries directly.
- (B) It makes SQS messages strictly FIFO.
- **(C) It guarantees atomic consistency between the database update and event publication, eliminating the dual-write problem where database commits succeed but network message publishing fails.** *(Correct)*
- (D) It eliminates the need for message consumers to be idempotent.

Explanation: The outbox pattern commits the entity and event within the same ACID database transaction, ensuring no events are lost if the application crashes before publishing.

Final Mastery Checklist

You have mastered the AWS Cloud Engineering track when you can:

- [ ] Explain workload IAM roles (Instance Profiles, ECS Task Roles, Lambda Execution Roles) and least privilege.
- [ ] Design secure secret retrieval and rotation using Secrets Manager and SSM Parameter Store.
- [ ] Distinguish CloudWatch, CloudTrail, and AWS X-Ray responsibilities and use cases.
- [ ] Define actionable monitoring alarms using Golden Signals (Latency, Traffic, Errors, Saturation).
- [ ] Design an automated, secure CI/CD pipeline using GitHub Actions, OIDC, and Amazon ECR.
- [ ] Implement bounded retries with jitter, circuit breakers, and graceful shutdown.
- [ ] Calculate and defend SLIs, SLOs, and Error Budgets for production APIs.
- [ ] Diagnose and resolve SQS queue backlogs without overloading downstream databases.
- [ ] Architect resilient Multi-AZ systems with automated failover and verified point-in-time recovery.
- [ ] Analyze AWS billing drivers and optimize costs using Gateway VPC Endpoints and right-sizing.
- [ ] Articulate and defend a complete, production-grade cloud backend architecture in a technical interview.

Handbook Completion Protocol

1. Save this chapter as `content/aws/06-aws-production-security-observability-cicd-cost-architecture-capstone.md`.
2. Verify that all six AWS source files exist and that the earlier chapters are preserved.
3. Check that the chapters collectively cover fundamentals, practical workflows, and production engineering/interview mastery.
4. Generate **one complete AWS handbook PDF only after all six source files have been saved and verified**: `docs/aws/BEH_AWS_COMPLETE_HANDBOOK.pdf`
5. Verify that the PDF includes all six chapters in order, with readable code blocks, tables, diagrams, and page breaks.
6. Verify the README progress and final PDF path.
7. Commit the verified AWS sources and final PDF using the exact commit message: `AWS done`